



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:
Разработка средства обнаружения аномалий в
Kubernetes-кластере на основе нейронных сетей

Студент ИУ8-121
(Группа)

(Подпись, дата)

Г.Н. Давыдов
(И.О.Фамилия)

Руководитель ВКР

(Подпись, дата)

О.В. Гудков
(И.О.Фамилия)

Консультант по исследовательской части

(Подпись, дата)

О.В. Гудков
(И.О.Фамилия)

Консультант по конструкторской части

(Подпись, дата)

О.В. Гудков
(И.О.Фамилия)

Консультант по технологической части

(Подпись, дата)

О.В. Гудков
(И.О.Фамилия)

Консультант по организационно-
экономической части

(Подпись, дата)

Н.Е. Беняев
(И.О.Фамилия)

Консультант по организационно-
правовому обеспечению ИБ

(Подпись, дата)

Е.А. Тарапанова
(И.О.Фамилия)

Нормоконтролер

(Подпись, дата)

Т.Е. Завадская
(И.О.Фамилия)

2026 г.

РЕФЕРАТ

Отчёт содержит 141 стр., 42 рис., 12 табл., 54 источн., 1 прил.

Ключевые слова: ОБНАРУЖЕНИЕ АНОМАЛИЙ, KUBERNETES, АУДИТ ЛОГИ, НЕЙРОННЫЕ СЕТИ, МАШИННОЕ ОБУЧЕНИЕ, КЛАССИФИКАТОР, ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ, КОНТЕЙНЕРНЫЕ СРЕДЫ.

Целью выпускной квалификационной работы является повышение безопасности Kubernetes-кластера путём внедрения средства на основе нейронных сетей для обнаружения аномального поведения.

В процессе выполнения работы рассмотрены существующие подходы к обнаружению аномалий в информационных системах, а также методы анализа audit-логов Kubernetes-кластерах. Проведён обзор современных методов машинного обучения и глубокого обучения, применяемых для задач выявления отклоняющегося поведения в последовательностях событий.

В ходе работы был выполнен анализ исходных данных, представляющих собой журналы аудита Kubernetes-кластера. На основе полученных данных были разработаны, обучены и исследованы модели обнаружения аномалий с использованием нейронных сетей, включая автоэнкодеры и их рекуррентные модификации. Особое внимание уделено модели автоэнкодера на основе LSTM (LSTM-AE), способной учитывать временную структуру последовательностей.

Оценка качества моделей проводилась с использованием метрик классификации, при этом в качестве основной метрики была выбрана площадь под PR-кривой (AUC_{PR}), как наиболее информативная в условиях несбалансированных данных. В результате экспериментов установлено, что модель LSTM-AE демонстрирует наилучшие показатели качества: значение AUC_{PR} составило 0,9996, precision — 0,931, recall — 1,00 и F1-мера — 0,964.

Разработано программное обеспечение, реализующее процесс предварительной обработки audit-логов, формирования признаков, обучения и применения модели для выявления аномалий в Kubernetes-кластере. Предложенное средство обеспечивает автоматизированный анализ событий и позволяет своевременно обнаруживать отклонения от нормального поведения в контейнерной среде.

В рамках работы также рассмотрены вопросы практической применимости разработанного средства, включая оценку его экономической целесообразности и соответствие требованиям действующих нормативных правовых актов Российской Федерации.

СОДЕРЖАНИЕ

РЕФЕРАТ	6
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	11
СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ	13
ВВЕДЕНИЕ	14
1 ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ	17
1.1 Обзор и анализ научной литературы в области обнаружения аномалий	17
1.2 Статические методы обнаружения аномалий	18
1.2.1 Сигнатурный анализ	19
1.2.2 Правило-ориентированные методы	20
1.2.3 Пороговые методы	20
1.2.4 Методы и черных белых списков	21
1.2.5 Методы, основанные на марковских процессах	21
1.3 Алгоритмы машинного обучения	22
1.3.1 Метод К-ближайших соседей	23
1.3.2 Алгоритм Isolation Forest	24
1.3.3 Алгоритм Random Cut Forest	25
1.4 Методы обнаружения аномалий на основе нейронных сетей	27
1.4.1 Рекуррентные нейронные сети (RNN)	27
1.4.2 Сети с долгой краткосрочной памятью (LSTM)	29
1.4.3 Автоэнкодеры	32
1.5 Архитектура Kubernetes	33
1.5.1 Компоненты плоскости управления	34
1.5.2 Компоненты рабочих узлов	36
1.5.3 Взаимодействие компонентов кластера	36
1.6 Механизм аудита и структура audit-логов Kubernetes	36
1.7 Выводы	37
2 КОНСТРУКТОРСКАЯ ЧАСТЬ	39
2.1 Обзор и анализ существующих подходов и методов в области обнаружения аномалий	39
2.1.1 GATAKU	39
2.1.2 KubAnomaly	42
2.1.3 DeepLog	43

2.2	Моделирование последовательностного подхода обнаружения аномалий	47
2.3	Выбор используемого метода по обнаружению аномалий в контейнерных средах	48
2.4	Архитектура средства обнаружения аномалий	50
2.5	Формулирование математической постановки задачи	53
2.5.1	Исходные данные	54
2.5.2	Выбор параметров нейронной сети	57
2.5.3	Выбор структуры нейронной сети	57
2.6	Обобщённая формулировка задачи	58
2.7	Методы решения задачи	59
2.8	Предобработка и кодирование событий audit-логов	60
2.9	Формирование обучающей, валидационной и тестовой выборок	64
2.9.1	Экспериментальная среда и кластер Kubernetes	64
2.9.2	Генерация событий audit-логов с использованием агентов	66
2.9.3	Формирование обучающей и валидационной выборок	67
2.9.4	Формирование тестовой выборки и внедрение аномалий	67
2.9.5	Анализ полученного датасета	68
2.10	Решения математической постановки задачи	70
2.10.1	Разработка и обучение моделей нейронных сетей для обнаружения аномалий	70
2.10.2	Сравнительный анализ моделей и выбор оптимальной архитектуры	82
2.11	Выводы	83
3	ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ	85
3.1	Применяемые технологии	85
3.2	Реализация процессов	86
3.3	Обучение модели НС	88
3.4	Обнаружение аномалий	90
3.5	Реализация программного средства	91
3.6	Взаимодействие модулей	94
3.7	Тестирование программного средства	95
3.8	Выводы	99
4	ОРГАНИЗАЦИОННО-ЭКОНОМИЧЕСКАЯ ЧАСТЬ	101
4.1	Анализ рынка и описание условий	102

4.2	Определение этапов работ	103
4.3	Расчёт трудоёмкости и определение этапов работ	105
4.4	Линейный график технологического процесса	107
4.5	Оптимизация ресурсов	109
4.6	Сетевой график работ	110
4.7	Анализ затрат на изготовление продукта	111
4.7.1	Затраты на заработную плату	111
4.7.2	Затраты на материалы и оборудование	114
4.7.3	Затраты на организацию рабочих мест и накладные расходы	115
4.7.4	Анализ расходов	117
4.8	Планирование цены и прогнозирование прибыли	117
4.9	Выводы	119
5	ОРГАНИЗАЦИОННО-ПРАВОВОЕ ОБЕСПЕЧЕНИЕ ИБ	121
5.1	Конституция Российской Федерации	122
5.2	Федеральный закон № 149-ФЗ «Об информации, информационных технологиях и о защите информации»	123
5.3	Федеральный закон № 152-ФЗ «О персональных данных»	124
5.4	Федеральный закон № 187-ФЗ «О безопасности критической информационной инфраструктуры Российской Федерации»	126
5.5	Уголовный кодекс Российской Федерации	127
5.6	Постановление Правительства Российской Федерации № 1119	128
5.7	Приказ ФСТЭК России № 21	129
5.8	Приказ ФСТЭК России № 117	131
5.9	Выводы	132
	ЗАКЛЮЧЕНИЕ	134
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	136
	ПРИЛОЖЕНИЕ А Код программы	141

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем документе применяют следующие термины с соответствующими определениями:

Автокодировщик (autoencoder) — нейронная сеть, обучаемая восстанавливать входные данные через компактное латентное представление, применяемая для извлечения признаков и обнаружения аномалий.

Аномалия — отклонение наблюдаемого поведения объекта, процесса или системы от ожидаемого нормального состояния.

Аудит-лог (audit log) — журнал событий безопасности, содержащий сведения о действиях пользователей, сервисных аккаунтов и компонентов системы при обращении к API Kubernetes.

Валидационная выборка — часть набора данных, используемая для контроля процесса обучения модели и настройки её гиперпараметров.

Выборка (dataset) — подмножество данных, используемое для обучения, валидации или тестирования модели машинного обучения.

Кластер Kubernetes — совокупность узлов и управляющих компонентов, обеспечивающих развертывание, масштабирование и выполнение контейнеризированных приложений.

Контейнер (container) — изолированная среда выполнения приложения, включающая исполняемый код, библиотеки и зависимости и использующая ядро хост-системы.

Контейнеризация (containerization) — технология упаковки и запуска приложений в контейнерах, обеспечивающая переносимость и независимость от инфраструктуры.

Контроллер (controller) — компонент плоскости управления Kubernetes, отслеживающий текущее состояние объектов кластера и приводящий его к заданному состоянию.

Машинное обучение (machine learning) — раздел искусственного интеллекта, в котором модели обучаются на данных для выявления закономерностей, прогнозирования и классификации.

Нейронная сеть (neural network) — метод машинного обучения, основанный на системе взаимосвязанных искусственных нейронов и применяемый для аппроксимации сложных зависимостей в данных.

Обучающая выборка — часть набора данных, используемая для настройки

параметров модели в процессе обучения.

Оркестрация — процесс автоматизации развертывания, масштабирования и управления контейнеризованными приложениями с учётом зависимостей и отказоустойчивости.

Плоскость управления (control plane) — набор компонентов Kubernetes, отвечающих за управление состоянием кластера, планирование задач и координацию работы узлов.

Под (pod) — минимальная единица развертывания в Kubernetes, содержащая один или несколько контейнеров, совместно использующих ресурсы и сетевой контекст.

Признак (feature) — измеримая характеристика объекта или события, используемая при обучении и работе модели машинного обучения.

Пространство имён (namespace) — логическая область в Kubernetes, применяемая для изоляции и группировки ресурсов внутри кластера.

Сервисный аккаунт (service account) — учётная запись, используемая приложениями и компонентами системы для доступа к API и ресурсам Kubernetes.

Тестовая выборка — часть набора данных, не используемая при обучении модели и предназначенная для итоговой оценки качества её работы.

Функция потерь (loss function) — математическая функция, определяющая степень расхождения между предсказаниями модели и истинными значениями.

Эпоха обучения — один полный проход алгоритма обучения по всей обучающей выборке.

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

В настоящем документе применяют следующие сокращения и обозначения:

ВКР — Выпускная квалификационная работа

ИБ — Информационная безопасность

ИИ — Искусственный интеллект

ИС — Информационная система

МО — Машинное обучение

НС — Нейронная сеть

ПО — Программное обеспечение

ФЗ — Федеральный закон

ФСТЭК — Федеральная служба по техническому и экспортному контролю

AD — Anomaly Detection

Adam — Adaptive Moment Estimation

AE — Autoencoder

AI — Artificial Intelligence

API — Application Programming Interface

AUC — Area Under Curve

CPU — Central Processing Unit

GNN — Graph Neural Network

GPU — Graphics Processing Unit

HTTP — HyperText Transfer Protocol

HTTPS — HyperText Transfer Protocol Secure

IF — Isolation Forest

IP — Internet Protocol

JSON — JavaScript Object Notation

K8s — Kubernetes

LSTM — Long Short-Term Memory

MSE — Mean Squared Error

PR-AUC — Precision-Recall Area Under Curve

RCF — Random Cut Forest

RNN — Recurrent Neural Network

UTC — Coordinated Universal Time

ВВЕДЕНИЕ

Актуальность исследования обусловлена широким распространением облачных технологий, контейнеризации и микросервисной архитектуры в современных информационных системах. Переход от монолитных приложений к распределённым сервисам позволил повысить масштабируемость и отказоустойчивость программных решений, однако одновременно привёл к усложнению процессов администрирования и обеспечения безопасности [1].

В настоящее время контейнеризация и оркестрация контейнеров являются базовыми технологиями при построении корпоративных и облачных систем. Одним из наиболее распространённых инструментов управления контейнеризированными приложениями выступает платформа Kubernetes, обеспечивающая автоматическое развертывание, масштабирование и координацию контейнеров в распределённой инфраструктуре. Широкое применение Kubernetes в промышленной эксплуатации подтверждает его значимость для современных информационных систем [2]. Вместе с тем динамическая структура Kubernetes-кластера создаёт дополнительные риски в области информационной безопасности.

Одним из наиболее информативных источников данных для анализа безопасности Kubernetes-кластера являются audit-логи, формируемые компонентом API-сервера. Указанные журналы содержат сведения о запросах к объектам кластера, включая тип операции, пользователя, целевой ресурс, код ответа и иные параметры запроса. Анализ таких данных позволяет выявлять нетипичные сценарии поведения, которые могут указывать на наличие атак, нарушений политики безопасности или ошибок конфигурации [3].

Однако использование audit-логов затрудняется высокой интенсивностью их формирования, значительным объёмом данных и сложностью выявления скрытых зависимостей между событиями. Традиционные сигнатурные и правило-ориентированные методы обнаружения аномалий недостаточно эффективны в условиях изменяющегося характера атак и разнообразия легитимного поведения пользователей. Кроме того, audit-логи представляют собой последовательности событий, в которых важную роль играет не только

факт отдельного обращения, но и порядок, контекст и временные зависимости между событиями. В связи с этим актуальным является применение методов машинного обучения и нейронных сетей, способных автоматически выявлять аномальные последовательности и скрытые закономерности в данных.

Таким образом, актуальность настоящей работы определяется следующими факторами:

- 1) ростом числа атак на облачные и контейнерные инфраструктуры, включая Kubernetes-кластеры [4];
- 2) высокой интенсивностью формирования audit-логов и сложностью их ручного анализа в распределённых системах;
- 3) недостаточной эффективностью традиционных сигнатурных и правило-ориентированных методов при выявлении новых и нетипичных сценариев атак;
- 4) необходимостью разработки автоматизированных средств обнаружения аномалий, учитывающих временную структуру и последовательный характер audit-событий в Kubernetes-кластере.

Целью выпускной квалификационной работы является повышение безопасности Kubernetes-кластера путём внедрения средства на основе нейронных сетей для обнаружения аномального поведения.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) провести обзор и анализ научной литературы в области обнаружения аномалий в контейнерных средах;
- 2) рассмотреть и сравнить существующие подходы и методы обнаружения аномалий в контейнерных средах на основе нейронных сетей;
- 3) выполнить моделирование одного из подходов обнаружения аномалий;
- 4) разработать архитектуру средства обнаружения аномалий в Kubernetes-кластере;
- 5) сформулировать математическую постановку задачи
- 6) сформировать или получить набор данных для обучения модели нейронной сети;
- 7) построить и обучить модель нейронной сети;
- 8) разработать программное обеспечение на основе выбранной модели НС;
- 9) обоснование экономической целесообразности разработанного средства;

10) правовое обоснование выполнения выпускной квалификационной работы.

1 ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ

1.1 Обзор и анализ научной литературы в области обнаружения аномалий

В современных условиях платформа Kubernetes фактически стала промышленным стандартом для развёртывания, масштабирования и управления контейнеризованными приложениями. Вместе с такими преимуществами, как гибкость, масштабируемость и отказоустойчивость, использование Kubernetes сопровождается усложнением процессов обеспечения информационной безопасности.

Сложность задачи обнаружения аномалий в контейнерных средах определяется многомерностью и разнородностью анализируемых данных. К таким данным относятся журналы аудита, метрики производительности, сетевой трафик и поведенческие характеристики контейнеров. Одновременно к системам обнаружения предъявляются строгие требования, связанные с вычислительными ограничениями, необходимостью работы в режиме, близком к реальному времени, и минимизацией числа ложных срабатываний. В работах, посвящённых безопасности контейнерных сред, выделяются ключевые механизмы обеспечения информационной безопасности, реализуемые в Kubernetes [2]. Эти механизмы охватывают различные уровни — от контроля доступа к API и управления правами до изоляции подов и защиты конфиденциальных данных. Однако даже их комплексное использование не гарантирует обнаружения ранее неизвестных атак и отклонений в поведении системы.

Целью систем обнаружения аномалий в Kubernetes-кластерах является выявление отклонений от нормального поведения, которые могут свидетельствовать о нарушениях политики безопасности, попытках несанкционированного доступа, злоупотреблении ресурсами или наличии вредоносной активности. Ручной анализ подобных событий затруднён из-за объёма и скорости поступления данных, что обуславливает необходимость применения автоматизированных методов анализа.

В существующих исследованиях и прикладных решениях можно выделить несколько основных подходов к обнаружению аномалий в контейнерных средах:

- статистические и правило-ориентированные методы, основанные на заранее заданных сигнатурах и эвристиках;

- методы, основанные на алгоритмах машинного обучения;
- методы, основанные на нейронных сетях и глубоком обучении.

Каждый из указанных подходов обладает своими преимуществами и ограничениями. Традиционные методы характеризуются высокой интерпретируемостью и низкими вычислительными затратами, однако плохо адаптируются к новым типам атак. Методы машинного обучения позволяют выявлять скрытые зависимости в данных, но требуют предварительной подготовки признаков и настройки моделей. Нейросетевые подходы, в свою очередь, демонстрируют высокую эффективность при анализе сложных и временных зависимостей, однако предъявляют повышенные требования к объёму данных и вычислительным ресурсам.

В рамках данного раздела будет проведён обзор и анализ научной литературы и прикладных исследований в области обнаружения аномалий в контейнерных средах. В частности, будут рассмотрены статические методы обнаружения аномалий, методы на основе алгоритмов машинного обучения, а также подходы, использующие нейронные сети.

1.2 Статические методы обнаружения аномалий

Статические методы обнаружения аномалий являются одним из базовых подходов к обеспечению информационной безопасности и широко применяются в системах мониторинга и защиты информационных систем. Данные методы основываются на заранее заданных правилах, сигнатурах и эвристиках, определяющих допустимое поведение системы и выявляющих отклонения от него. В отличие от методов машинного обучения, статические подходы не требуют предварительного этапа обучения и функционируют на основе экспертных знаний о предметной области [5].

В контейнерных средах, в частности в Kubernetes-кластерах, статические методы применяются для анализа audit-логов, сетевого взаимодействия, конфигурационных объектов и поведения пользователей и сервисных аккаунтов. Основной задачей является выявление заранее известных шаблонов аномального поведения, таких как попытки несанкционированного доступа, эскалации привилегий, использование запрещённых операций или отклонение от установленных политик безопасности.

К числу основных статических методов обнаружения аномалий относятся:

– **Сигнатурный анализ (signature-based detection)** — основан на сопоставлении наблюдаемых событий с заранее известными шаблонами атак. Данный подход широко используется в системах обнаружения вторжений и позволяет эффективно выявлять известные угрозы с высокой точностью. Однако он не способен обнаруживать новые или модифицированные атаки [6].

– **Правило-ориентированные методы (rule-based detection)** — предполагают использование набора логических правил, описывающих допустимое и недопустимое поведение системы. В Kubernetes такие правила могут определять ограничения на использование привилегированных контейнеров, доступ к API или действия сервисных аккаунтов. Подобные механизмы в Kubernetes реализуются, например, с использованием Admission Controller [7].

– **Пороговые методы (threshold-based detection)** — основаны на сравнении наблюдаемых метрик с заданными пороговыми значениями. Превышение допустимых уровней использования ресурсов (CPU, памяти) или частоты запросов может свидетельствовать о наличии аномалии. Простота реализации является преимуществом данного подхода, однако его эффективность зависит от корректности выбора порогов.

– **Методы белых и чёрных списков (whitelist/blacklist)** — предполагают явное задание разрешённых или запрещённых сущностей (пользователей, IP-адресов, операций). Отклонение от заданного списка интерпретируется как потенциальная аномалия. Данные методы часто применяются для контроля доступа и сетевого взаимодействия.

– **Методы, основанные на марковских процессах** — анализируют последовательности событий и вероятности переходов между состояниями системы.

1.2.1 Сигнатурный анализ

Сигнатурный анализ представляет собой метод, основанный на поиске совпадений между наблюдаемыми событиями и заранее известными шаблонами атак. Формально детектор можно представить как функцию:

$$D(x) = \begin{cases} 1, & \text{если } x \in S_{attack} \\ 0, & \text{иначе} \end{cases}$$

где x — наблюдаемое событие, S_{attack} — множество известных сигнатур атак [8].

Данный подход широко применяется в системах IDS (Intrusion Detection Systems). Его основным преимуществом является высокая точность обнаружения известных атак и низкая вычислительная сложность. Однако он не способен выявлять ранее неизвестные угрозы.

1.2.2 Правило-ориентированные методы

Правило-ориентированные методы используют набор логических условий для описания допустимого поведения системы. Формально правило можно представить как предикат:

$$R(x) = f(x_1, x_2, \dots, x_n) \rightarrow \{0, 1\}$$

где x_i — признаки события, а f — логическая функция.

Инструменты для реализации такой логики (например, Falco [9]) реализуют сопоставление правил с событиями контейнеров и предоставляют как стандартный набор правил, так и механизм расширения этого набора пользовательскими правилами; аналогично, policy-ориентированные решения (Open Policy Agent [10]) применяют декларативные ограничения на этапе развертывания контейнера в кластере.

Преимущество правило-ориентированных методов состоит в их детерминированности и объяснимости. Однако та же детерминированность определяет и фундаментальное ограничение: правила не способны обнаружить ранее не встречавшиеся, тонкие или композиционные паттерны поведения (zero-day и эволюционирующие атаки), а также требуют постоянного ручного сопровождения - обновления и тестирования наборов правил по мере появления новых уязвимостей и изменчивости рабочих нагрузок.

1.2.3 Пороговые методы

Пороговые методы основаны на анализе статистических характеристик метрик системы. Пусть имеется наблюдаемая метрика x_t , тогда аномалия определяется как:

$$x_t \notin [\mu - k\sigma, \mu + k\sigma]$$

где μ — среднее значение, σ — стандартное отклонение, k — коэффициент чувствительности.

Также могут использоваться более сложные методы:

- CUSUM (кумулятивные суммы);
- EWMA (экспоненциально взвешенное среднее);

Основное ограничение — чувствительность к изменению распределения данных и отсутствие учёта многомерных зависимостей.

1.2.4 Методы и черных белых списков

Метод белых списков основан на явном задании допустимых объектов:

$$D(x) = \begin{cases} 1, & x \in W \\ 0, & x \notin W \end{cases}$$

где W — whitelist.

Метод черных списков, напротив, основан на запрете заранее известных нежелательных объектов:

$$D(x) = \begin{cases} 0, & x \in B \\ 1, & x \notin B \end{cases}$$

где B — blacklist.

В Kubernetes такие методы применяются для контроля:

- разрешённых/запрещённых IP-адресов;
- сервисных аккаунтов;
- допустимых образов контейнеров;
- фильтрации вредоносных сетевых запросов.

Недостатком является высокая статичность и необходимость регулярного обновления списков.

1.2.5 Методы, основанные на марковских процессах

Марковские методы рассматривают поведение системы как последовательность состояний $S = \{s_1, s_2, \dots, s_n\}$, где переход между состояниями описывается условной вероятностью

$$P(s_t \mid s_{t-1}).$$

Таким образом, вероятность наблюдаемой последовательности состояний определяется как произведение вероятностей переходов:

$$P(s_1, s_2, \dots, s_T) = \prod_{t=2}^T P(s_t | s_{t-1}).$$

В задачах обнаружения аномалий модель обучается на нормальных последовательностях событий, формируя матрицу переходных вероятностей, отражающую типичное поведение системы. Если вероятность наблюдаемой последовательности оказывается ниже заданного порога, такая последовательность классифицируется как аномальная.

В контексте Kubernetes марковские модели применяются для анализа audit-логов, где состояния соответствуют типам API-запросов (например: create, delete, patch). Это позволяет выявлять нетипичные последовательности действий пользователей или сервисов. Основным преимуществом данного подхода является учёт временной структуры событий и зависимостей между последовательными действиями.

Однако ограничение классической марковской модели заключается в предположении о зависимости только от текущего состояния (свойство отсутствия памяти). Это снижает эффективность при анализе сложных атакующих сценариев, где важны долгосрочные зависимости и контекст.

1.3 Алгоритмы машинного обучения

Классические алгоритмы машинного обучения (МО) широко применяются для обнаружения аномалий в контейнерных средах благодаря способности выявлять скрытые закономерности в данных и определять отклонения от типичного поведения системы. В отличие от статических методов, такие алгоритмы опираются не на заранее заданные правила, а на статистические свойства обучающей выборки и особенности распределения наблюдений [5].

Для анализа контейнерных сред и Kubernetes-кластеров классические алгоритмы МО могут применяться к различным типам данных: audit-логам, метрикам использования ресурсов, сетевой телеметрии, событиям взаимодействия между подами и признакам поведения пользователей или сервисных аккаунтов.

К числу наиболее распространённых алгоритмов машинного обучения, применяемых для обнаружения аномалий, относятся:

- **К-ближайших соседей (K-Nearest Neighbors, K-NN)** — метод, основанный на оценке близости объекта к другим наблюдениям в пространстве признаков [11];
- **Isolation Forest (iForest)** — алгоритм, основанный на случайной изоляции объектов с помощью ансамбля деревьев [12];
- **Random Cut Forest (RCF)** — метод, использующий ансамбль случайных разрезов пространства признаков и подходящий для потоковых данных [13].

1.3.1 Метод К-ближайших соседей

K-NN — алгоритм классификации и регрессии, основанный на гипотезе компактности, которая предполагает, что расположенные близко друг к другу объекты в пространстве признаков имеют схожие значения целевой переменной или принадлежат к одному классу [11].

Принцип работы заключается в том, что для каждого нового наблюдения x алгоритм ищет k ближайших «соседей» в обучающей выборке и на основе расстояния/плотности соседей принимает решение (классификация) или вычисляет степень аномальности (anomaly score). Существуют несколько метрик расстояний:

- Евклидово расстояние: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$;
- Манхэттен (L1) расстояние: $d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$;
- Метрика Чебышева: $d(\mathbf{x}, \mathbf{y}) = \max_{i=1, \dots, n} |x_i - y_i|$.

Преимущества:

- простота реализации: алгоритм легко адаптируется к разным типам данных;
- гибкость: может использоваться для задач классификации, регрессии и выявления аномалий;

Ограничения:

- при большом объёме данных и высоком количестве признаков вычисление расстояний до всех соседей может быть ресурсоёмким;
- работает только с числовыми признаками, поэтому необходима предобработка для данных.

1.3.2 Алгоритм Isolation Forest

Алгоритм Isolation Forest (iForest) — метод обнаружения аномалий, основанный на идее изоляции объектов путём построения ансамбля случайных бинарных деревьев. Его фундаментальная концепция заключается в том, что образцы, которые проникают глубже в дерево, с меньшей вероятностью будут аномалиями, поскольку для их изоляции требуется больше разрезов, а образцы, которые оказываются в более коротких ветвях, указывают на аномалии, поскольку дереву проще отделить их от других наблюдений[12].

На рисунке 1 показана концепция обнаружения аномалий с помощью Isolation Forest.

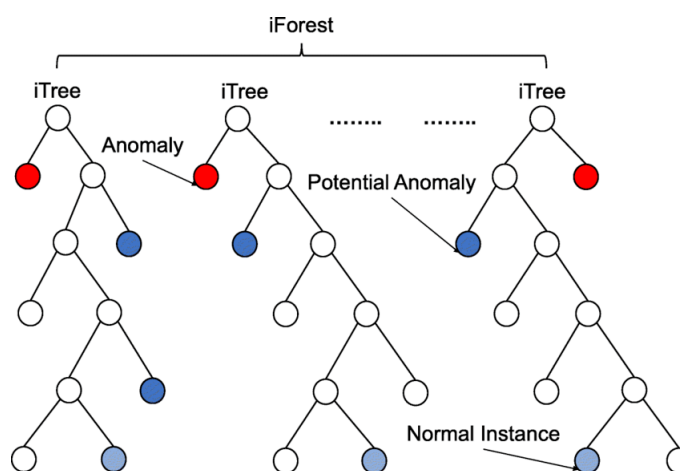


Рисунок 1 – Схема выявления аномалий на основе Isolation Forest [14].

Принцип работы алгоритма заключается в следующем. Из исходного множества данных формируется ансамбль деревьев iTREE t , каждое из которых строится на случайно выбранной подвыборке данных размера ψ . При построении дерева на каждом узле случайно выбирается признак q и случайный порог p из интервала $[\min_{x \in X} x_q, \max_{x \in X} x_q]$; затем пространство рекурсивно делится по этому порогу.

Построение iTREE происходит следующим образом:

- Выбирается подвыборка размера ψ .
- Рекурсивно: пока $|X| > 1$ и глубина ($height$) $< height_limit$:
 - случайно выбрать признак q из множества признаков;
 - случайно выбрать порог p из интервала $[\min_q, \max_q]$;

- разделить точки на множества $\{x \in X \mid x_q < p\}$ и $\{x \in X \mid x_q \geq p\}$ и рекурсивно строить дочерние узлы.

- Остановить, если $|X| = 1$ либо достигнут предельный уровень (height limit).

Для оценки аномальности объекта сначала вычисляют среднюю длину пути по всем деревьям ансамбля, $E(h(x))$, приведенную в формуле 1.

$$E(h(x)) = \frac{1}{t} \sum_{i=1}^t h_i(x) \quad (1)$$

где t — число деревьев.

Далее вычисляют нормирующую функцию $c(n)$, используемую для приведения средней длины пути к масштабу, независимому от размера выборки n , вычисляемую по формуле 2.

$$c(n) = 2H_{n-1} - \frac{2(n-1)}{n} \quad (2)$$

где H_n - гармоническое число

Наконец, для получения численной оценки аномальности применяют нормализацию средней длины пути $E(h(x))$ по значению функции $c(n)$; окончательная формула для вычисления значения аномальности приведена в 3.

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \quad (3)$$

Значение $s(x, n)$, близкое к 1, указывает на высокую степень аномальности.

1.3.3 Алгоритм Random Cut Forest

Алгоритм Random Cut Forest (RCF) — это метод обнаружения аномалий, основанный на построении ансамбля случайных деревьев разрезов. В отличие от Isolation Forest, который изолирует точки с помощью последовательных случайных бинарных разбиений подвыборок, RCF формирует деревья непосредственно на основе случайных геометрических разрезов пространства признаков, что позволяет корректно обновлять и поддерживать модель при потоковом поступлении данных [13].

Основная идея заключается в том, что аномальные точки — это такие объекты, добавление или удаление которых вызывает значительное изменение геометрической структуры дерева, то есть существенно изменяет распределение разрезов, формирующих дерево. Алгоритм использует понятие размера проекции (displacement) и сопоставимой дисперсии (co-displacement), которые измеряют, насколько сильно добавление или удаление точки влияет на «геометрическую плотность» соседних точек в пространстве.

Алгоритм построения деревьев в RCF схож с алгоритмом iForest, за исключением того, как выбирается признак q : вероятность выбора q пропорциональна длине стороны данного признака. Пусть имеется множество точек $S = \{x_1, x_2, \dots, x_n\} \subset R^d$. Далее рекурсивно строится дерево $T(S)$:

- 1) Высчитываются длины сторон гиперпрямоугольника по формуле 4.

$$l_i = \max_{x \in S} x_i - \min_{x \in S} x_i \quad (4)$$

- 2) Строится гиперпрямоугольник (bounding box)

$$B(S) = [\min_{x \in S} x_1, \max_{x \in S} x_1] \times \dots \times [\min_{x \in S} x_d, \max_{x \in S} x_d],$$

охватывающий все точки множества S .

- 3) Вероятность выбора признака $q \in Q$ высчитывается по формуле 5.

$$P(q) = \frac{l_q}{\sum_{i \in Q} l_i} \quad (5)$$

- 4) Выбирается порог p , равномерно выбранный из интервала $[\min_{x \in S} x_q, \max_{x \in S} x_q]$.

- 5) Разрезом (q, p) пространство $B(S)$ разделяется на две части:

$$S_L = \{x \in S \mid x_q < p\}$$

и

$$S_R = \{x \in S \mid x_q \geq p\}$$

В отличие от Isolation Forest, RCF не ограничивается метрикой глубины узла, а оценивает влияние объекта на геометрическую структуру данных. Это делает метод более устойчивым к изменениям плотности и эффективным для потоковых сценариев, где требуется онлайн-обновление модели без полной перестройки.

1.4 Методы обнаружения аномалий на основе нейронных сетей

Методы обнаружения аномалий на основе нейронных сетей в настоящее время являются одним из наиболее перспективных направлений анализа поведения информационных систем. Их основное преимущество заключается в способности автоматически извлекать скрытые закономерности из данных, учитывать нелинейные зависимости и моделировать сложные временные или структурные связи, которые трудно формализовать с помощью правил или алгоритмов машинного обучения.

К числу наиболее распространённых нейросетевых подходов к обнаружению аномалий относятся:

- **Рекуррентные нейронные сети (Recurrent Neural Networks, RNN)** — используются для анализа последовательных данных и временных зависимостей;
- **Сети с долгой краткосрочной памятью (Long Short-Term Memory, LSTM)** — разновидность рекуррентных сетей, лучше сохраняющая долгосрочный контекст;
- **Автоэнкодеры (Autoencoders, AE)** — используются для моделирования нормального поведения и выявления аномалий по ошибке реконструкции.

1.4.1 Рекуррентные нейронные сети (RNN)

Рекуррентные нейронные сети (RNN) — это класс нейронных сетей, предназначенных для обработки последовательных данных с учётом временных зависимостей. RNN содержат рекуррентные связи, позволяющие скрытому состоянию на каждом временном шаге сохранять информацию о предыдущих входах. Это создаёт эффект памяти и контекста [15]. Благодаря этому RNN подходят для задач, где важны порядок элементов и история наблюдений, например для прогнозирования временных рядов, анализа текста и речи [16].

На рисунке 2 показана архитектура RNN. На каждом временном шаге t входной вектор x_t преобразуется с помощью матриц весов и функции активации в скрытое состояние h_t , которое затем используется для получения выхода y_t . Ключевое отличие RNN заключается в том, что при вычислении текущего состояния учитывается предыдущее скрытое состояние h_{t-1} , что и обеспечивает механизм памяти.

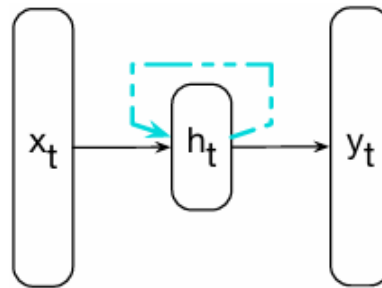


Рисунок 2 – Архитектура RNN [16].

RNN удобно представлять как сеть с прямой связью, «развернутую» по времени, где каждый узел скрытого слоя принимает на вход ранее вычисленное состояние. Такой подход показан на рисунке 3.

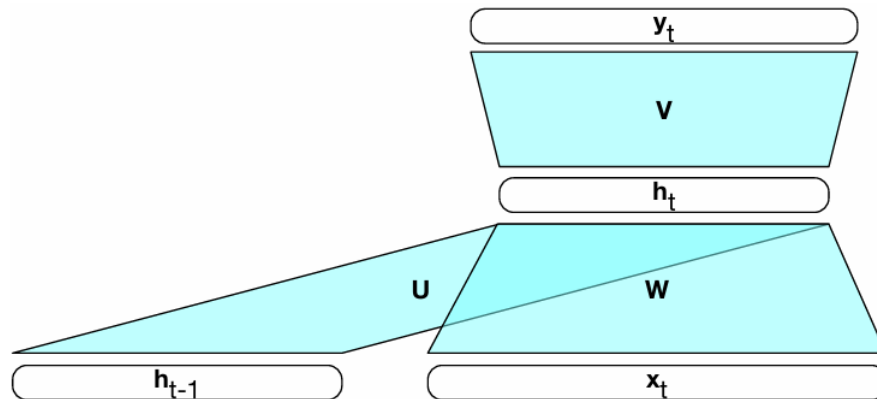


Рисунок 3 – Архитектура RNN, представленная как сеть с прямой связью [16].

Прямой проход RNN описывается следующими уравнениями. Скрытое состояние h_t на шаге t вычисляется по рекуррентному правилу, представленному в формуле 6.

$$h_t = g(Wx_t + Uh_{t-1} + b_h), \quad (6)$$

где W — матрица весов для входа, U — матрица рекуррентных весов, b_h — вектор смещений, а $g(\cdot)$ — нелинейная функция активации, например $\tanh$ или ReLU . Выход сети y_t на шаге t определяется выражением, приведённым в формуле 7.

$$y_t = f(Vh_t + b_y), \quad (7)$$

где V — матрица весов между скрытым и выходным слоями, b_y — вектор смещений выходного слоя, а $f(\cdot)$ — функция активации, часто используемая для нормализации выходов, например softmax для задач классификации. Процедура вычисления выполняется последовательно по временным шагам: сначала h_1 , затем h_2 и так далее до h_T . Таким образом, результат зависит от порядка элементов входной последовательности.

Преимущества RNN:

- 1) Способность моделировать последовательные и временные зависимости между входными данными.
- 2) Эффективное использование параметров: одни и те же веса применяются на всех временных шагах.
- 3) Обработка входов переменной длины.
- 4) Возможность онлайн-обучения на новых данных без полной перестройки модели.

Недостатки RNN:

- 1) Проблемы затухающих и взрывающих градиентов при обучении, затрудняющие захват долгосрочных зависимостей.
- 2) Ограниченная способность к долгосрочной памяти в архитектурах.
- 3) Последовательная природа вычислений затрудняет параллельное обучение и увеличивает вычислительную нагрузку.

1.4.2 Сети с долгой краткосрочной памятью (LSTM)

Long Short-Term Memory (LSTM) — это разновидность рекуррентных нейронных сетей (RNN), разработанная для решения проблемы затухающих и взрывающих градиентов, характерной для классических RNN. Благодаря этому LSTM эффективно моделируют долгосрочные зависимости в последовательных данных, что сделало их одной из наиболее популярных архитектур для анализа временных рядов, текста и других последовательностей [17].

Основная идея LSTM заключается в управлении потоком информации во времени с помощью механизмов ворот (gates). Ворота позволяют сети решать, какую информацию сохранить, обновить или забыть. Для этого в архитектуру LSTM добавляется вектор состояния ячейки (cell state), который играет роль долговременной памяти.

Каждый LSTM-блок содержит три основных типа ворот:

1) **Ворота забывания (Forget gate)** — определяют, какая часть информации из предыдущего состояния c_{t-1} должна быть сохранена или удалена;

2) **Входные ворота (Input gate)** — контролируют добавление новой информации в состояние ячейки;

3) **Выходные ворота (Output gate)** — определяют, какая часть обновлённого состояния c_t передается в скрытое состояние h_t .

Ворота забывания вычисляют взвешенную сумму скрытого слоя предыдущего состояния h_{t-1} и текущего входного вектора x_t и пропускают её через сигмоидную функцию (формула 8). Эта маска f_t затем поэлементно умножается (произведение Адамара) на вектор предыдущего контекста c_{t-1} , чтобы удалить из него информацию, которая больше не требуется (формула 9).

$$f_t = \sigma(U_f h_{t-1} + W_f x_t) \quad (8)$$

$$k_t = c_{t-1} \odot f_t \quad (9)$$

Входные ворота управляют добавлением новой информации в память. Сначала вычисляется кандидат на обновление памяти g_t , представляющий возможное новое содержимое контекста (формула 10).

$$g_t = \tanh(U_g h_{t-1} + W_g x_t) \quad (10)$$

Функция $\tanh(\cdot)$ ограничивает значения в диапазоне $(-1, 1)$, что повышает устойчивость обновлений. Далее вычисляется сигмоидная маска i_t , определяющая степень добавления новых данных в память (формула 11).

$$i_t = \sigma(U_i h_{t-1} + W_i x_t) \quad (11)$$

Эти два вектора поэлементно умножаются (формула 12), что формирует итоговое обновление памяти.

$$j_t = g_t \odot i_t \quad (12)$$

Итоговое состояние ячейки вычисляется как сумма модифицированного старого состояния k_t и нового добавления j_t (формула 13).

$$c_t = k_t + j_t \quad (13)$$

Выходные ворота определяют, какая часть обновлённого контекста c_t используется для формирования скрытого состояния h_t . Сначала вычисляется маска выхода o_t (формула 14).

$$o_t = \sigma(U_o h_{t-1} + W_o x_t) \quad (14)$$

Затем скрытое состояние определяется как поэлементное произведение маски выхода o_t и нелинейно преобразованного контекста (формула 15).

$$h_t = o_t \odot \tanh(c_t) \quad (15)$$

Рисунок 4 иллюстрирует полный процесс вычислений в одном блоке LSTM. Блок принимает на вход текущее наблюдение, скрытое состояние предыдущего шага и состояние ячейки, после чего формирует обновлённые значения на выходе. Именно скрытое состояние h_t используется как выход LSTM на каждом временном шаге.

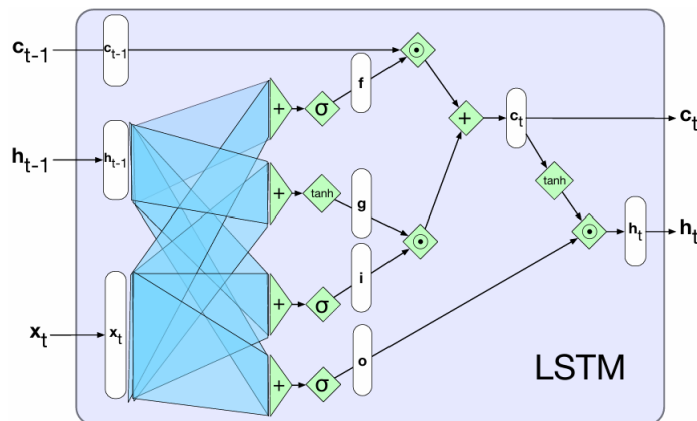


Рисунок 4 – Архитектура LSTM [16].

Архитектура LSTM естественным образом подходит для анализа метрик и логов Kubernetes-кластера, поскольку она оптимизирована для работы с последовательными и временными рядами.

1.4.3 Автоэнкодеры

Автокодировщик (Autoencoder, AE) — это нейросетевая модель, которая обучается сжимать входные данные в компактное латентное представление и восстанавливать исходные данные по этому представлению. Цель обучения состоит в минимизации ошибки восстановления, например MSE или бинарной кросс-энтропии, благодаря чему модель извлекает информативные признаки входных данных [18].

Архитектура автокодировщика состоит из двух основных частей:

- **Encoder** — преобразует входной вектор признаков x в компактное скрытое представление z ;
- **Decoder** — восстанавливает исходный вектор $\tilde{x}$ по скрытому представлению z .

На рисунке 5 представлена типичная архитектура автокодировщика. Входной вектор признаков проходит через несколько полносвязных слоёв Encoder, которые последовательно уменьшают размерность представления. Центральный слой формирует латентное пространство, содержащее компактное описание входных данных. Затем Decoder восстанавливает исходный вектор признаков.

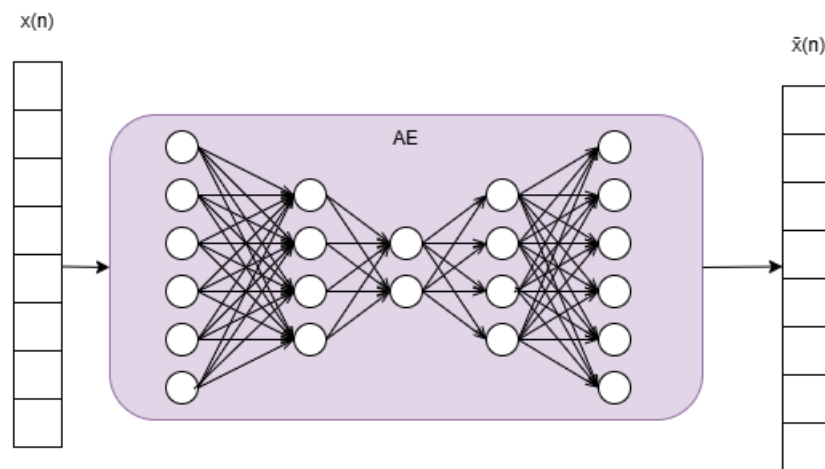


Рисунок 5 – Архитектура классического Autoencoder.

Математически процесс кодирования описывается в формуле 16.

$$z = g(W_e x + b_e) \quad (16)$$

где W_e — матрица весов Encoder, b_e — вектор смещений, а $g(\cdot)$ — функция активации.

Процесс декодирования, описанный в формуле 17, восстанавливает исходный вектор признаков.

$$\tilde{x} = f(W_d z + b_d) \quad (17)$$

где W_d — матрица весов Decoder, b_d — вектор смещений, а $f(\cdot)$ — функция активации выходного слоя.

Таким образом, автокодировщик аппроксимирует функцию восстановления, представленную в формуле 18.

$$\tilde{x} = f(W_d g(W_e x + b_e) + b_d) \quad (18)$$

1.5 Архитектура Kubernetes

Kubernetes представляет собой систему оркестрации контейнеризированных приложений с открытым исходным кодом, предназначенную для автоматизации процессов развертывания, масштабирования и управления контейнерами в распределённой инфраструктуре [19]. Кластер Kubernetes представляет собой совокупность физических или виртуальных машин, называемых узлами (nodes), которые совместно выполняют задачи управления и выполнения контейнеризированных приложений. На рисунке 6 представлена связь между компонентами Kubernetes.

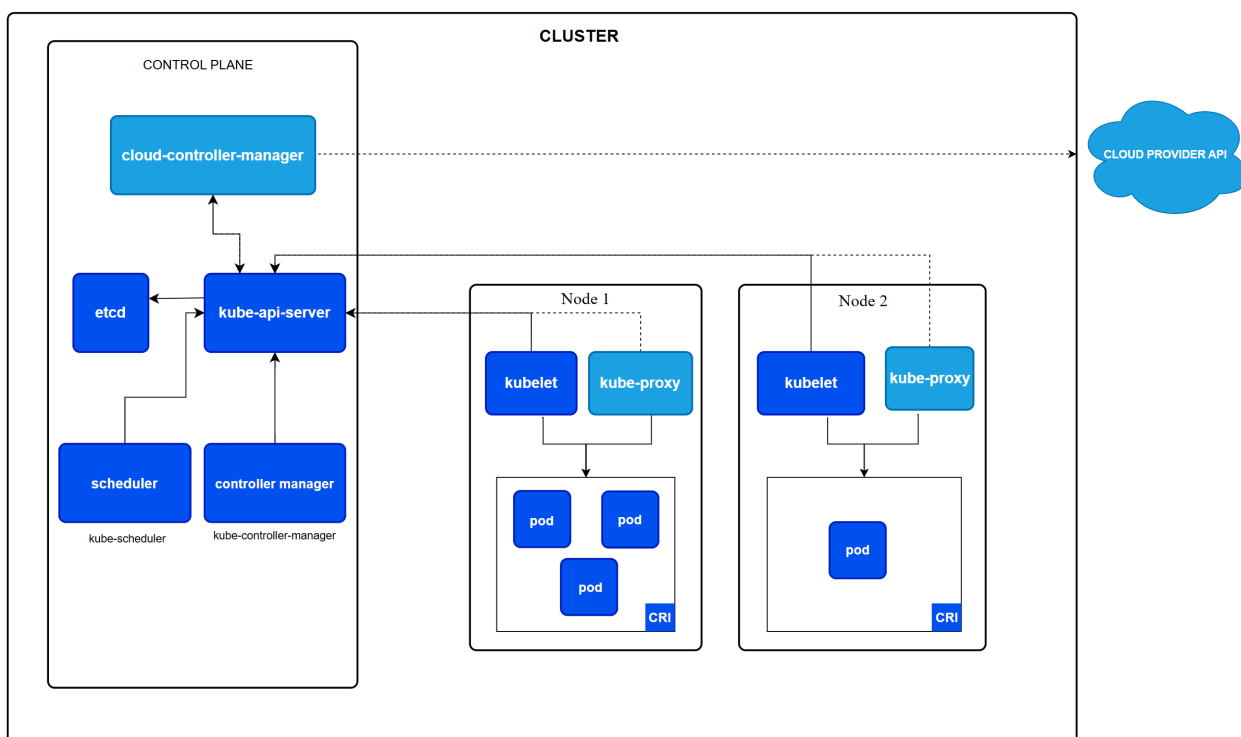


Рисунок 6 – Компоненты кластера Kubernetes [20].

В архитектуре кластера принято выделять два основных типа узлов:

- **Узлы плоскости управления (Control Plane nodes)** — узлы, на которых размещаются компоненты, отвечающие за управление состоянием кластера, планирование рабочих нагрузок и контроль выполнения задач.
- **Рабочие узлы (Worker nodes)** — узлы, на которых непосредственно запускаются контейнеры приложений и выполняются рабочие нагрузки.

1.5.1 Компоненты плоскости управления

Плоскость управления Kubernetes (Control Plane) обеспечивает централизованное управление кластером и поддерживает согласованность его состояния. Основные компоненты плоскости управления включают следующие элементы.

API-сервер (kube-apiserver) является центральным компонентом архитектуры Kubernetes и выступает в качестве интерфейса взаимодействия между пользователями, внутренними компонентами системы и кластером. Все операции управления кластером выполняются через API-сервер с использованием REST API [21]. kube-apiserver принимает входящие запросы,

выполняет их аутентификацию и авторизацию, проверяет корректность данных и записывает изменения состояния кластера в распределённое хранилище. Кроме того, API-сервер является источником audit-логов, фиксирующих информацию обо всех обращениях к API кластера.

etcd представляет собой распределённое отказоустойчивое хранилище типа «ключ–значение», предназначенное для хранения конфигурации и текущего состояния кластера. В etcd сохраняются сведения о ресурсах Kubernetes, таких как поды, сервисы, пространства имён и другие объекты. Все изменения состояния системы сначала фиксируются в etcd, после чего становятся доступными другим компонентам плоскости управления.

kube-scheduler — компонент, отвечающий за планирование рабочих нагрузок. Его задача заключается в выборе наиболее подходящего рабочего узла для размещения нового пода. Планировщик анализирует требования пода к ресурсам, текущую загрузку узлов, политики размещения и ограничения, после чего принимает решение о назначении пода на конкретный узел.

kube-controller-manager представляет собой набор контроллеров, которые отслеживают состояние объектов кластера и обеспечивают его соответствие заданному конфигурационному состоянию. Контроллеры реализуют механизм обратной связи, автоматически исправляя отклонения между текущим и желаемым состоянием системы.

Среди наиболее важных контроллеров можно выделить:

- **Node Controller** — контроллер, отслеживающий состояние рабочих узлов и реагирующий на их отказ или недоступность;
- **Replication Controller** и **Deployment Controller** — контроллеры, обеспечивающие поддержание заданного числа экземпляров подов;
- **Job Controller** — контроллер, управляющий выполнением пакетных заданий;
- **Service Account Controller** — контроллер, отвечающий за создание сервисных аккаунтов и токенов доступа.

Cloud Controller Manager используется в облачных средах и обеспечивает интеграцию Kubernetes с инфраструктурой облачного провайдера, включая управление сетевыми ресурсами, балансировщиками нагрузки и хранилищами.

1.5.2 Компоненты рабочих узлов

Рабочие узлы (Worker nodes) предназначены для непосредственного выполнения контейнеризированных приложений. Каждый рабочий узел включает несколько ключевых компонентов.

kubelet — основной агент узла, который обеспечивает взаимодействие рабочего узла с плоскостью управления. kubelet получает инструкции от API-сервера и отвечает за запуск, остановку и мониторинг контейнеров на узле.

kube-proxy — компонент, реализующий функциональность сервисов в Kubernetes. Он отвечает за маршрутизацию сетевого трафика между подами и сервисами, обеспечивая балансировку нагрузки и корректную доставку запросов.

Container Runtime — среда выполнения контейнеров, используемая для запуска и управления контейнерами на узле. В современных кластерах Kubernetes наиболее распространёнными реализациями являются containerd и CRI-O.

1.5.3 Взаимодействие компонентов кластера

Взаимодействие между компонентами Kubernetes осуществляется через API-сервер, который выступает центральной точкой управления системой. Пользователи, автоматизированные сервисы и внутренние компоненты отправляют запросы к API-серверу, который проверяет их корректность и вносит изменения в состояние кластера, записывая их в хранилище etcd.

Рабочие узлы регулярно взаимодействуют с API-сервером для получения информации о назначенных им задачах и передачи данных о состоянии запущенных подов. Контроллеры плоскости управления, в свою очередь, анализируют состояние объектов и выполняют действия, направленные на поддержание заданной конфигурации системы.

1.6 Механизм аудита и структура audit-логов Kubernetes

Кластер проверяет действия, совершаемые пользователями, приложениями, использующими API Kubernetes, и самой плоскостью управления. Каждый запрос на различных этапах его выполнения создаёт событие аудита, которое затем обрабатывается в соответствии с определённой политикой и записывается в бэкэнд. Политика определяет, что именно записывается, а бэкэнды сохраняют записи. В настоящее время в качестве бэкэндов используются файловая система и веб-хуки [3].

Audit-логи представляют собой важный источник информации о поведении пользователей и сервисов в кластере и широко применяются для задач мониторинга безопасности, расследования инцидентов и анализа активности API. Однако из-за распределённой природы Kubernetes одно пользовательское действие может инициировать цепочку вторичных операций, выполняемых различными компонентами системы, что приводит к появлению большого числа взаимосвязанных записей в журнале. Как отмечается в работе [22], такие события часто оказываются разбросанными по журналу и не имеют явной связи между собой, что затрудняет реконструкцию исходного действия и анализ контекста происходящих операций.

Механизм аудита создаёт события на различных стадиях обработки запроса. Определены следующие стадии:

- `RequestReceived` — событие фиксируется сразу при получении запроса аудиторским обработчиком (до делегирования дальше по обработчикам);
- `ResponseStarted` — заголовки ответа отправлены, тело ответа ещё не завершено (генерируется для долгоживущих запросов, например `watch`);
- `ResponseComplete` — тело ответа полностью отправлено;
- `Panic` — события, генерируемые при возникновении `panic` в процессе обработки.

Политика содержит упорядоченный список правил: при обработке события применяется первое совпавшее правило, которое и задаёт уровень аудита для события. Доступные уровни:

- `None` — не логировать события, соответствующие правилу;
- `Metadata` — логировать только метаданные (пользователь, таймштамп, ресурс, `verb` и т.д.), без тела запроса/ответа;
- `Request` — логировать метаданные и тело запроса (не применяется к `non-resource` запросам);
- `RequestResponse` — логировать метаданные, тело запроса и тело ответа (не применяется к `non-resource` запросам).

1.7 Выводы

В исследовательской части выпускной квалификационной работы был выполнен обзор и анализ предметной области обнаружения аномалий в контейнерных средах, а также рассмотрены особенности обеспечения безопасности в Kubernetes-кластерах.

В ходе исследования были проанализированы основные подходы к обнаружению аномалий, включая статические методы, классические алгоритмы машинного обучения и методы на основе нейронных сетей. Наиболее перспективными для решения поставленной задачи были признаны нейросетевые методы, способные выявлять сложные нелинейные зависимости и анализировать последовательности событий.

Отдельное внимание было уделено архитектуре Kubernetes и механизмам audit-логирования. Показано, что audit-логи API-сервера являются одним из наиболее информативных источников данных для обнаружения аномалий, так как содержат сведения о действиях пользователей, сервисных аккаунтов и компонентов кластера. Именно последовательный характер audit-событий позволяет использовать методы, учитывающие временные зависимости, в том числе рекуррентные нейронные сети и автоэнкодеры.

Таким образом, в исследовательской части были определены исходные данные для разработки программного средства обнаружения аномалий в Kubernetes-кластере. Полученные результаты послужат основой для проектирования архитектуры системы, формирования набора данных, построения модели и её последующей программной реализации.

2 КОНСТРУКТОРСКАЯ ЧАСТЬ

2.1 Обзор и анализ существующих подходов и методов в области обнаружения аномалий

В данном разделе представлен обзор и сравнительный анализ современных подходов к обнаружению аномалий в контейнерных средах, с акцентом на решения, применяемые в Kubernetes-кластерах. Особое внимание уделяется методам, основанным на использовании нейронных сетей, поскольку именно они в последние годы демонстрируют наибольшую эффективность при обработке сложных данных, характерных для распределённых систем.

В рамках данного раздела рассматриваются три современных решения: GATAKU, KubAnomaly и DeepLog. Выбор этих систем обусловлен тем, что они представляют различные парадигмы применения нейронных сетей для анализа поведения распределённых систем:

- использование графовых нейронных сетей (GNN) для моделирования сетевых взаимодействий между компонентами кластера (GATAKU);
- применение полносвязных нейронных сетей (MLP) для анализа агрегированных событий мониторинга (KubAnomaly);
- использование рекуррентных нейронных сетей (LSTM) для моделирования последовательностей логов и выявления временных аномалий (DeepLog).

2.1.1 GATAKU

В работе [23] рассматривается использование графовых нейронных сетей (GNN) для обнаружения аномалий в кластерах Kubernetes. Авторы предлагают средство обнаружения аномалий на основе GNN в Kubernetes под названием GATAKU (GNN-based Anomaly Detection in Kubernetes). В качестве источников сетевых событий используются CNI-плагин Cilium [24] и ingress-контроллер Traefik [25]. Главная цель работы — разработать модель, основанную на графовых нейронных сетях, способную эффективно и точно выявлять потенциальные угрозы и аномалии в кластере Kubernetes на основе сетевых событий.

В кластере развернуто простое многосервисное приложение, включающее:

- 1) **Frontend**: сервис в двух репликах, доступный из интернета через Traefik Ingress.

2) **Backend**: сервис в четырёх репликах, принимающий запросы от Frontend и обращающийся к сервису данных и базе данных.

3) **Сервис базы данных** (например, Postgres) в трёх репликах, обрабатывающий запросы от Backend.

4) **Data-сервис**: сервис, принимающий запросы от Backend и отправляющий некоторые запросы во внешние сервисы за пределами кластера.

Поды периодически генерируют трафик с помощью утилиты curl для имитации нормального взаимодействия между сервисами.

Для моделирования аномального трафика авторы рассматривают три сценария атак:

- Атаки типа «отказ в обслуживании» (DoS): запускается вредоносный под, который случайным образом выбирает целевой доброкачественный под в том же пространстве имен и начинает отправлять множество HTTP-запросов на него с помощью утилиты hey.

- Несанкционированное сканирование портов для атак разведки: сценарий похожий на DoS, только вредоносный порт начинает сканирование портов с помощью утилиты nmap.

- Сканирование веб-приложений на веб-уязвимости: запускается инструмент OWASP ZAP, который применяется для сканирования уязвимостей приложения.

Внедрение GATAKU выполняется в два этапа: сбор данных (запуск сценариев атак в тестовом кластере) и обучение модели с контролируемым обучением для обнаружения аномалий в сетевых данных.

Разработка модели включает три этапа:

1) **Предобработка данных**. Реализован конвейер предобработки, преобразующий каждый признак в числовой или категориальный формат. Используются различные кодировщики и нормализаторы для IP-адресов, портов, флагов TCP/UDP и других признаков. Помимо исходных характеристик введены дополнительные агрегирующие признаки (по портам, IP-адресам, временным интервалам) для вычисления индексов разнообразия.

2) **Построение графа**. Для формирования графовой структуры используется метод k -ближайших соседей (kNN): для каждой точки данных находят k наиболее похожих по признакам соседей, после чего каждая точка рассматривается как узел графа с рёбрами к k ближайшим узлам. Такой метод

выбран благодаря простоте реализации и пригодности для наборов данных с относительно небольшой размерностью. Авторы подчеркивают, что данный алгоритм хорошо работает на наборах с относительно небольшим числом измерений.

3) **Разработка модели НС.** В качестве модели используется графовая сверточная сеть (GCN), представленная на рисунке 7. Архитектура включает три сверточных слоя, слой регуляризации (dropout), функцию активации ReLU в скрытых слоях и Softmax в выходном слое. Модель формирует вероятности принадлежности точки данных к соответствующим классам аномалий.

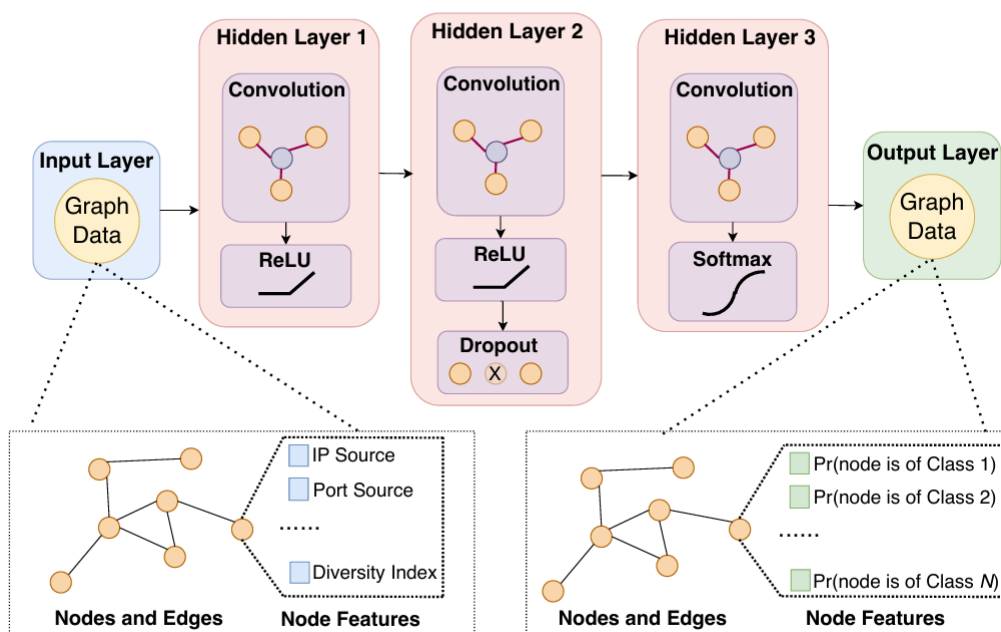


Рисунок 7 – Обзор модели GCN в GATAKU [23].

По результатам экспериментов GATAKU показала небольшое улучшение качества по сравнению с методами RF и SVM. Однако модель подвержена ограничениям по масштабируемости: построение графа (kNN), обучение GCN и обработка графовых структур создают значительную вычислительную нагрузку. Авторы отмечают, что в ресурсно-ограниченных средах применение GNN требует оптимизаций.

2.1.2 KubAnomaly

Авторы исследования [26] предлагают KubAnomaly — систему, обеспечивающую возможности мониторинга безопасности для обнаружения аномалий на платформе оркестрации Kubernetes. Данная система представляет собой модуль мониторинга контейнеров для Kubernetes с использованием НС для создания моделей классификации, которые усиливают способность системы выявлять аномальное поведение, такое как атаки на веб-сервисы, атаки, использующие распространённые и zero-day уязвимости, а также комбинированные атаки на различные компоненты кластера.

KubAnomaly реализует архитектуру с агентами на нодах и централизованным анализатором; основная идея — собирать «сырые» мониторинговые события, агрегировать их в интервалы, строить векторы признаков и классифицировать каждое окно как нормальное или аномальное с помощью обученной модели. Архитектура представлена на рисунке 8.

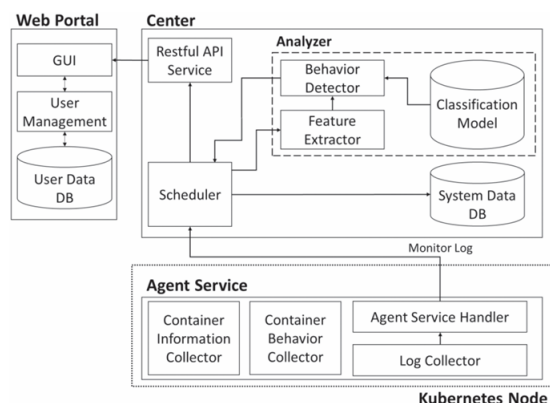


Рисунок 8 – Архитектура KubAnomaly [26].

Агенты системы KubAnomaly используются для сбора логов мониторинга из контейнеров на основе Docker и также поддерживают Kubernetes. Для сбора событий, описывающих поведение контейнера, используется Falco [9]. Авторы статьи выделили список событий, которые система отслеживает. Агент отправляет эти данные в центр обработки каждые 10 секунд.

Рабочий процесс KubAnomaly разбит на четыре этапа:

- 1) Сбор логов: агенты отправляют данные в центр обработки каждые 10 секунд.

2) Извлечение признаков: после получения исходных журналов мониторинга и информации о ПО контейнера журналы преобразуются в нужный формат. Далее исходные журналы мониторинга используются в качестве входных признаков для обучения модели.

3) Нормализация данных: для увеличения скорости сходимости и точности результата модели МО данные нормализуются.

4) Построение модели классификации аномалий: модель является полносвязной (fully-connected / MLP) сетью из четырёх слоёв. В качестве функций активации с первого по третий слой используются экспоненциальные линейные единицы (ELU), которые могут ускорять обучение в глубоких нейросетях и обеспечивать более высокую точность классификации. На выходе используется функция Softmax. Исходные данные подаются на первый слой; на выходе возвращается вероятность принадлежности события определённому типу аномалии.

Предложенная модель обнаружения аномалий KubAnomaly может выявлять аномальное поведение контейнеров непосредственно в Kubernetes. Она рассматривается как первая линия защиты для быстрого выявления отклонений. KubAnomaly демонстрирует, что агрегация событий в короткие временные окна и использование нейросетевой классификации дают практичный механизм обнаружения аномалий для контейнерных платформ с высокой точностью на подготовленных наборах данных. Результаты показали, что предложенная модель успешно может выявлять атаки на веб-сервисы с общей точностью до 96%. При этом решение требует качественно размеченных данных и внимания к производительности агента. Авторы предлагают конфигурации агента и парсинга и показывают, что при аккуратном проектировании накладные расходы остаются приемлемыми.

2.1.3 DeepLog

Авторы работы [27] предлагают DeepLog — фреймворк для обнаружения аномалий в реальном времени и диагностики на основе системных логов. Идея состоит в том, чтобы рассматривать поток лог-сообщений как последовательность (аналогичную естественному языку), которая следует определенным шаблонам и правилам грамматики, и моделировать её с помощью рекуррентной нейросети LSTM. Данный подход позволяет DeepLog автоматически изучать шаблоны журналов на основе нормального исполнения и обнаруживать аномалии, когда

шаблоны журналов отклоняются от модели, обученной на данных журналов при нормальном исполнении. Кроме того авторы предусмотрели постепенное обновление DeepLog в режиме онлайн, чтобы она могла адаптироваться к новым шаблонам журналов со временем.

Для подготовки логов DeepLog сначала парсит неструктурированные текстовые записи и выделяет log key (шаблон сообщения) и параметры (значения), авторы опираются на потоковый парсер Spell для получения структурированного представления логов и идентификации параметров/идентификаторов, позволяющих отделять последовательности, порождённые параллельными задачами.

DeepLog использует не только ключи журналов, но и значения метрик в записи журнала для обнаружения аномалий, благодаря чему способен выявлять разные типы аномалий. В итоге в DeepLog представлено два типа моделей:

1) **Модель аномалий по ключам (Log-Key Anomaly Detection).** Модель на основе LSTM обучается предсказывать следующий лог-ключ по окну предыдущих ключей. В режиме детекции фактический следующий ключ считается аномальным, если он не попадает в множество из top-k наиболее вероятных предсказаний сети (порог k настраиваем). Такой подход эффективно ловит нарушения в порядке и в логике выполнения.

2) **Модель аномалий по параметрам (Parameter-Value Anomaly Detection).** Для каждого шаблона логов строится отдельная модель на основе LSTM, моделирующая последовательность векторных параметров/метрик, связанных с этим шаблоном, как мультивариантный временной ряд. Это позволяет выявлять аномалии, связанные с неожиданными значениями параметров или с изменением времени между событиями (performance anomalies).

Общий рабочий цикл DeepLog состоит из двух этапов:

1) **Этап обучения.** Данные для обучения DeepLog представляют собой записи журналов из нормального пути выполнения системы. Каждая запись журнала парсится на ключ журнала и вектор значений параметров. Последовательность ключей журнала, полученная из обучающего файла журнала, используется DeepLog для обучения модели обнаружения аномалий ключей журнала и для построения моделей рабочего процесса выполнения системы в целях диагностики. Для каждого уникального ключа k DeepLog также обучает и поддерживает модель для выявления аномалий, связанных с отклонением от параметров этого ключа.

2) **Этап онлайн-детекции.** Новая запись журнала разбивается на ключ журнала и вектор значений параметров. DeepLog сначала использует ключ журнала модель обнаружения аномалий для проверки, является ли входящий ключ лога нормальным. Если да, DeepLog дополнительно проверяет вектор значений параметров с помощью модели обнаружения аномалий значений параметров для этого ключа лога. Новый лог будет помечен как аномалия, если либо его ключ лога, либо предполагается, что его вектор значений параметров является аномальным. Наконец, если он помечен как аномальный, модель рабочего процесса DeepLog предоставляет пользователю семантическую информацию для диагностики аномалии. Шаблоны выполнения могут изменяться со временем или не включались в исходные обучающие данные. DeepLog также предоставляет возможность сбора отзывов пользователей. Если пользователь сообщает о обнаруженной аномалии как о ложном срабатывании, DeepLog может использовать это как помеченную запись для поэтапного обновления своих моделей, чтобы учитывать и адаптироваться к новому шаблону.

На рисунке 9 приведена архитектура DeepLog.

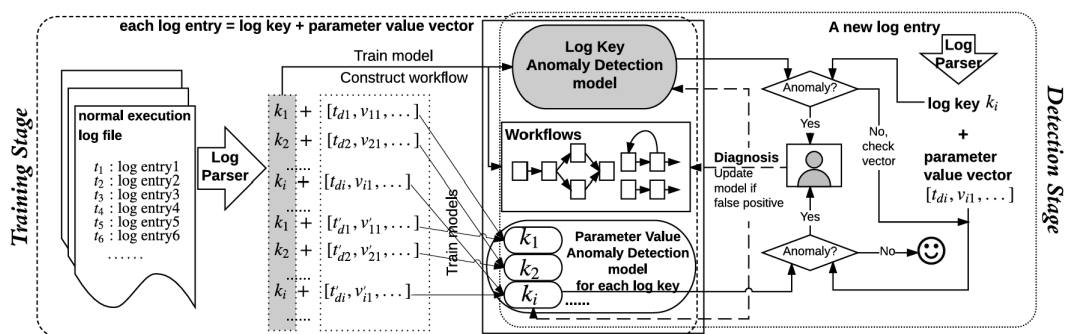


Figure 1: DeepLog architecture.

Рисунок 9 – Архитектура DeepLog [27].

Компоненты DeepLog можно сформулировать схематично следующим образом:

- **Log Parser:** потоковый парсер, преобразующий неструктурированные логи в пары log key, параметр, а также выделяющий идентификаторы для группировки записей по «сессиям» или исполнению. Это критический шаг для отделения параллельных потоков исполнения
- **Key Model (LSTM):** многослойная LSTM-сеть, принимающая окно предыдущих ключей и выдающая распределение вероятностей для следующего ключа; используется правило top-k для обнаружения аномалий.

- **Parameter Models (LSTM per key):** для каждого лог-ключа — строится отдельная LSTM-модель, которая анализирует вектор параметров/интервалы между событиями как временной ряд и выявляет аномальные значения или задержки.
- **Workflow Builder / Diagnosis:** модуль, который на этапе обучения конструирует workflow-модели (например, с помощью кластеризации со-встречаемости или на базе предсказаний LSTM) и после детекции использует эти модели для помощи в локализации причины аномалии.
- **Online Updater / Feedback:** механизм инкрементального обновления весов: при ошибочной классификации пользователи могут пометить запись как нормальную, и модель может обновить свои веса онлайн, чтобы адаптироваться к новым шаблонам исполнения.

В экспериментальной части авторы показали высокую эффективность DeepLog на нескольких больших наборах логов (включая HDFS и OpenStack): при обучении на очень небольшой доле «чистых» логов (менее 1%) DeepLog достигает практически 100% точности обнаружения на оставшейся части HDFS; похожие результаты продемонстрированы и для логов OpenStack. Также проведён анализ влияния параметров (глубины сети, числа юнитов, размера окна h и значения g для top- g), из которого видно, что система относительно устойчива к настройкам и обеспечивает низкую стоимость предсказания (=1 мс на запись на стандартной рабочей станции).

Авторы подчёркивают преимущества DeepLog: способность автоматически изучать сложные порядковые закономерности в логах (благодаря LSTM), интеграция анализа ключей и параметров для более полного покрытия типов аномалий, а также поддержка онлайн-адаптации и инструментов для последующей диагностики. В то же время, как и другие подходы на основе глубокого обучения, DeepLog требует этапа парсинга и достаточного объёма нормальных данных для надёжного обучения, а интерпретируемость отдельных внутренних состояний сети остаётся ограниченной.

2.2 Моделирование последовательностного подхода обнаружения аномалий

Для демонстрации работоспособности подхода по обнаружению аномалий в Kubernetes-кластере использован публичный набор данных HDFS_v1 из коллекции LogHub. Данный набор содержит более 11 миллионов лог-записей, сгруппированных по идентификатору блока, и включает разметку нормальных и аномальных трасс выполнения системы [28].

В рамках эксперимента рассматривается применение последовательностного подхода к анализу лог-событий, при котором каждая запись журнала рассматривается как элемент временной последовательности, отражающей поведение распределённой системы. Такой подход позволяет учитывать не только отдельные события, но и их порядок, контекст и взаимосвязи во времени, что особенно важно для задач обнаружения сложных аномалий в распределённых инфраструктурах.

В качестве базовой архитектуры для реализации данного подхода используется DeepLog [27], основанная на рекуррентной нейронной сети LSTM. Данная архитектура зарекомендовала себя как эффективное решение для анализа последовательностей логов и выявления отклонений от нормального поведения системы за счёт моделирования вероятностных зависимостей между последовательными событиями. DeepLog рассматривает поток логов как последовательность дискретных событий (log keys) и обучается предсказывать следующее событие на основе предыдущего контекста, что позволяет выявлять аномалии как нарушения ожидаемой структуры последовательности.

Каждая трасса логов преобразуется в последовательность событий фиксированной длины. Модель обучается на нормальных последовательностях и далее применяется к тестовой выборке, содержащей как нормальные, так и аномальные примеры. Для каждой последовательности вычисляется показатель аномальности s_i , на основе которого принимается решение о наличии отклонения:

$$\hat{y}_i = \begin{cases} 1, & s_i > \tau, \\ 0, & s_i \leq \tau. \end{cases} \quad (19)$$

Порог τ подбирается по валидационной выборке для достижения баланса между полнотой и точностью обнаружения.

Результаты эксперимента представлены на рисунке 10.

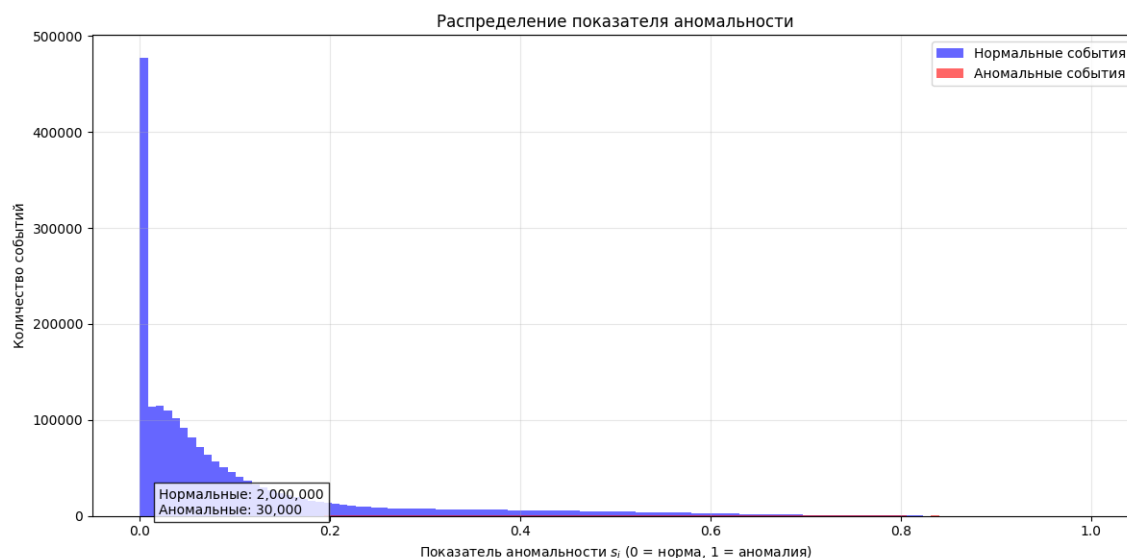


Рисунок 10 – Распределение показателя аномальности s_i для нормальных и аномальных трасс

Полученные результаты показывают, что нормальные и аномальные последовательности формируют различные распределения значений s_i , что подтверждает применимость последовательностного логического подхода для обнаружения аномалий в системных логах и демонстрирует его работоспособность на реальных данных.

2.3 Выбор используемого метода по обнаружению аномалий в контейнерных средах

На основе проведённого анализа существующих подходов и методов обнаружения аномалий в распределённых и контейнерных средах можно сделать вывод о том, что наиболее перспективными являются методы, основанные на нейронных сетях. Это обусловлено тем, что такие методы способны выявлять сложные нелинейные зависимости в данных, учитывать временные и контекстные связи между событиями, а также адаптироваться к изменяющемуся поведению системы.

Ключевой особенностью рассматриваемой задачи является отсутствие размеченных данных в процессе обучения. В реальных Kubernetes-кластерах аномальные события являются редкими, неполными или заранее неизвестными, что делает невозможным формирование полноценного обучающего набора с корректной разметкой классов. В таких условиях применение методов контролируемого обучения (supervised learning) существенно ограничено и не обеспечивает требуемой обобщающей способности модели.

Дополнительным усложняющим фактором является то, что каждый Kubernetes-кластер обладает индивидуальной архитектурой, набором развернутых сервисов, политиками безопасности, конфигурацией сетевого взаимодействия и характером нагрузки. В результате поведение «нормальной» системы существенно варьируется от кластера к кластеру, а также изменяется во времени в рамках одного и того же кластера. Это приводит к тому, что универсальный набор шаблонов или правил поведения практически невозможно сформировать без существенных ограничений по применимости.

В связи с этим в рамках данной работы рассматриваются методы неконтролируемого обучения (unsupervised learning), позволяющие обучать модель исключительно на данных нормального поведения системы. К числу таких методов относятся нейронные сети на основе автоэнкодеров (Autoencoder, AE).

В качестве основных рассматриваемых архитектур выбраны следующие варианты автоэнкодеров:

- **Автоэнкодер (AE)** — базовая архитектура, состоящая из энкодера и декодера, предназначенная для восстановления входных данных и выявления аномалий по величине ошибки реконструкции;
- **RNN-AE** — автоэнкодер, в котором энкодер и декодер реализованы с использованием рекуррентных нейронных сетей, что позволяет учитывать последовательностную структуру входных данных;
- **LSTM-AE** — расширение RNN-AE с использованием LSTM-слоёв, обеспечивающих более эффективное моделирование долгосрочных зависимостей в последовательностях логов и событий.

Выбор данных архитектур обусловлен их способностью работать с неразмеченными данными, характерными для задач мониторинга контейнерных сред, а также их доказанной эффективностью в задачах обнаружения аномалий на основе ошибки реконструкции. В рамках дальнейших разделов работы будет проведено проектирование и реализация выбранных моделей, а также их сравнительный анализ в контексте применения для обнаружения аномалий в Kubernetes-кластере.

2.4 Архитектура средства обнаружения аномалий

Разрабатываемое программное средство должно быть ориентировано на работу в условиях распределённой инфраструктуры и учитывать особенности функционирования Kubernetes-кластера, в частности высокую динамичность среды, большое количество генерируемых событий и необходимость обработки потоковых данных в режиме, близком к реальному времени.

В качестве источника данных используются audit-логи Kubernetes, агрегируемые и хранящиеся в системе OpenSearch. Для выполнения анализа данные извлекаются из хранилища, проходят этап предобработки и преобразуются в формат, пригодный для подачи на вход НС. Ключевым элементом системы является модель обнаружения аномалий на основе АЕ, позволяющая выявлять отклонения на основе ошибки реконструкции последовательностей. Такой подход обеспечивает возможность обнаружения ранее неизвестных типов аномалий без необходимости наличия размеченных данных.

Архитектура разработанного программного средства построена по принципу *Master–Worker* и ориентирована на разделение управляющих и вычислительных функций между отдельными компонентами системы. Такой подход позволяет обеспечить асинхронную обработку задач, упростить масштабирование вычислительной части и повысить устойчивость системы к росту объёма анализируемых данных.

Общая схема архитектуры программного средства представлена на рисунке 11. В состав системы входят центральный компонент *Master*, реализующий интерфейс для взаимодействия с пользователем, а также набор вычислительных узлов *Worker*, выполняющих обучение нейросетевых

моделей и детектирование аномалий. Взаимодействие между компонентами осуществляется посредством очереди задач, реализованной с использованием брокера сообщений *Redis*, что позволяет отделить пользовательские операции от ресурсоёмких вычислений.

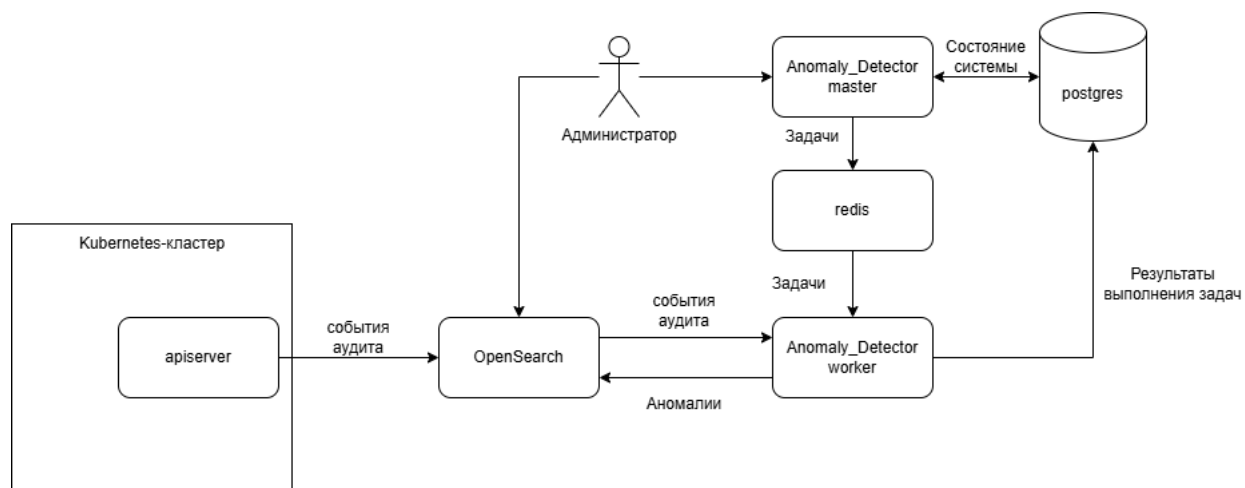


Рисунок 11 – Архитектура программного средства обнаружения аномалий

Компонент *Master* выполняет функции управления системой. Через него осуществляется аутентификация пользователей, создание и настройка интеграций с источниками данных, управление объектами моделей, а также запуск задач обучения и детектирования. Кроме того, *Master* отвечает за хранение и обработку метаданных, связанных с пользователями, моделями, задачами и результатами выполнения.

Компонент *Worker* предназначен для выполнения вычислительно затратных операций. В частности, он обеспечивает извлечение audit-логов из OpenSearch, преобразование данных с помощью конвейера формирования признаков, формирование обучающих последовательностей, обучение модели НС и последующее выявление аномальных событий.

Компонент *Redis* выполняет функции брокера сообщений между *Master* и *Worker*-узлами.

В качестве хранилища метаданных используется *PostgreSQL*. В базе данных фиксируются сведения о пользователях, интеграциях, моделях, задачах, результатах обработки и метриках обучения. Использование реляционной СУБД позволяет поддерживать целостность данных и удобно реализовать связи между сущностями системы.

Источником исходных данных для анализа выступает OpenSearch, в котором хранятся audit-логи Kubernetes-кластера.

Структура базы данных отражает основные сущности предметной области и их взаимосвязи. Диаграмма базы данных представлена на рисунке 12.

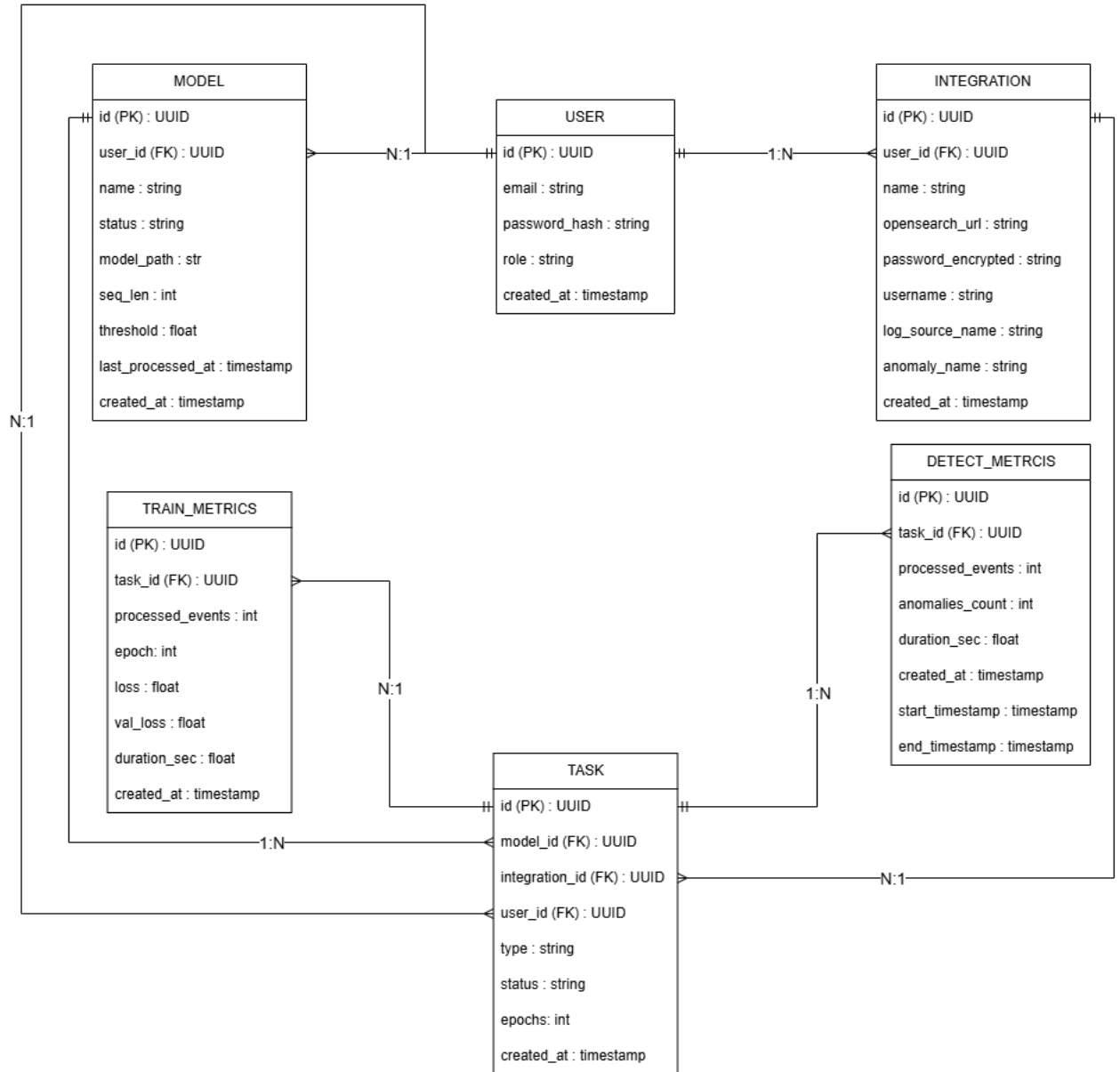


Рисунок 12 – UML-диаграмма базы данных программного средства

В базе данных выделены следующие основные сущности:

- *User* — хранит информацию о пользователях системы, включая учётные данные и роль;
- *Integration* — описывает подключение к источнику данных OpenSearch и содержит параметры доступа;

- *Model* — хранит информацию о моделях НС, включая путь до весов модели в S3-хранилище, текущий статус модели, порог обнаружения аномалий;
- *Task* — представляет задачи системы (обучение или обнаружение), выполняемые worker-компонентами;
- *Train_Metrics* — хранит метрики обучения моделей на различных эпохах;
- *Detect_Metrics* — содержит результаты выполнения задач на обнаружение аномалий в audit-логах за определенный период, включая статистику обработки и количество обнаруженных аномалий.

2.5 Формулирование математической постановки задачи

Аномалиями в поведении пользователей Kubernetes-кластера можно назвать действия, имеющие нетипичные признаки и (или) нетипичные паттерны взаимодействия с ресурсами кластера. Современные кластеры, развёрнутые на основе платформы Kubernetes, регистрируют все обращения к кластеру через механизм audit-логирования, реализованный в компоненте kube-apiserver [3].

Задача обнаружения аномалий в audit-логах Kubernetes формализуется как задача обнаружения отклонений. При наличии размеченной тестовой выборки она может рассматриваться как задача бинарной классификации. События рассматриваются как принадлежащие одному из двух классов: нормальные и аномальные. Введём формальные статистические гипотезы:

- Нулевая гипотеза H_0 (по умолчанию): событие является нормальным, т.е. $y = 0$.

- Альтернативная гипотеза H_1 : событие является аномальным, т.е. $y = 1$.

В рамках этих гипотез события можно классифицировать следующим образом:

- TP (true positives) — корректно обнаруженные аномалии (атаку);
- FP (false positives) — ложные тревоги (нормальные события, помеченные как аномальные);
- FN (false negatives) — пропущенные аномалии (атаки);
- TN (true negatives) — корректно обнаруженные нормальные события.

Задача бинарной классификации заключается в определении принадлежности объектов к одному из двух классов и может решаться методами МО с учителем при наличии размеченной выборки [29]. В задачах информационной безопасности, включая анализ audit-логов Kubernetes,

размеченные данные об аномалиях часто ограничены или отсутствуют. В этом случае задача может формулироваться как задача обнаружения отклонений с использованием методов обучения без учителя [5]. При таком подходе модель обучается на данных, характеризующих нормальное поведение, а аномалии выявляются как объекты, существенно отличающиеся от изученного распределения признаков.

Рассматриваемая задача выбора оптимальной модели обнаружения аномалий включает как задачу обучения параметров нейронной сети, так и задачу выбора её архитектуры. Эти задачи имеют различные входные данные и критерии оптимальности, поэтому математическая постановка разбивается на две частные подзадачи.

1) Первая подзадача заключается в обучении параметров нейронной сети при фиксированной архитектуре. В рамках данной подзадачи необходимо определить значения параметров модели, которые наилучшим образом описывают нормальное поведение пользователей кластера на основе обучающей выборки. Эта задача является задачей оптимизации функции потерь модели.

2) Вторая подзадача заключается в выборе оптимальной структуры нейронной сети из заданного множества допустимых архитектур. Для каждой архитектуры выполняется обучение параметров, после чего производится оценка качества обнаружения аномалий на тестовой выборке. Таким образом, данная подзадача представляет собой задачу оптимизации по архитектуре модели.

Разделение задачи на две подзадачи позволяет отделить процесс обучения параметров модели от процесса выбора её структуры и обеспечить корректное сравнение различных архитектур по единому критерию качества.

2.5.1 Исходные данные

Каждая запись audit-лога представляет собой структурированное сообщение в формате JSON, содержащее сведения о выполненном запросе, субъекте, инициировавшем операцию, целевом ресурсе и результате выполнения запроса. Audit-лог можно представить в виде упорядоченной во времени последовательности событий, как показано в формуле (20).

$$\mathcal{L} = (e_1, e_2, \dots, e_T) \quad (20)$$

В ходе анализа структуры audit-логов были выделены поля, наиболее информативные для последующего применения методов обнаружения аномалий, характеризующие событие e_t , как показано в формуле (21). Аналогичный набор признаков использовался в работе [22] для определения контекста между событиями с помощью НС.

$$e_t = (v_t, u_t, ua_t, r_t, sr_t, ns_t, c_t, ip_t, t_t), \quad (21)$$

где:

- v_t — значение поля `verb`, определяющее тип операции, выполняемой пользователем или сервисным аккаунтом над ресурсом Kubernetes (например, `get`, `list`, `create`, `update`, `delete`, `watch`);
- u_t — значение поля `user.username`, содержащее имя пользователя или сервисного аккаунта, от имени которого был выполнен запрос к API-серверу Kubernetes;
- ua_t — значение поля `userAgent`, содержащее строку идентификации клиента, осуществившего запрос к API (например, `kubectl`, клиентские библиотеки Kubernetes или автоматизированные сервисы);
- r_t — значение поля `objectRef.resource`, определяющее тип ресурса Kubernetes, к которому осуществляется обращение (например, `Pods`, `deployments`, `configmaps`, `secrets`);
- sr_t — значение поля `objectRef.subresource`, определяющее подресурс объекта Kubernetes, если обращение производится не к основному ресурсу, а к его части (например, `status`, `scale`, `log`);
- ns_t — значение поля `objectRef.namespace`, определяющее пространство имён Kubernetes, в котором расположен целевой ресурс;
- c_t — значение поля `responseStatus.code`, содержащее HTTP-код ответа API-сервера, характеризующий результат выполнения запроса (например, 200, 201, 403, 404);
- ip_t — значение поля `sourceIPs`, содержащее список IP-адресов источников запроса, с которых был выполнен доступ к API-серверу Kubernetes;
- t_t — значение поля `requestReceivedTimestamp`, содержащее временную метку получения запроса API-сервером и определяющее момент времени, в который событие было зафиксировано в audit-логе.

Выбор указанных полей обусловлен тем, что они отражают ключевые характеристики взаимодействия пользователей и сервисов с ресурсами кластера: тип выполняемой операции, субъект действия, используемый клиент, объект доступа и результат выполнения запроса. Совместное использование указанных признаков позволяет формировать последовательности событий, характеризующие типичное поведение субъектов системы, что является основой для последующего применения методов обнаружения аномалий.

Для кодирования событий в числовой вектор используется функция Φ , определяемой формулой (22), которая преобразует сырую последовательность структурированных записей audit-лога $\mathcal{L} = \{e_1, \dots, e_T\}$ к единому признаковому представлению.

$$\Phi : \mathcal{E} \rightarrow \mathbb{R}^d, \quad (22)$$

где $\mathcal{E}$ — множество всех возможных событий audit-лога. В результате формируется последовательность векторов признаков для каждого события e_t , определяемая формулой (23).

$$x_t = \Phi(e_t) \in \mathbb{R}^d. \quad (23)$$

Таким образом, формируется выборка, определяемая формулой (24).

$$\mathcal{X} = \{x_1, x_2, \dots, x_T\}, \quad x_t \in \mathbb{R}^d. \quad (24)$$

Предполагается, что исходная выборка разбивается на три подмножества:

- обучающая выборка $\mathcal{X}_{train} \subset \mathcal{X}$, содержащая объекты, соответствующие нормальному поведению пользователей в кластере, используемая для обучения параметров моделей;
- валидационная выборка $\mathcal{X}_{val} \subset \mathcal{X}$, используемая для оценки значения функции потерь и выбора оптимальных параметров модели;
- тестовая выборка $\mathcal{X}_{test} \subset \mathcal{X}$, содержащая размеченные объекты и используемая для итоговой оценки качества моделей.

Пусть задано множество допустимых архитектур нейронных сетей

$$\mathcal{A} = \{A_\alpha\},$$

Для каждой архитектуры A_α определяется множество параметров модели θ . Модель обозначим как

$$f_{\alpha,\theta} : \mathbb{R}^d \rightarrow \mathbb{R},$$

где

α - структура сети,

θ - обучаемые параметры.

2.5.2 Выбор параметров нейронной сети

Входными данными подзадачи являются:

- обучающая выборка событий $\mathcal{X}_{train}$;
- валидационная выборка $\mathcal{X}_{val}$;
- фиксированная архитектура нейронной сети A_α ;
- функция потерь модели $\mathcal{F}$.

Для определения параметров нейронной сети используется функция потерь, зависящая от выбранной архитектуры модели. Обучение параметров осуществляется на обучающей выборке $\mathcal{X}_{train}$, а значение функции потерь вычисляется на валидационной выборке $\mathcal{X}_{val}$, что позволяет оценивать обобщающую способность модели и предотвращать её переобучение. Оптимальные параметры модели определяются согласно формуле (25).

$$\theta^*(\alpha) = \arg \min_{\theta} \frac{1}{|\mathcal{X}_{val}|} \sum_{x \in \mathcal{X}_{val}} \mathcal{F}(x; \alpha, \theta), \quad (25)$$

где $\mathcal{F}$ — функция потерь, зависящая от выбранного типа модели (например, ошибка реконструкции Autoencoder [18]).

Результатом решения подзадачи является оптимальный набор параметров модели $\theta^*(\alpha)$.

2.5.3 Выбор структуры нейронной сети

Входными данными подзадачи являются:

- множество допустимых архитектур нейронных сетей $\mathcal{A}$;
- обученные модели $f_{\alpha,\theta^*(\alpha)}$ для каждой архитектуры;
- тестовая выборка $\mathcal{X}_{test}$;
- вектор меток $\mathcal{Y}_{test}$.

Для оценки качества моделей обнаружения аномалий используется метрика площади под PR-кривой (AUC-PR). Важность выбора PR-метрики при дисбалансе классов показана в работе [30]. PR-кривая отражает зависимость точности (Precision), вычисляемой по формуле (26), от полноты (Recall), вычисляемой по формуле (27), при варьировании порога обнаружения.

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (26)$$

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (27)$$

Пусть имеется тестовая выборка $\mathcal{X}_{test}$ с известными метками $\mathcal{Y}_{test} = \{y_i\}$, $y_i \in \{0, 1\}$.

Для каждой архитектуры A_α вычисляется набор показателей аномальности, определяемый формулой (28).

$$S_\alpha = \{s_\alpha(x_1), \dots, s_\alpha(x_n)\}, \quad (28)$$

где $s_\alpha(x_i) = f_{\alpha, \theta^*(\alpha)}(x_i)$ - показатель аномальности события

Качество детекции оценивается с использованием метрики PR-AUC, определяемой формулой (29).

$$Q(\alpha) = \text{AUC}_{PR}(\mathcal{Y}_{test}, S_\alpha). \quad (29)$$

Тогда задача выбора структуры нейронной сети формализуется формулой (30).

$$\alpha^* = \arg \max_{\alpha \in \mathcal{A}} \text{AUC}_{PR}(\mathcal{Y}_{test}, S_\alpha). \quad (30)$$

2.6 Обобщённая формулировка задачи

Таким образом, задача сводится к двум подзадачам:

- 1) выбор параметров сети при фиксированной архитектуре на наборе $\mathcal{X}_{train}$;
- 2) выбор оптимальной структуры сети по критерию максимизации PR-AUC на наборе $\mathcal{X}_{test}$.

Итоговая оптимизационная задача задаётся системой формул (31)–(32). Оптимальные параметры, полученные из формулы (31), используются при решении задачи, представленной формулой (32).

$$\theta^*(\alpha) = \arg \min_{\theta} \frac{1}{|\mathcal{X}_{val}|} \sum_{x \in \mathcal{X}_{val}} \mathcal{F}(x; \alpha, \theta), \quad (31)$$

$$\alpha^* = \arg \max_{\alpha \in \mathcal{A}} \text{AUC}_{PR}(\mathcal{Y}_{test}, S_{\alpha}). \quad (32)$$

Полученная архитектура A_{α^*} и параметры $\theta^*(\alpha^*)$ задают оптимальную модель обнаружения аномалий в рамках выбранного множества архитектур $\mathcal{A}$.

2.7 Методы решения задачи

Решение поставленной задачи предполагает последовательное выполнение нескольких этапов: подготовку и предобработку исходных данных, формирование признакового описания событий, обучение моделей нейронных сетей и оценку их качества.

На этапе подготовки данных формируется набор событий audit-логов $\mathcal{X}_{train}$ и $\mathcal{X}_{test}$. Для тестового набора выполняется разметка событий на классы (нормальные и аномальные), при этом набор должен включать как типичные операции легитимных пользователей, так и возможные сценарии атак, попыток эскалации привилегий, массового сканирования ресурсов и несанкционированного доступа.

На этапе предобработки данных осуществляется преобразование исходных событий audit-логов в численные векторы признаков, пригодные для обучения НС. Основные задачи включают:

- **Очистка и унификация данных:** устранение пропусков, нормализация числовых значений, приведение категориальных полей к единому формату. Это обеспечивает согласованность представления событий и снижает вероятность ошибок при обучении моделей.

- **Кодирование категориальных признаков:** преобразование различных типов категориальных данных (например, идентификаторы пользователей, типы операций, виды ресурсов) в численные представления. Возможны подходы, такие как one-hot кодирование, хеширование категорий или обучаемые embedding-слои. Это позволяет модели выявлять закономерности в действиях пользователей и взаимодействиях с ресурсами.

– **Построение временных и контекстных признаков:** выделение информации о временных паттернах событий (циклические характеристики, рабочее/нерабочее время), истории действий пользователя, последовательности обращений к ресурсам и агрегированных статистик по различным объектам кластера. Такие признаки повышают информативность представления событий и улучшают способность модели обнаруживать нетипичные последовательности действий.

– **Интеграция флагов и дополнительных индикаторов:** формирование булевых или численных индикаторов, отражающих специфические характеристики запросов, состояния ресурсов или контекста операции. Например, можно учитывать доступ к чувствительным ресурсам, статус ответа системы или происхождение запроса. Эти признаки помогают выявлять потенциально аномальные сценарии работы.

В результате предобработки каждое событие представляется в виде вектора признаков фиксированной размерности, содержащего числовые, категориальные и контекстные компоненты.

Для классификации событий целесообразно использовать НС различных архитектур, включая RNN [15], LSTM [17] и автокодировщики [18]. Для каждой архитектуры сети A_α выполняется обучение на наборе $\mathcal{X}_{train}$ с целью минимизации функции потерь $\mathcal{F}$, а затем рассчитывается показатель аномальности $s(x)$ для тестовых событий $\mathcal{X}_{test}$. Качество моделей оценивается с использованием метрики PR-AUC, что особенно важно при дисбалансе классов. По результатам сравнительного анализа выбирается оптимальная архитектура α^* и соответствующие параметры $\theta^{(\alpha^*)}$, формирующие рабочую модель обнаружения аномалий в Kubernetes-кластере.

2.8 Предобработка и кодирование событий audit-логов

Целью предобработки является переход от сырой последовательности структурированных записей audit-лога к единому признаковому представлению. Для обучения нейронной сети каждое событие e_t кодируется в числовой вектор с помощью функции Φ .

Основные этапы преобразования одного события e_t в вектор x_t :

- 1) Обработка пропусков и удаление записей, не подлежащих дальнейшей обработке.
- 2) Кодирование категориальных и текстовых полей в числовые векторы.

3) Нормализация числовых признаков и формирование итогового вектора-конкатенации.

Из поля `requestReceivedTimestamp` выделяются циклические признаки для часа и дня недели, а также извлекается бинарный признак принадлежности времени рабочему интервалу. Соответствующий вектор временных признаков задаётся формулой (33).

$$\Phi_{\text{time}} = \begin{bmatrix} \sin(2\pi \text{ hour}/24) \\ \cos(2\pi \text{ hour}/24) \\ \sin(2\pi \text{ dow}/7) \\ \cos(2\pi \text{ dow}/7) \\ \text{is_working_hours} \end{bmatrix} \quad (33)$$

Значение метки времени приводится к локальному времени перед извлечением признаков, а рабочие часы конфигурируются для конкретной организации (например, 09:00–18:00). Ожидаемая размерность временных признаков: $\dim(\Phi_{\text{time}}) = 5$.

Поле `verb` имеет небольшой конечный набор значений (например, `get`, `list`, `create`, `update`, `delete`, `watch`, ...), поэтому для него хорошо подходит one-hot-кодирование. Ожидаемая размерность: $\dim(\Phi_{\text{verb}}) = 16$.

Поле `user.username`, характеризующее пользователей и сервисные аккаунты, относится к категориальным признакам с высокой кардинальностью (high-cardinality). Прямое one-hot-кодирование таких признаков приводит к очень большой размерности пространства признаков и ухудшает масштабируемость алгоритмов машинного обучения. В подобных случаях часто применяется метод `feature hashing` (hashing trick), позволяющий отображать произвольные строковые признаки в фиксированное пространство признаков с использованием хеш-функции [31]. Данный подход позволяет существенно сократить размерность признакового пространства и отказаться от хранения словаря категорий, что особенно важно при работе с потоковыми данными и большим числом уникальных значений.

В соответствии с этим подходом строковое значение пользователя отображается в индекс вектора фиксированной размерности H . В текущей задаче выбрана размерность $H = 64$. Дополнительно из поля `user.username` извлекается признак типа субъекта (например, сервисный аккаунт, пользователь или компонент инфраструктуры). Итоговая размерность кодирования пользователей составляет $\dim(\Phi_{\text{user}}) = H + 3 = 67$.

Поле `userAgent` содержит описание клиентского приложения; для него имеет смысл использовать one-hot-кодирование для наиболее часто используемых агентов в компании, например `{kubectl, helm, controller, browser, curl, other}`, а все прочие значения приводить к этим типам регулярными выражениями. Ожидаемая размерность: $\dim(\Phi_{\text{userAgent}}) = 6$.

Поле `objectRef.resource` вместе с `objectRef.subresource` образуют токен вида `resource/subresource` (например, `Pods/status`). Такая строка кодируется с помощью hashing trick с размерностью $H_{\text{res}} = 64$. Дополнительно вводится булев признак `sensitive`, указывающий на обращение к чувствительным ресурсам (например, `secrets, serviceaccounts, tokenreviews`). Ожидаемая размерность кодирования ресурсов: $\dim(\Phi_{\text{resource}}) = H_{\text{res}} + 1 = 65$.

Поле `objectRef.namespace` определяет пространство имён, к которому был выполнен запрос. Для кодирования используется набор булевых индикаторов:

- `is_kube_system` — пространство имён относится к системным компонентам Kubernetes;
- `is_prod` — пространство имён содержит производственные приложения;
- `is_infra` — пространство имён используется инфраструктурными сервисами;

Итоговая размерность кодирования пространства имён: $\dim(\Phi_{\text{namespace}}) = 3$.

Поле `sourceIPs` содержит список IP-адресов, ассоциированных с источником поступившего запроса. В зависимости от принадлежности адреса к различным сетевым сегментам выполняется его преобразование в набор бинарных признаков, отражающих контекст происхождения запроса.

Для кодирования источника запроса используются булевы признаки:

- `internal_flag` — адрес принадлежит внутренней сети;

- `cluster_internal_flag` — запрос инициирован изнутри Kubernetes-кластера;
- `external_flag` — внешний источник запроса.

Такое представление позволяет учитывать сетевую природу источника запроса и различать внутренние и внешние взаимодействия на уровне модели. В результате IP-адрес преобразуется в компактный бинарный вектор фиксированной длины. Ожидаемая размерность кодирования IP-адреса: $\dim(\Phi_{ip}) = 3$.

Поля `responseStatus.code` и `responseStatus.status` описывают результат обработки запроса. Для уменьшения размерности HTTP-коды группируются по классам $\{1xx, 2xx, 3xx, 4xx, 5xx\}$. Каждый класс кодируется отдельным бинарным признаком (one-hot-кодирование). Дополнительно вводится булев признак `failure_flag`, позволяющий явно выделить неуспешные запросы, которые могут свидетельствовать о попытках подбора прав доступа или некорректном использовании API, вычисляемый по формуле (34).

$$\text{failure_flag} = \begin{cases} 1, & \text{если } code \geq 400, \\ 0, & \text{иначе.} \end{cases} \quad (34)$$

Итоговая размерность кодирования статуса ответа: $\dim(\Phi_{\text{response}}) = 6$.

Таким образом, функция кодирования Φ может быть представлена как объединение частных функций кодирования отдельных полей, что формально задаётся выражением (35).

$$\Phi(e_t) = \text{concat}(\Phi_{\text{time}}, \Phi_{\text{verb}}, \Phi_{\text{user}}, \Phi_{\text{userAgent}}, \Phi_{\text{resource}}, \Phi_{\text{namespace}}, \Phi_{ip}, \Phi_{\text{response}}). \quad (35)$$

Полученный вектор признаков фиксированной размерности используется далее как вход нейронной сети.

2.9 Формирование обучающей, валидационной и тестовой выборок

Корректное формирование обучающих и тестовых данных играет важную роль в обеспечении объективной оценки качества моделей и предотвращении переобучения. Для формирования датасета в данной работе использовался экспериментальный Kubernetes-кластер, в котором был настроен аудит. На основе этих событий формировались обучающая, валидационная и тестовая выборки, применяемые в процессе обучения и оценки нейронных сетей.

2.9.1 Экспериментальная среда и кластер Kubernetes

Сбор данных осуществлялся в экспериментальной среде, включающей Kubernetes-кластер, состоящий из трёх узлов: одного узла плоскости управления (узел управления, control plane node) и двух рабочих узлов (worker nodes). Архитектура кластера представлена на рисунке 13.

Схема сбора audit-логов Kubernetes для обучения ИС

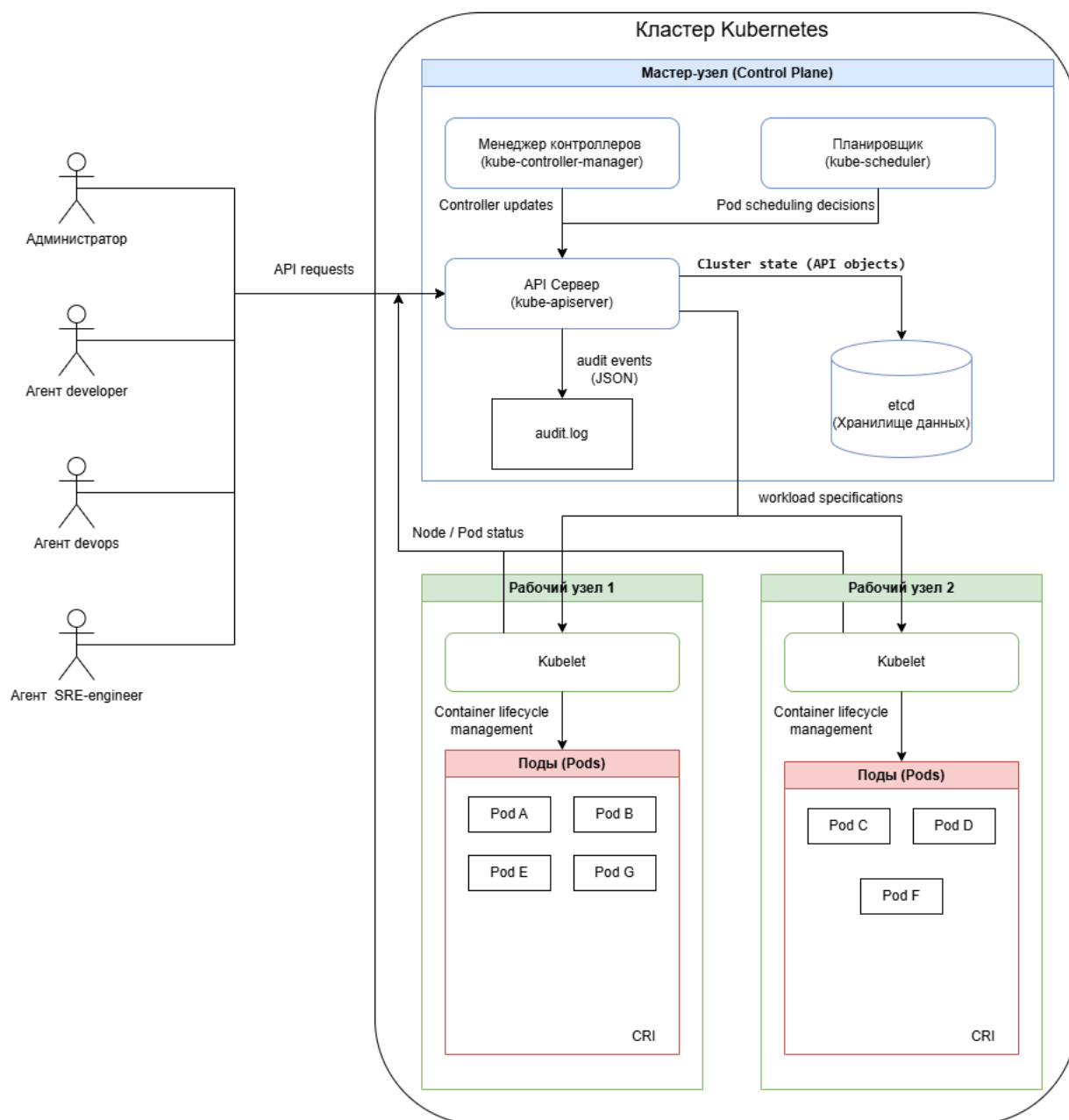


Рисунок 13 – Архитектура экспериментальной среды

В кластере был включён механизм audit-логирования Kubernetes, фиксирующий все обращения к API-серверу. Конфигурация политики аудита обеспечивала регистрацию основных параметров запросов. Среди них: тип операции, пользователь, ресурс, пространство имён, источник

запроса и результат выполнения операции. Audit-логи собирались в процессе взаимодействия пользователей и автоматизированных сервисов с кластером и далее использовались для формирования набора данных, применяемого при обучении и тестировании моделей обнаружения аномалий.

2.9.2 Генерация событий audit-логов с использованием агентов

Для генерации событий аудита использовались специализированные агенты-симуляторы поведения, имитирующие типичное поведение различных ролей пользователей в инфраструктуре Kubernetes. Использование агентов позволило автоматически формировать последовательности операций, соответствующие реальным сценариям эксплуатации кластера.

В рамках эксперимента были реализованы три типа агентов:

- **developer** — агент, моделирующий поведение разработчика, взаимодействующего с кластером для развертывания и тестирования приложений. Данный агент выполняет операции создания и обновления Deployment, получения информации о подах, просмотра логов контейнеров и взаимодействия с конфигурационными объектами;

- **devops** — агент, моделирующий действия инженера DevOps, отвечающего за управление инфраструктурой и автоматизацию процессов развертывания. Агент выполняет операции управления ресурсами, масштабирования приложений, обновления конфигураций и мониторинга состояния сервисов;

- **SRE-engineer** — агент, моделирующий деятельность инженера по надёжности сервисов (Site Reliability Engineer). Основные действия включают диагностику проблем, анализ состояния системы, получение метрик, просмотр логов и выполнение административных операций по восстановлению работоспособности сервисов.

Каждый агент функционировал на основе заранее заданного профиля поведения, определяющего допустимые типы операций, используемые ресурсы Kubernetes, характерные пространства имён и типичные временные паттерны активности. В результате взаимодействия агентов с кластером формировалась последовательность событий audit-логирования, отражающая нормальное поведение пользователей в системе.

2.9.3 Формирование обучающей и валидационной выборок

На первом этапе формирования датасета агенты выполняли только действия, соответствующие их стандартным профилям поведения. В результате формировался набор событий audit-логов, отражающий нормальное функционирование системы и типичное взаимодействие пользователей с кластером.

Полученные события использовались для формирования обучающей и валидационной выборок:

- обучающая выборка $\mathcal{X}_{train}$ формировалась из основной части событий нормального поведения и использовалась для обучения параметров моделей нейронных сетей;
- валидационная выборка $\mathcal{X}_{val}$ выделялась из той же совокупности событий и применялась для контроля процесса обучения.

Поскольку рассматриваемая задача относится к задачам обнаружения аномалий, обучающая выборка преимущественно содержит события нормального поведения. Это позволяет модели сформировать представление о типичных паттернах взаимодействия пользователей с кластером.

2.9.4 Формирование тестовой выборки и внедрение аномалий

После формирования обучающей и валидационной выборок была сформирована тестовая выборка $\mathcal{X}_{test}$, предназначенная для оценки качества моделей обнаружения аномалий.

Для этого в профили поведения агентов были встроены дополнительные паттерны взаимодействия с кластером, которые нарушают заданные политики доступа или статистические паттерны нормального поведения. Эти действия имитировали аномальное поведение, потенциально связанное с нарушением безопасности или нетипичной активностью пользователя.

К таким паттернам могли относиться:

- доступ к нетипичным для роли ресурсам Kubernetes;
- выполнение операций, не характерных для конкретного агента;
- обращения к чувствительным ресурсам, таким как secrets;
- нетипичная последовательность операций;
- повышенная частота запросов к API-серверу.

События, сгенерированные при выполнении таких сценариев, помечались как аномальные. Таким образом, тестовая выборка содержала как нормальные, так и аномальные события, что позволило использовать её для объективной оценки качества моделей обнаружения аномалий.

2.9.5 Анализ полученного датасета

Ниже представлен анализ сформированного датасета audit-логов Kubernetes. Цель анализа — оценить пригодность данных для обучения моделей обнаружения аномалий, сбалансированность обучающей и тестовой выборок и соответствие распределений заданным профилям агентов. Для тестовой выборки приведён анализ внедрённых аномалий и их типов.

В таблице 1 приведён обзор обучающей, валидационной и тестовой выборок. Число событий и временной диапазон соответствуют сформированным подмножествам $\mathcal{X}_{\text{train}}$, $\mathcal{X}_{\text{val}}$ и $\mathcal{X}_{\text{test}}$. Тестовая выборка содержит поле разметки label (0/1) и долю аномальных событий.

Таблица 1 – Сводная таблица обучающей и тестовой выборки

Датасет	Событий	Начало (время)	Конец (время)	Разметка (label)	Доля аномалий
$\mathcal{X}_{\text{train}}$	140 305	2026-02-25 12:33:09	2026-03-11 12:29:37	Нет	—
$\mathcal{X}_{\text{train}}$	46 768	2026-02-25 12:33:09	2026-03-11 12:29:37	Нет	—
$\mathcal{X}_{\text{test}}$	45 237	2026-03-01 16:12:01	2026-03-05 16:11:41	Есть	5.31%

Для оценки взаимосвязей между категориальными признаками была построена матрица зависимости Крамера, приведенная на рисунке 14. Коэффициент Крамера (Cramér's V) является статистической мерой силы связи между категориальными признаками и принимает значения в диапазоне от 0 до 1. Значения, близкие к нулю, указывают на отсутствие статистической зависимости между признаками, тогда как значения, близкие к единице, свидетельствуют о сильной связи между ними [32].

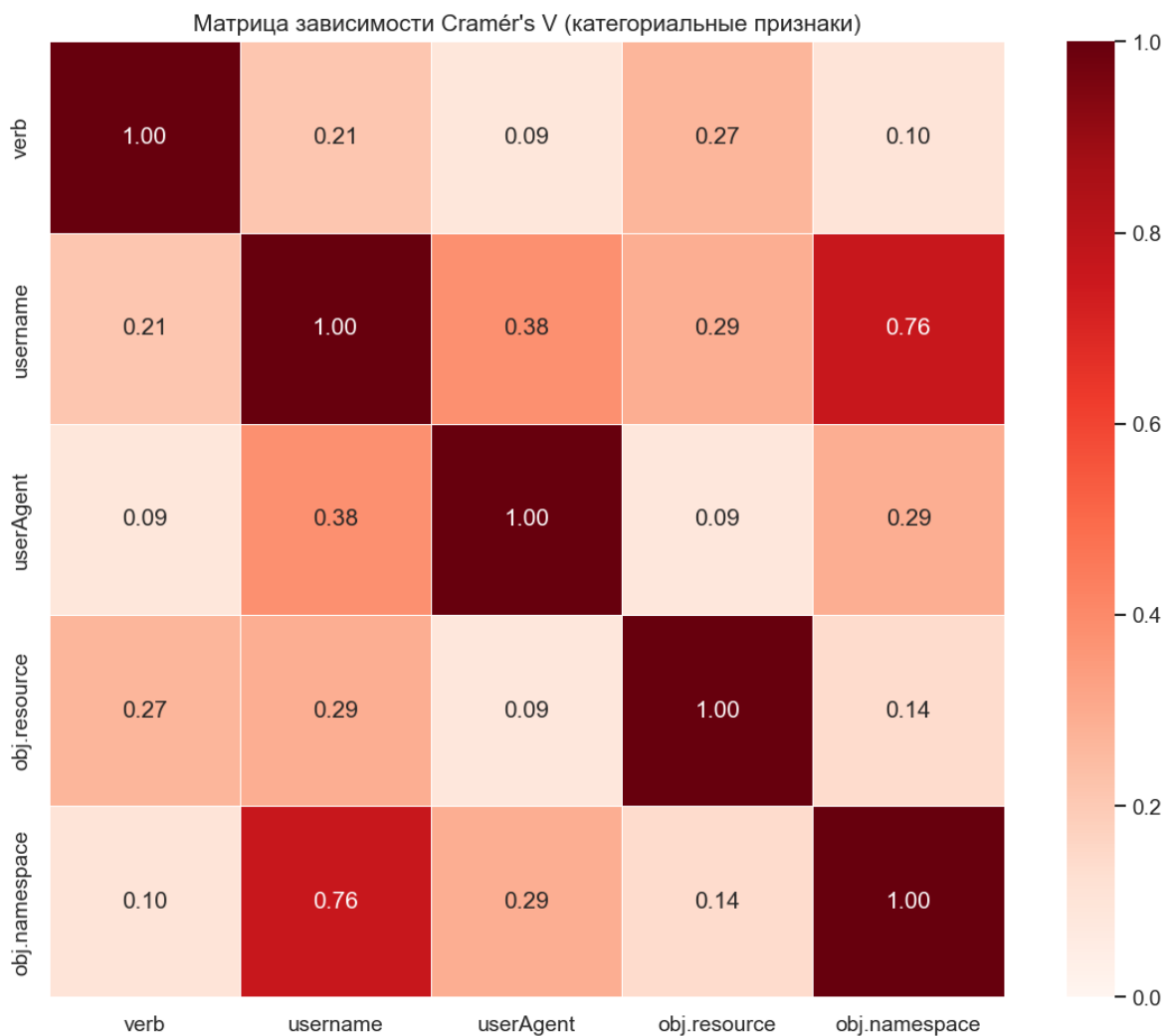


Рисунок 14 – Матрица зависимости Крамера.

Анализ матрицы зависимости показывает, что большинство признаков характеризуются слабой или умеренной степенью взаимосвязи. Это является ожидаемым результатом для аудит-логов Kubernetes, поскольку различные параметры события формируются независимо и отражают разные аспекты взаимодействия пользователя или компонента системы с API-сервером.

Наиболее сильная зависимость наблюдается между признаками `username` и `obj.namespace`, значение коэффициента Крамера для данной пары составляет 0.76. Это указывает на то, что конкретные пользователи или сервисные аккаунты чаще всего взаимодействуют с ограниченным набором пространств имён. Такое поведение соответствует модели разграничения доступа Kubernetes (RBAC), при которой пользователи и сервисные аккаунты обычно имеют доступ только к определённым пространствам имён.

Умеренная зависимость наблюдается между признаками `username` и `userAgent` (0.38). Это может быть связано с тем, что различные пользователи или автоматизированные процессы используют характерные клиенты для взаимодействия с API Kubernetes, например утилиту `kubectl`, CI/CD системы или встроенные компоненты кластера.

2.10 Решения математической постановки задачи

В данном разделе рассматриваются методы и подходы, используемые для решения сформулированной математической задачи обнаружения аномалий в Kubernetes-кластере. На основе ранее выполненного анализа существующих методов и особенностей предметной области определяется практическая реализация предложенной модели, включая этапы подготовки данных, выбора архитектуры нейронной сети, обучения модели и оценки её качества.

В рамках данного раздела подробно описываются этапы обучения различных архитектур НС и их сравнение.

2.10.1 Разработка и обучение моделей нейронных сетей для обнаружения аномалий

На основе подготовленных данных разрабатываются модели нейронных сетей, предназначенные для обнаружения аномалий в поведении пользователей и сервисных аккаунтов Kubernetes-кластера.

Для каждой архитектуры выполняется обучение модели на обучающей выборке и настройка её параметров с использованием валидационной выборки. Процесс обучения направлен на минимизацию функции потерь, характеризующей способность модели корректно описывать нормальное поведение пользователей и выявлять аномальные события.

Для достижения этой цели рассматриваются три класса нейронных сетей:

- Автокодировщики (Autoencoder, AE).
- Рекуррентные нейронные сети (RNN) на основе Autoencoder.
- Долгосрочная краткосрочная память (LSTM) на основе Autoencoder.

Autoencoder

В данной работе автокодировщик используется для обнаружения аномалий в событиях audit-логов Kubernetes. После предобработки каждое событие представляется вектором признаков $x \in \mathbb{R}^d$, который подаётся на вход модели. Автокодировщик обучается восстанавливать нормальные события.

Функция потерь, используемая при обучении модели, определяется среднеквадратичной ошибкой, представленной в формуле 36.

$$\mathcal{L}_{MSE} = \frac{1}{d} \|x_i - \tilde{x}_i\|_2^2 \quad (36)$$

После обучения модели для каждого события вычисляется показатель аномальности, определённый в формуле 37.

$$s_i = \|x_i - \tilde{x}_i\|_2^2 \quad (37)$$

Чем выше значение s_i , тем сильнее событие отклоняется от нормального поведения.

Для обучения автокодировщика используется обучающая выборка $\mathcal{X}_{train}$, содержащая события нормального поведения пользователей Kubernetes-кластера. Валидационная выборка $\mathcal{X}_{val}$ применяется для контроля процесса обучения и предотвращения переобучения. Обучение выполняется итерационно с использованием алгоритма обратного распространения ошибки. На каждой эпохе вычисляется значение функции потерь на обучающей и валидационной выборках, что позволяет отслеживать динамику обучения модели.

Для оптимизации параметров сети используется адаптивный градиентный метод оптимизации Adam, обеспечивающий устойчивую и быструю сходимость. В процессе обучения применяются методы регуляризации, такие как Dropout и ранняя остановка (Early Stopping).

На рисунке 15 представлен график изменения функции потерь на обучающей и валидационной выборках.

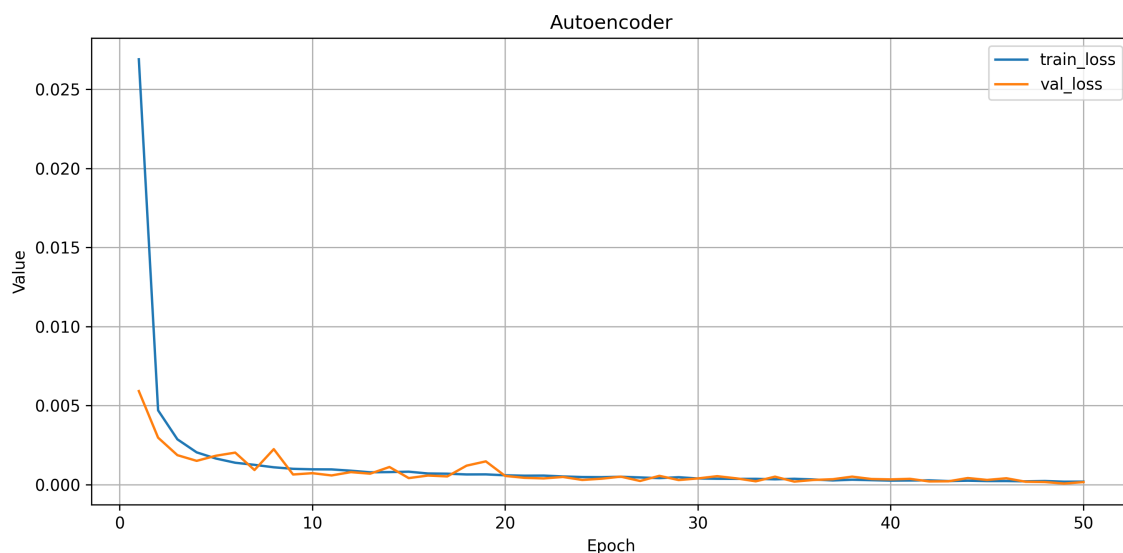


Рисунок 15 – Изменение функции потерь Autoencoder в процессе обучения.

После завершения обучения для событий тестовой выборки $\mathcal{X}_{test}$ вычисляется ошибка реконструкции. Для разделения нормальных и аномальных событий вводится пороговое значение τ , как показано в формуле 38.

$$y_i = \begin{cases} 1, & \text{если } s_i > \tau, \\ 0, & \text{иначе.} \end{cases} \quad (38)$$

На рисунке 16 показано распределение ошибки реконструкции для нормальных и аномальных событий.

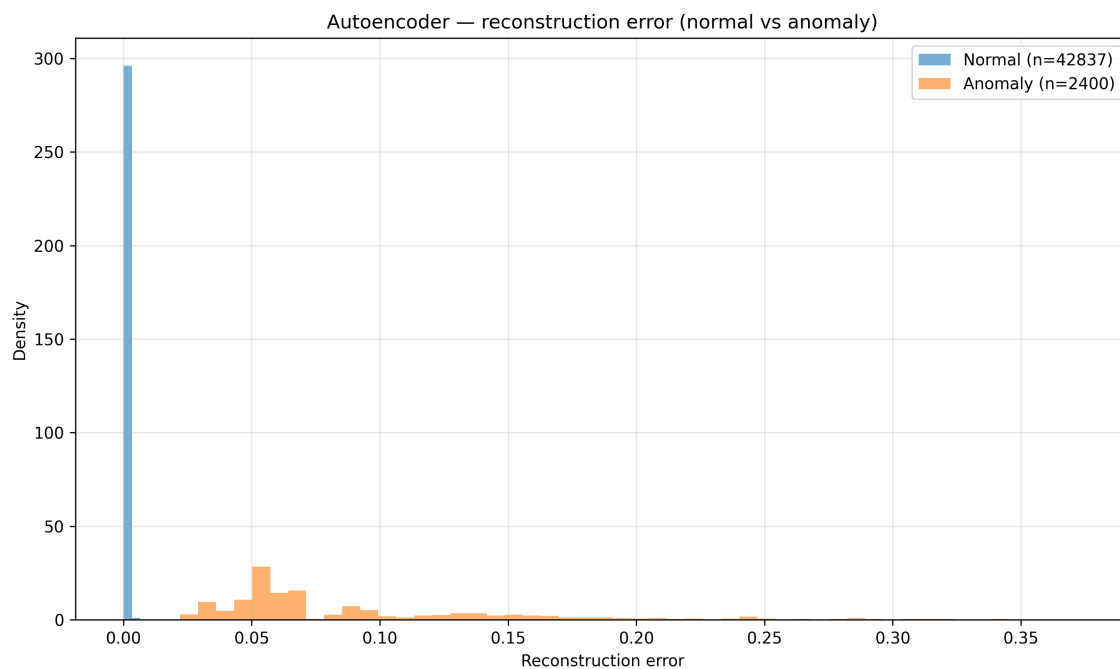


Рисунок 16 – Распределение ошибки реконструкции для модели Autoencoder.

Для количественной оценки качества обнаружения аномалий используется метрика площади под PR-кривой (PR-AUC), является предпочтительной при сильном дисбалансе классов.

На рисунке 17 представлена PR-кривая для модели Autoencoder.

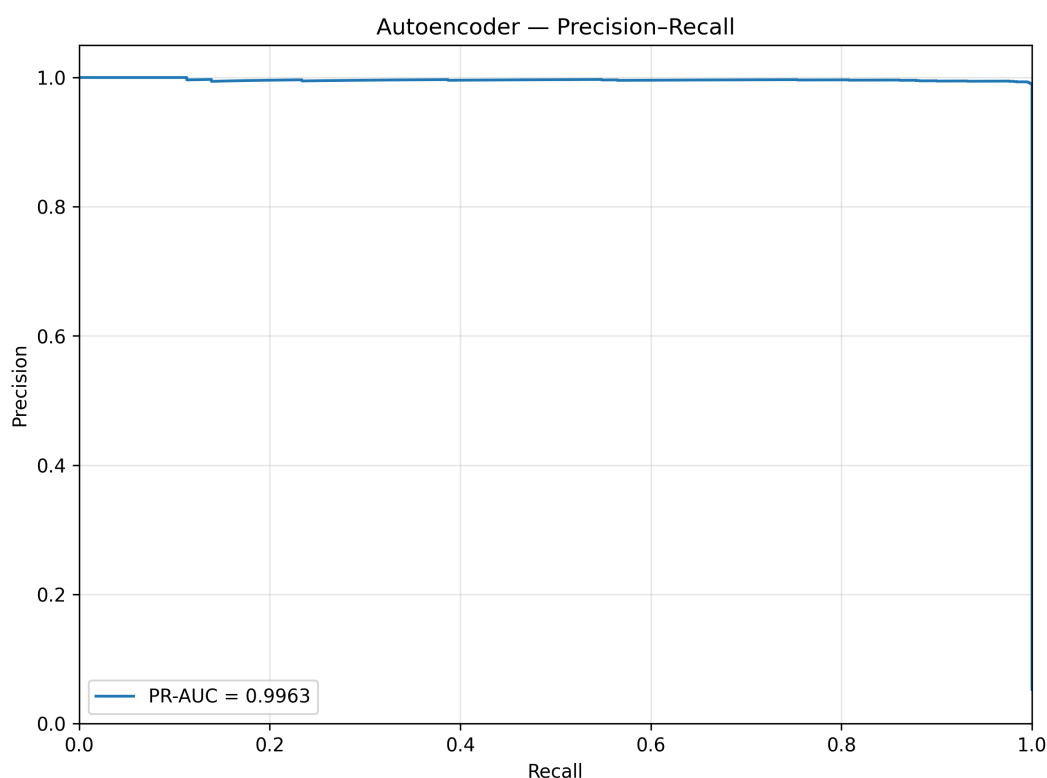


Рисунок 17 – PR-кривая для модели Autoencoder.

RNN-Autoencoder

В данной работе RNN используется в архитектуре *autoencoder* (RNN-AE), где слои *encoder* и *decoder* состоят из RNN-блоков. Такая структура позволяет моделировать последовательности событий audit-логов и выявлять аномалии, связанные с нетипичным поведением пользователей или сервисных аккаунтов.

На рисунке 18 приведена архитектура RNN-Autoencoder, состоящая из двух частей:

- **Encoder** — один или несколько RNN-слоёв, преобразующих последовательность входных векторов x_t в фиксированный вектор контекста z , содержащий сводную информацию о последовательности событий;
- **Decoder** — RNN-слои, которые на основе вектора z восстанавливают исходную последовательность событий $\tilde{x}_t$.

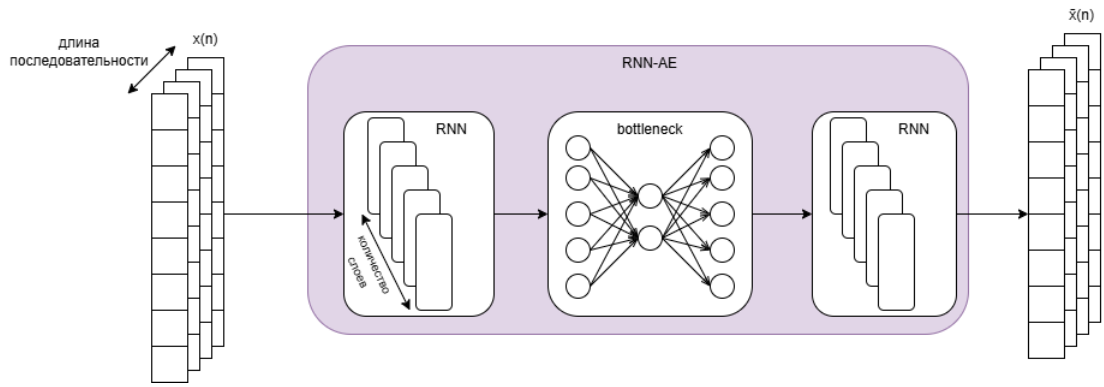


Рисунок 18 – Архитектура RNN-Autoencoder.

Функция потерь, используемая для обучения модели, определяется среднеквадратичной ошибкой, представленной в формуле 39.

$$\mathcal{L}_{MSE} = \frac{1}{T} \sum_{t=1}^T \frac{1}{d} \|x_{it} - \tilde{x}_{it}\|_2^2 \quad (39)$$

где T - длина последовательности, подающаяся в модель.

После обучения модели показатель аномальности вычисляется по формуле 40.

$$s_t = \frac{1}{T} \sum_{t=1}^T \|x_{it} - \tilde{x}_{it}\|_2^2 \quad (40)$$

где s_t — показатель аномальности для события t . События с s_t , превышающим пороговое значение, считаются аномальными.

На рисунке 19 представлен график изменения функции потерь модели RNN-AE на обучающей и валидационной выборках.

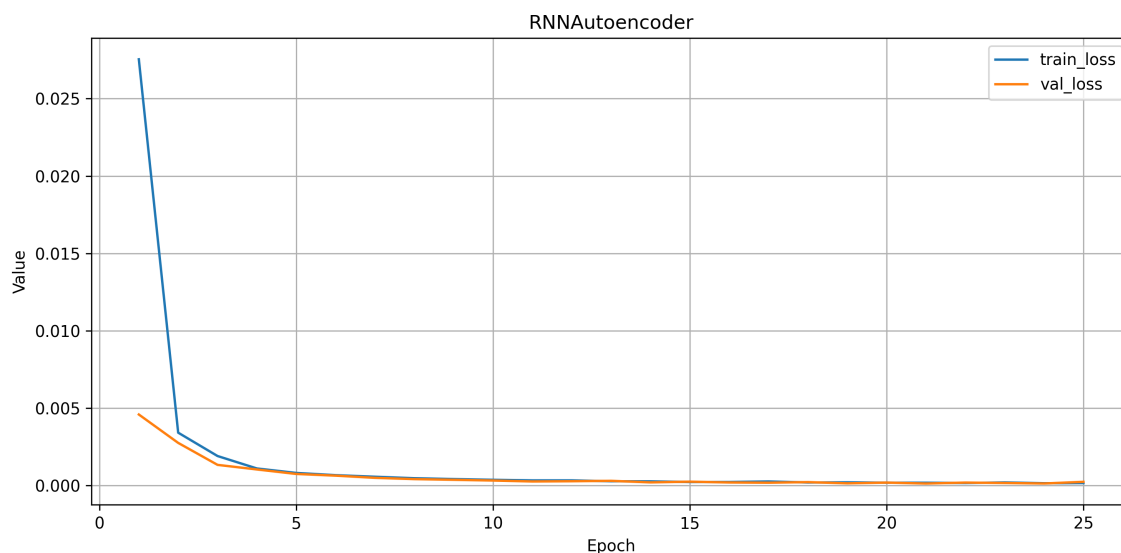


Рисунок 19 – Изменение функции потерь модели RNN-AE в процессе обучения.

После завершения обучения для каждого события тестовой выборки вычисляется показатель аномальности s_t , основанный на величине ошибки реконструкции.

На рисунке 20 показано распределение ошибки реконструкции для нормальных и аномальных событий. Как правило, аномальные события характеризуются более высокими значениями ошибки, что позволяет использовать этот показатель для их обнаружения.

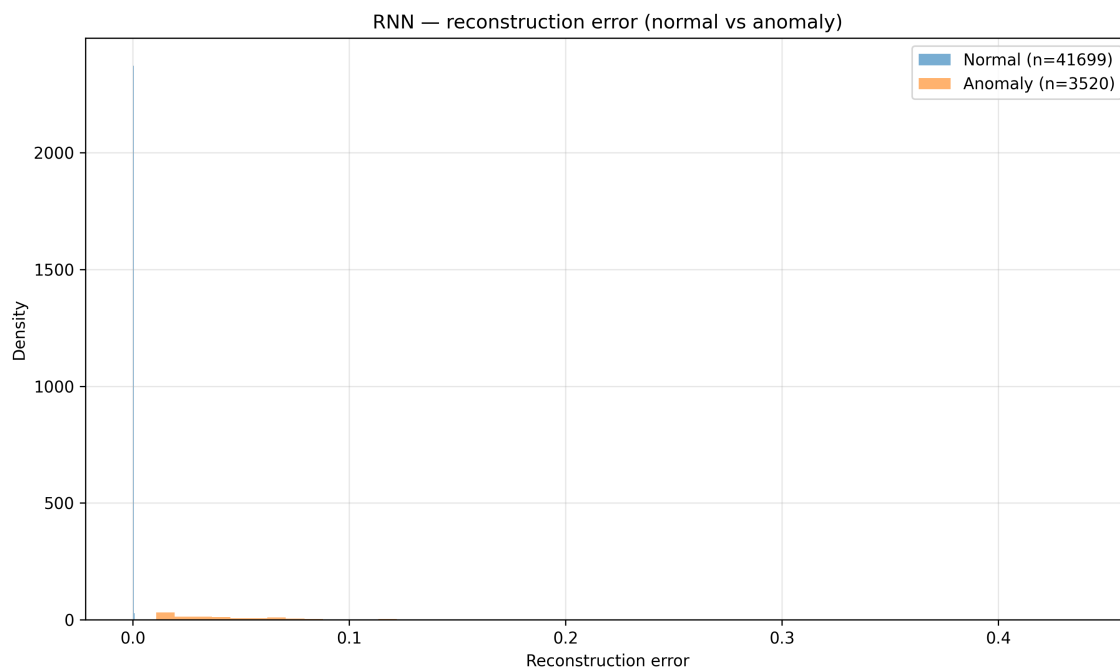


Рисунок 20 – Распределение ошибки реконструкции для модели RNN-AE.

Для количественной оценки качества обнаружения аномалий используется метрика площади под PR-кривой (PR-AUC), которая особенно полезна при сильном дисбалансе классов.

На рисунке 21 представлена PR-кривая для модели RNN-AE.

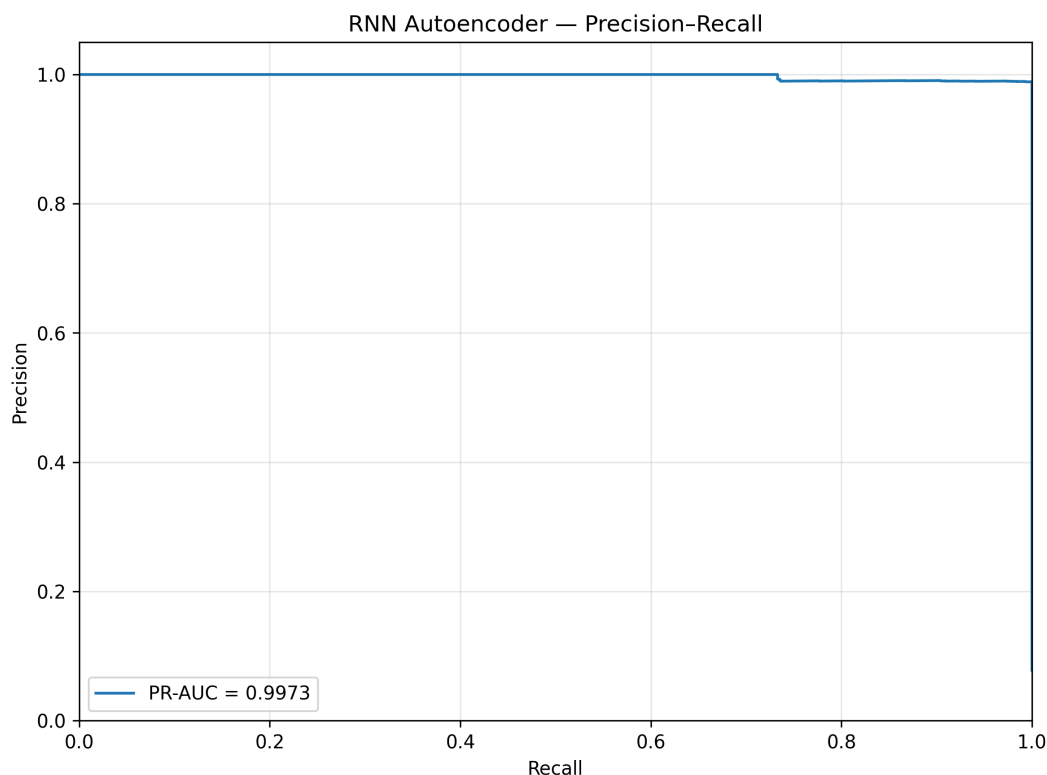


Рисунок 21 — PR-кривая для модели RNN-AE.

LSTM-Autoencoder

В данной работе LSTM применяется в архитектуре *autoencoder* (LSTM-AE), где слои *encoder* и *decoder* состоят из LSTM-блоков. Такая структура позволяет учитывать временную последовательность событий audit-логов и выявлять аномальные паттерны поведения пользователей и сервисных аккаунтов.

На рисунке 22 приведена архитектура LSTM-AE, состоящая из двух частей:

- **Encoder** — один или несколько LSTM-слоёв, преобразующих последовательность входных векторов x_t в фиксированный вектор контекста z , содержащий сводную информацию о последовательности событий;
- **Decoder** — LSTM-слои, которые на основе вектора z восстанавливают исходную последовательность событий $\tilde{x}_t$.

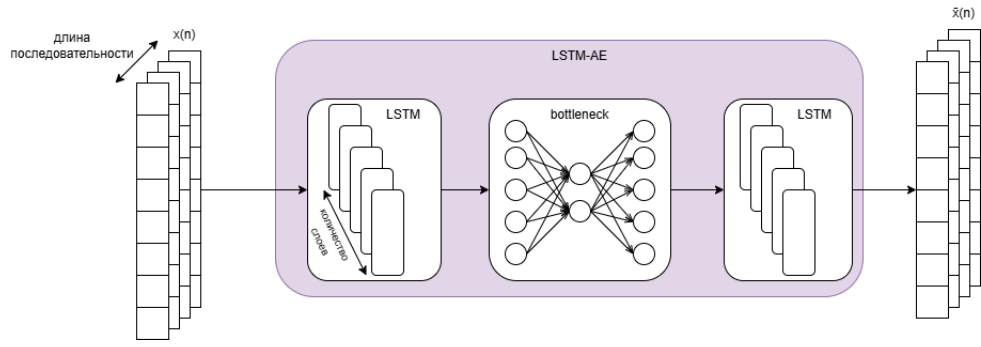


Рисунок 22 – Архитектура LSTM-Autoencoder.

Функция потерь для обучения LSTM-AE определяется среднеквадратичной ошибкой, представленной в формуле 41.

$$\mathcal{L}_{MSE} = \frac{1}{T} \sum_{t=1}^T \frac{1}{d} \|x_{it} - \tilde{x}_{it}\|_2^2 \quad (41)$$

где T - длина последовательности, подающаяся в модель.

После обучения модели показатель аномальности вычисляется по формуле 42.

$$s_t = \frac{1}{T} \sum_{t=1}^T \|x_{it} - \tilde{x}_{it}\|_2^2 \quad (42)$$

где s_t — показатель аномальности для события t . События с s_t выше порогового значения считаются аномальными.

На рисунке 23 приведён график изменения функции потерь LSTM-AE на обучающей и валидационной выборках.

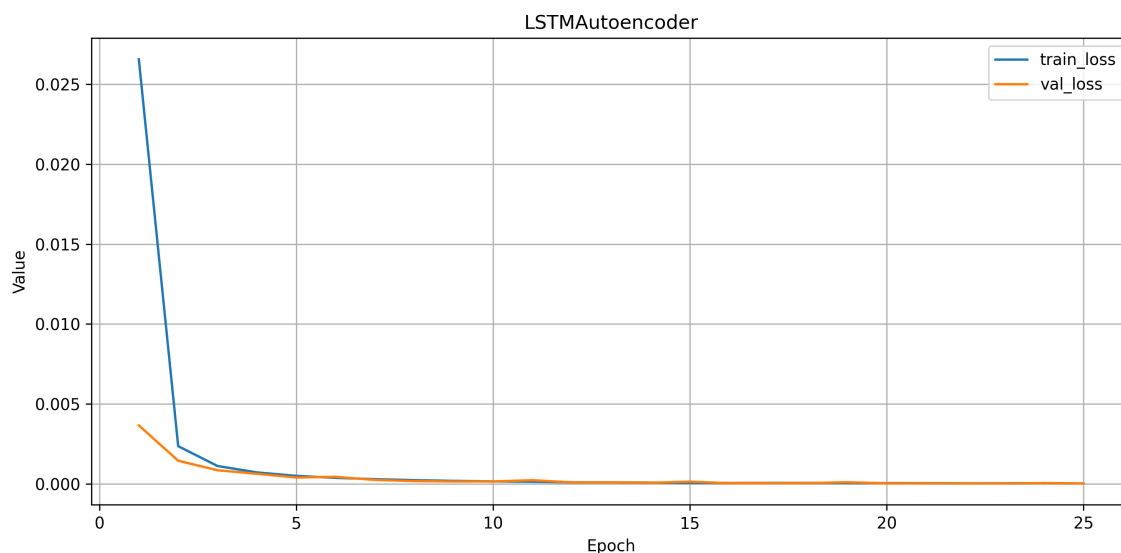


Рисунок 23 – Изменение функции потерь LSTM-AE на обучающей и валидационной выборках.

После завершения обучения для каждого события тестовой выборки вычисляется показатель аномальности s_t , основанный на величине ошибки реконструкции.

На рисунке 24 показано распределение ошибки реконструкции для нормальных и аномальных событий. Видно, что аномальные события обычно характеризуются более высокими значениями ошибки, что позволяет использовать этот показатель для их обнаружения.

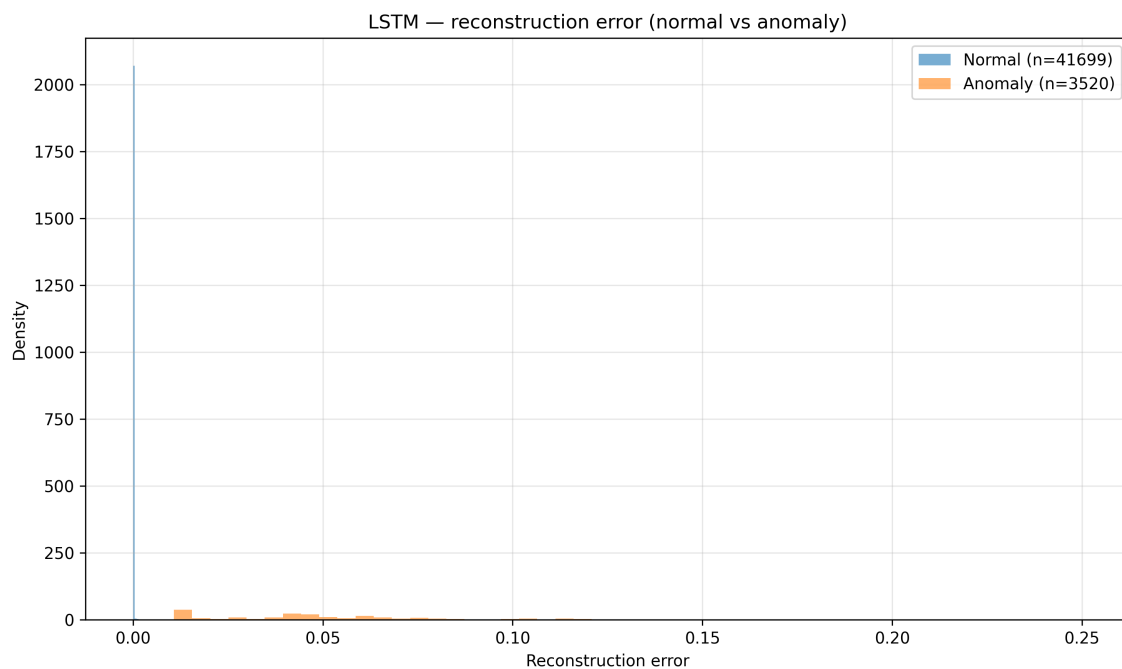


Рисунок 24 – Распределение ошибки реконструкции для нормальных и аномальных событий.

На рисунке 25 приведена PR-кривая, отражающая зависимость точности (Precision) от полноты (Recall) при изменении порога обнаружения.

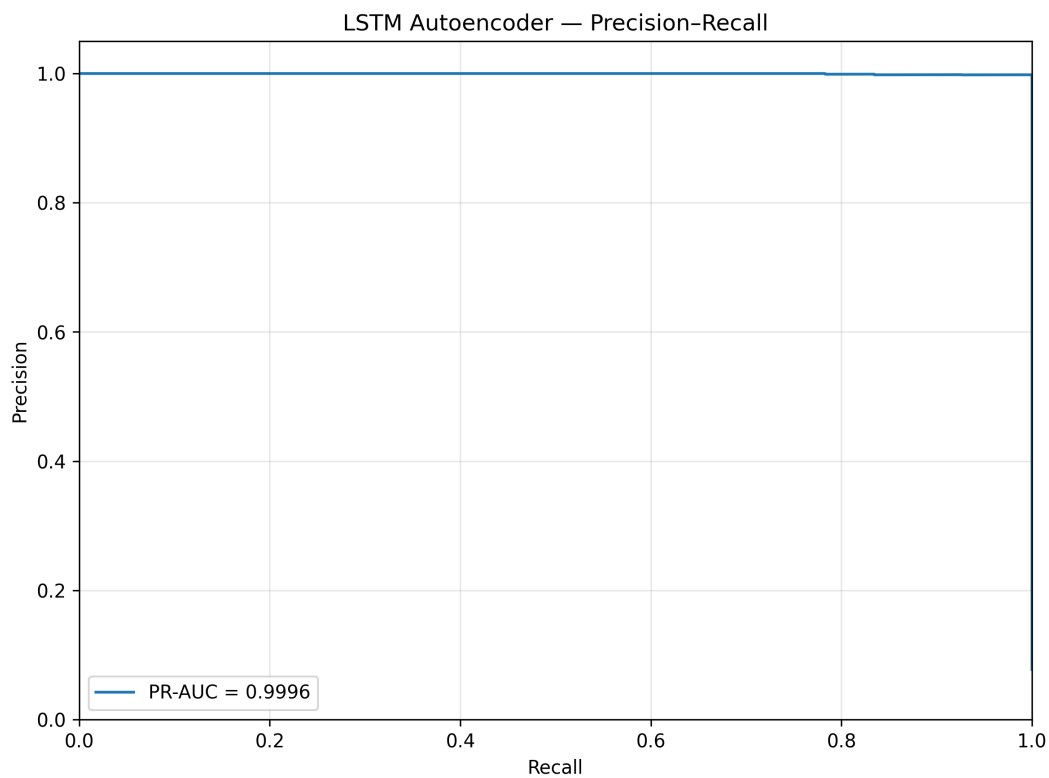


Рисунок 25 – PR-кривая для модели LSTM-AE.

2.10.2 Сравнительный анализ моделей и выбор оптимальной архитектуры

После обучения моделей различных архитектур проводится их сравнительный анализ по метрикам качества обнаружения аномалий. Основной задачей данного этапа является определение архитектуры нейронной сети, обеспечивающей наилучшие результаты обнаружения аномалий на тестовой выборке.

Каждая модель обучалась на одинаковой обучающей выборке, содержащей преимущественно нормальные события, и оценивалась на тестовой выборке, включающей как нормальные, так и аномальные события.

В качестве основной метрики оценки качества использовалась площадь под кривой Precision–Recall (PR-AUC), которая является наиболее информативной при сильном дисбалансе классов, характерном для задачи обнаружения аномалий.

Дополнительно анализировались следующие метрики:

- **ROC-AUC** — площадь под ROC-кривой;
- **Precision** — точность обнаружения аномалий;
- **Recall** — полнота обнаружения;
- **F1-score** — гармоническое среднее точности и полноты.

В таблице 2 представлены результаты оценки моделей на тестовой выборке.

Таблица 2 – Сравнение моделей по метрикам качества

Модель	ROC-AUC	PR-AUC	Precision	Recall	F1
Autoencoder	0.9998	0.9963	0.914	1.00	0.955
RNN-AE	0.9998	0.9973	0.935	1.00	0.967
LSTM-AE	0.9999	0.9996	0.931	1.00	0.964

Как видно из результатов, все модели демонстрируют высокое качество обнаружения аномалий. Однако между ними наблюдаются различия:

- классический Autoencoder показывает наименьшее значение PR-AUC среди рассмотренных моделей;
- RNN-AE демонстрирует улучшение качества за счёт учета последовательной структуры данных;
- LSTM-AE достигает наилучшего значения PR-AUC и ROC-AUC, что свидетельствует о более точном разделении нормальных и аномальных событий.

Важно отметить, что все модели достигают максимального значения полноты ($\text{Recall} = 1.0$), что означает отсутствие пропущенных аномалий. При этом различия проявляются в количестве ложных срабатываний, отражаемых метрикой Precision.

На основании проведённого сравнительного анализа в качестве оптимальной модели была выбрана архитектура **LSTM-Autoencoder**.

Данный выбор обусловлен следующими факторами:

- наивысшее значение PR-AUC, что критично для задач с дисбалансом классов;
- способность эффективно моделировать долгосрочные зависимости в последовательностях событий;
- более чёткое разделение распределений ошибки реконструкции для нормальных и аномальных событий.

Несмотря на то, что RNN-AE показывает сопоставимое значение F1-метрики, использование LSTM позволяет лучше учитывать сложные временные зависимости, характерные для audit-логов Kubernetes. Таким образом, LSTM-AE обеспечивает наилучший баланс между точностью обнаружения и устойчивостью модели, что делает её наиболее подходящей для практического применения в задаче обнаружения аномалий.

2.11 Выводы

В конструкторской части выпускной квалификационной работы была последовательно решена задача разработки средства обнаружения аномалий в Kubernetes-кластере на основе нейронных сетей. На первом этапе был выполнен обзор и анализ существующих подходов и методов обнаружения аномалий в контейнерных средах, в ходе которого были рассмотрены несколько современных подходов, используемых в кластерах Kubernetes.

В рамках моделирования подхода был исследован принцип обнаружения аномалий на основе анализа audit-логов, а также выбран подход DeepLog как удачно зарекомендовавшая себя архитектура для работы с последовательностями событий. Это позволило подтвердить применимость рекуррентных моделей для анализа логов Kubernetes и обосновать дальнейший выбор нейросетевых архитектур для решения поставленной задачи.

Была разработана архитектура средства обнаружения аномалий, включающая этапы сбора audit-логов. Была сформулирована математическая постановка задачи, определены входные данные, критерии обучения и метрики качества, а также описан принцип выбора оптимальной архитектуры по значению PR-AUC в условиях сильного дисбаланса классов.

В разделе, посвящённом решению математической постановки задачи, были детально описаны процедуры предобработки audit-логов, кодирования признаков, формирования обучающей, валидационной и тестовой выборок, а также обучения и сравнения моделей AE, RNN-AE и LSTM-AE. Проведённый сравнительный анализ показал, что все рассмотренные модели демонстрируют высокое качество обнаружения аномалий, однако наилучшие результаты по совокупности метрик показала архитектура LSTM-AE, обеспечившая наибольшее значение PR-AUC и наиболее устойчивое разделение нормальных и аномальных событий.

Таким образом, в конструкторской части была обоснована и разработана архитектура средства обнаружения аномалий в Kubernetes-кластере, выбраны методы обработки данных и нейросетевая модель, наиболее подходящая для анализа последовательностей audit-логов. Полученные результаты создают основу для последующей программной реализации разработанного решения и его экспериментальной проверки.

3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

3.1 Применяемые технологии

Разработка программного средства обнаружения аномалий в Kubernetes-кластере осуществлялась с использованием современного стека технологий, ориентированного на создание масштабируемых распределённых систем и обработку данных с применением методов машинного обучения.

В качестве основного языка программирования был выбран Python [33], который широко применяется при разработке систем анализа данных и машинного обучения благодаря развитой экосистеме библиотек и инструментов.

Для реализации серверной части приложения использован фреймворк FastAPI [34], обеспечивающий высокую производительность за счёт поддержки асинхронного ввода-вывода и удобных инструментов для разработки REST API. Запуск приложения осуществляется с помощью ASGI-сервера Uvicorn [35], позволяющего эффективно обрабатывать большое количество одновременных запросов.

Для работы с базой данных применена библиотека SQLAlchemy [36], реализующая объектно-реляционное отображение и обеспечивающая удобное взаимодействие с реляционной СУБД PostgreSQL [37]. В качестве драйвера подключения используется asyncpg [38], что позволяет организовать асинхронную работу с базой данных и повысить общую производительность системы.

Организация фоновых вычислений реализована с использованием системы распределённых задач Celery [39]. Данный инструмент позволяет выносить ресурсоёмкие операции, такие как обучение нейросетевых моделей и выполнение задач детектирования, в отдельные worker-процессы. Это обеспечивает асинхронность выполнения и возможность горизонтального масштабирования системы.

Для реализации модели обнаружения аномалий использована библиотека PyTorch [40], предоставляющая гибкие средства для построения и обучения нейронных сетей, включая рекуррентные архитектуры, такие как LSTM. Для выполнения вспомогательных задач предобработки данных и оценки качества моделей применяется библиотека scikit-learn, широко используемая для решения задач машинного обучения [41].

В качестве источника данных используется система OpenSearch [42], из которой извлекаются audit-логи Kubernetes-кластера. Для взаимодействия с данным хранилищем применяется клиентская библиотека `opensearch-py` [43], обеспечивающая выполнение запросов и получение данных для последующего анализа.

Выбранный стек технологий обеспечивает необходимый уровень производительности, расширяемости и удобства разработки, а также позволяет эффективно реализовать распределённую систему обнаружения аномалий, работающую с потоками событий в Kubernetes-кластере.

3.2 Реализация процессов

Разработанное программное средство включает набор взаимосвязанных процессов, обеспечивающих полный цикл работы системы обнаружения аномалий: от взаимодействия с пользователем и источником данных до обучения модели и выполнения детектирования. Общая схема основных процессов программного средства представлена на рисунке 26.

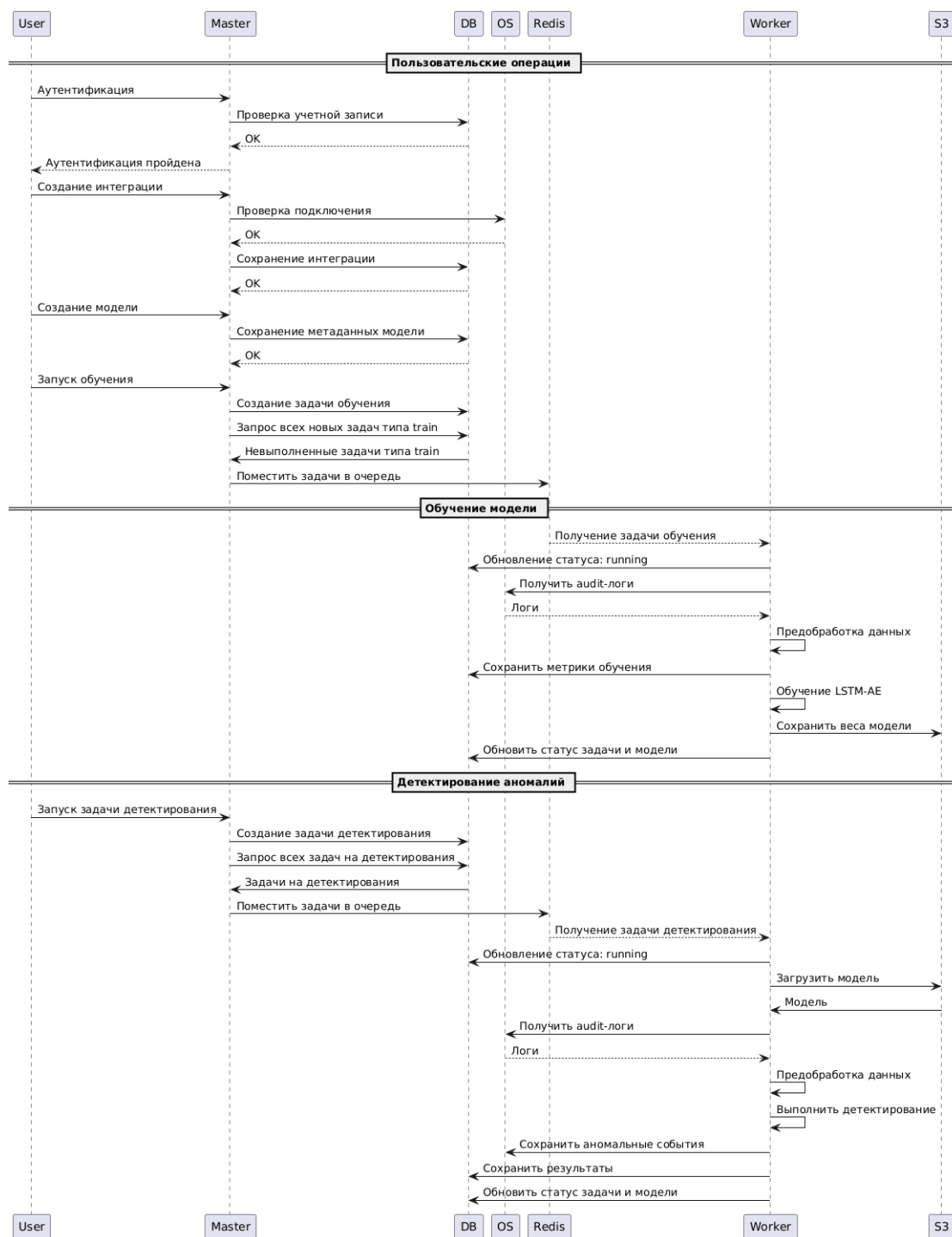


Рисунок 26 – Общая схема процессов программного средства

Основные процессы системы можно разделить на две группы. К первой группе относятся процессы, связанные с управлением пользователями и настройкой системы: регистрация, аутентификация, создание интеграции с OpenSearch и создание модели обнаружения аномалий. Данные процессы

выполняются через центральный компонент системы и не требуют запуска ресурсоёмких вычислений. Их результатом является формирование записей в базе данных и подготовка параметров для последующего создания задач на обучение или детектирование аномалий.

Ко второй группе относятся процессы, связанные с обработкой данных и работой НС. К ним относятся запуск обучения модели, запуск режима обнаружения аномалий, получение данных из OpenSearch, формирование признакового описания и вычисление итоговых метрик. Данные операции выполняются асинхронно с использованием worker-компонентов и очереди задач Redis, что позволяет не блокировать основной поток обработки пользовательских запросов.

Процесс аутентификации начинается с обращения пользователя к компоненту Master, который выполняет проверку учётной записи в базе данных PostgreSQL. После успешной проверки система возвращает результат аутентификации пользователю. Аутентификация реализуется на основе проверки логина и пароля с последующим формированием токена доступа, который используется для обращения к защищённым методам API.

Процесс создания интеграции с OpenSearch включает указание параметров подключения, таких как адрес сервера, имя пользователя, пароль и имя индекса. После сохранения параметров выполняется проверка корректности подключения к источнику данных. В случае успешной проверки информация об интеграции сохраняется в базе данных и становится доступной для последующего использования при обучении модели и детектировании аномалий.

Аналогичным образом при создании модели Master формирует и сохраняет метаданные модели, соответствующие этапу её жизненного цикла.

3.3 Обучение модели НС

Процесс обучения модели является одним из ключевых в разработанном программном средстве. Инициация обучения осуществляется пользователем через API центрального компонента системы. При этом создаётся задача типа *train* со статусом *starting*, информация о которой сохраняется в базе данных. Функции планирования выполнения задач реализованы в составе компонента

Master. Планировщик периодически выполняет опрос базы данных с целью получения новых задач обучения, не отправленных ранее в очередь. Для таких задач статус изменяется на *pending*, после чего они помещаются в очередь сообщений Redis для последующей асинхронной обработки.

Worker-компонент извлекает задачу из очереди, переводит её в состояние *running* и приступает к выполнению. На первом этапе осуществляется загрузка audit-логов из OpenSearch. Полученные данные проходят процедуру предобработки, включающую очистку, нормализацию и формирование последовательностей признаков фиксированной длины. На следующем этапе выполняется обучение модели на основе архитектуры LSTM-AE с использованием подготовленных данных. В процессе обучения вычисляются метрики, характеризующие эффективность модели, которые сохраняются в БД. По завершении обучения веса модели сохраняются во внешнем хранилище.

После успешного завершения всех этапов модель переводится в состояние *ready*, что делает её доступной для использования в задачах обнаружения аномалий. Статус соответствующей задачи обновляется на *done*, что свидетельствует о завершении процесса обучения.

Блок-схема алгоритма обучения приведена на рисунке 27.

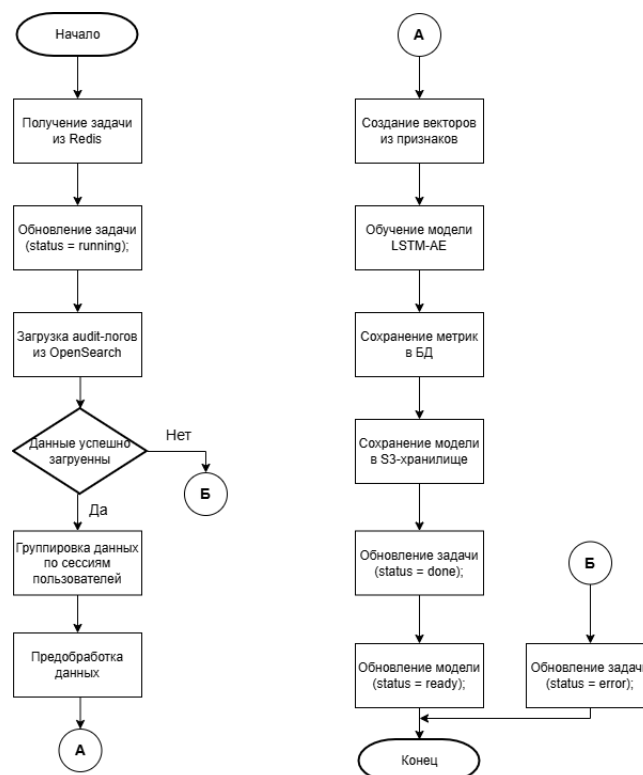


Рисунок 27 – Блок-схема алгоритма обучения

3.4 Обнаружение аномалий

Процесс обнаружения аномалий реализуется по аналогичному принципу. Пользователь инициирует выполнение задачи типа *detect* со статусом *starting*, информация о которой сохраняется в базе данных. Планировщик периодически выполняет запрос к базе данных для получения новых задач обнаружения аномалий со статусом *starting*, которые ранее не были отправлены в очередь. Для таких задач статус изменяется на *pending*, после чего они помещаются в очередь сообщений Redis для последующей обработки.

Worker-компонент извлекает задачу из очереди и переводит её в состояние *running*. На первом этапе осуществляется получение актуальных audit-логов из OpenSearch. Далее данные преобразуются в формат, совместимый с входом обученной модели, включая предобработку и формирование последовательностей признаков.

После подготовки данных выполняется вычисление показателя аномальности с использованием обученной модели LSTM-AE. В случае, если значение ошибки превышает заданный порог, соответствующее событие классифицируется как аномальное. Все аномальные события сохраняются в отдельном индексе OpenSearch

Результаты анализа сохраняются в базе данных и используются для последующего отображения пользователю. После завершения обработки задача переводится в состояние *starting*, чтобы планировщик смог ее запланировать в очереди.

Блок-схема алгоритма обнаружения аномалий приведена на рисунке 28.

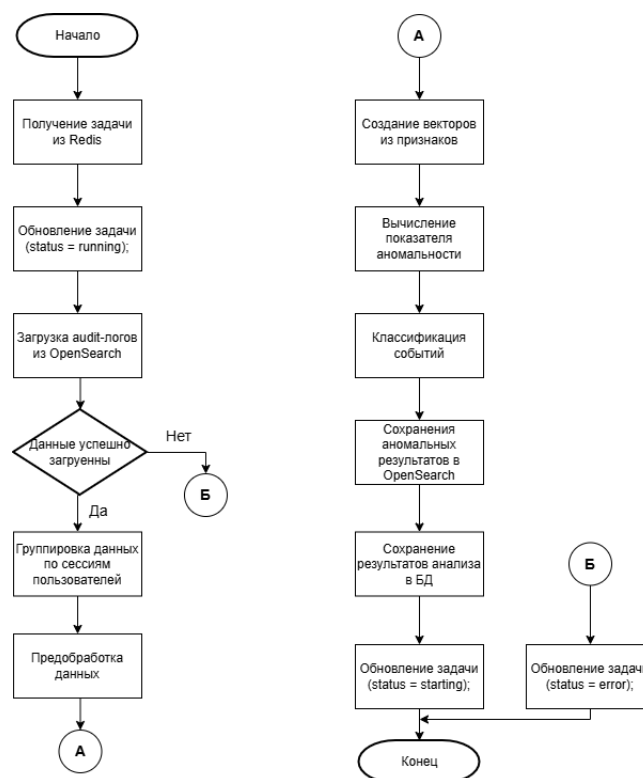


Рисунок 28 – Блок-схема алгоритма обнаружения аномалий

3.5 Реализация программного средства

Реализация программного средства обнаружения аномалий в логах Kubernetes выполнена в виде многомодульного приложения, построенного по принципам чистой архитектуры. Такой подход позволяет разделить ответственность между компонентами системы, повысить её расширяемость и упростить сопровождение.

Программное средство состоит из следующих основных компонентов:

- серверное приложение (backend);
- процесс фоновой обработки задач (worker);
- клиентский веб-интерфейс (frontend);
- инфраструктурные компоненты (база данных, очередь задач, хранилище логов).

Логическая структура приложения организована в виде слоёв:

- слой представления (frontend);
- слой API;
- прикладной слой (business logic);
- доменный слой;
- инфраструктурный слой.

Общая структура зависимостей модулей серверного приложения представлена на рисунке 29.

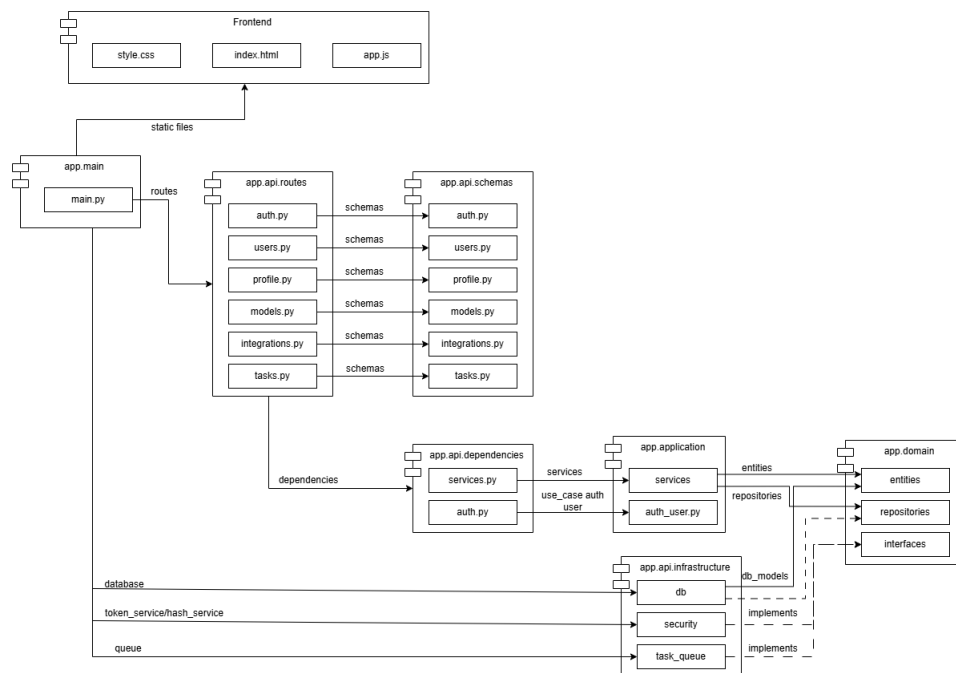


Рисунок 29 – UML-диаграмма зависимостей модулей основного приложения

Общая структура зависимостей модулей процесса фоновой обработки задач представлена на рисунке 30.

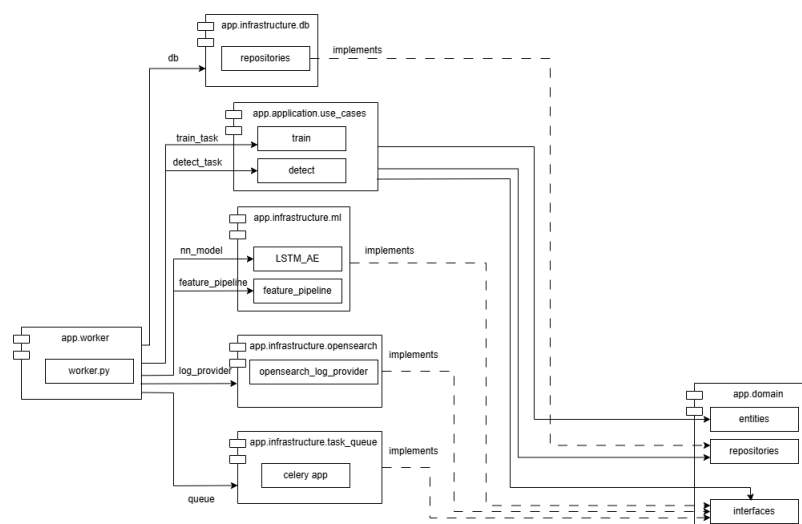


Рисунок 30 – UML-диаграмма зависимостей процесса фоновой обработки

Основным модулем системы является файл `main.py`, который выполняет инициализацию веб-приложения. В данном модуле создаётся экземпляр FastAPI-приложения, подключаются маршруты API, настраивается обработка ошибок, а также конфигурируются внешние зависимости, включая подключение к базе данных, систему аутентификации и очередь задач. Кроме того, данный модуль отвечает за раздачу статических файлов пользовательского интерфейса и настройку CORS, что обеспечивает корректное взаимодействие между клиентской и серверной частями.

Модули, расположенные в директории `app/api/routes/`, реализуют REST API приложения. Каждый модуль отвечает за отдельную функциональную область (аутентификация, управление пользователями, моделями, задачами и интеграциями). Маршруты принимают HTTP-запросы и передают их на обработку в прикладной слой.

Для описания структуры входных и выходных данных используются модули `app/api/schemas/`, реализующие схемы на основе Pydantic. Это обеспечивает строгую типизацию, валидацию данных и единый контракт взаимодействия с клиентом.

Прикладной слой представлен модулями `app/application/services/` и `app/application/use_cases/`. Данный слой реализует бизнес-логику системы и сценарии использования. Сервисы инкапсулируют операции над сущностями предметной области, такими как пользователи, модели и задачи. Сценарии (*use cases*) описывают конкретные процессы, включая:

- постановку задачи обучения модели;
- запуск процесса детектирования аномалий;
- управление жизненным циклом задач.

Ключевыми компонентами являются модули `train.py` и `detect.py`, реализующие обучение и применение моделей на основе LSTM-AE.

Модуль `worker.py` реализует отдельный процесс обработки фоновых задач с использованием Celery. Он выполняет ресурсоёмкие операции, такие как обучение моделей и анализ логов, вне основного HTTP-потoka. Такое разделение позволяет повысить отзывчивость системы и обеспечить масштабируемость за счёт распределённого выполнения задач.

Доменный слой содержит описание сущностей предметной области, интерфейсов репозитория и абстракций, используемых в системе. Он не зависит от конкретных технологий хранения данных или реализации моделей, что обеспечивает независимость бизнес-логики от инфраструктуры.

Инфраструктурный слой включает реализацию взаимодействия с внешними системами:

- модуль работы с базой данных (PostgreSQL);
- модуль взаимодействия с OpenSearch для получения audit-логов;
- модуль машинного обучения (реализация LSTM-AE);
- модуль очереди задач (Celery, Redis);
- модуль безопасности (JWT-аутентификация).

Данный слой реализует интерфейсы, определённые в доменном слое, обеспечивая тем самым связанность компонентов системы.

Клиентская часть реализована в виде одностраничного приложения, включающего файлы:

- `index.html` — структура интерфейса;
- `style.css` — оформление;
- `app.js` — логика взаимодействия с API.

Взаимодействие с сервером осуществляется посредством REST API. Клиентская логика обеспечивает отправку запросов, получение результатов и их визуализацию.

3.6 Взаимодействие модулей

Взаимодействие компонентов системы организовано следующим образом:

- 1) Пользователь взаимодействует с веб-интерфейсом, который формирует HTTP-запросы к серверу.
- 2) Запрос поступает в модуль маршрутов FastAPI, где происходит его валидация и передача в прикладной слой.
- 3) Прикладной слой инициирует выполнение соответствующего сценария, который может включать обращение к базе данных или постановку задачи в очередь.
- 4) В случае длительных операций (обучение модели или детектирование) создаётся задача в очереди Celery.
- 5) Процесс `worker.py` извлекает задачу из очереди, выполняет обработку данных, взаимодействует с OpenSearch и сохраняет результаты в базу данных.

6) Клиент периодически запрашивает статус задачи и получает результаты через API.

Таким образом, система реализует асинхронную модель взаимодействия, позволяющую эффективно обрабатывать большие объёмы данных и обеспечивать масштабируемость при увеличении нагрузки.

3.7 Тестирование программного средства

В данном разделе приводится демонстрация работы разработанного программного средства обнаружения аномалий в audit-логах Kubernetes-кластера. Рассматривается полный сценарий взаимодействия пользователя с системой — от настройки источника данных до получения результатов обнаружения аномалий.

Общий процесс работы с приложением включает следующие этапы:

- 1) аутентификация пользователя;
- 2) настройка интеграции с источником данных (OpenSearch);
- 3) создание модели обнаружения аномалий;
- 4) запуск обучения модели;
- 5) запуск задачи обнаружения аномалий;
- 6) анализ полученных результатов.

Взаимодействие пользователя с системой осуществляется с помощью web-интерфейса, представленного на рисунке 31.

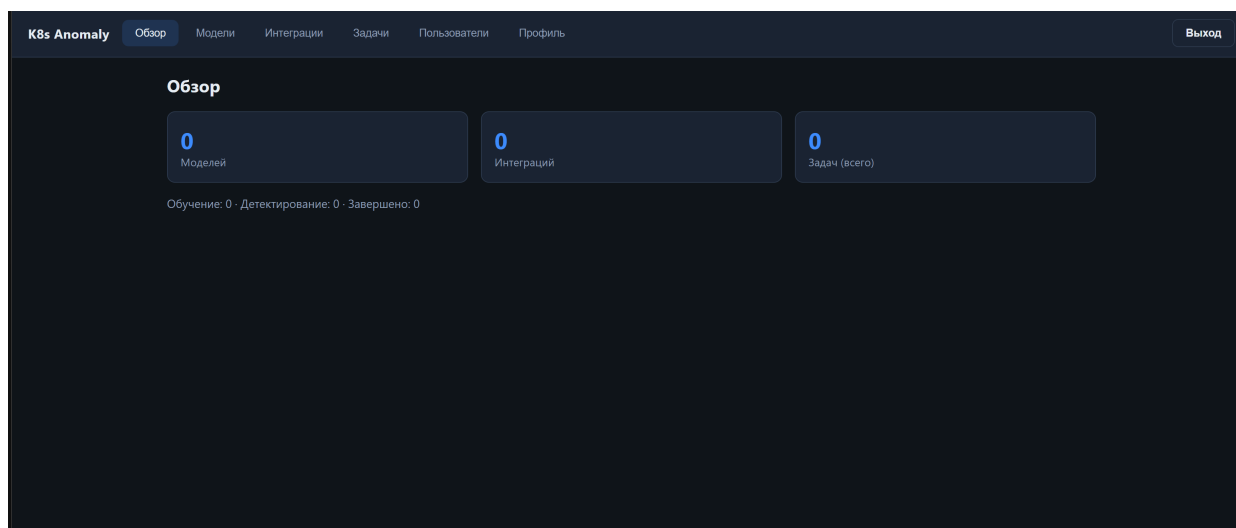
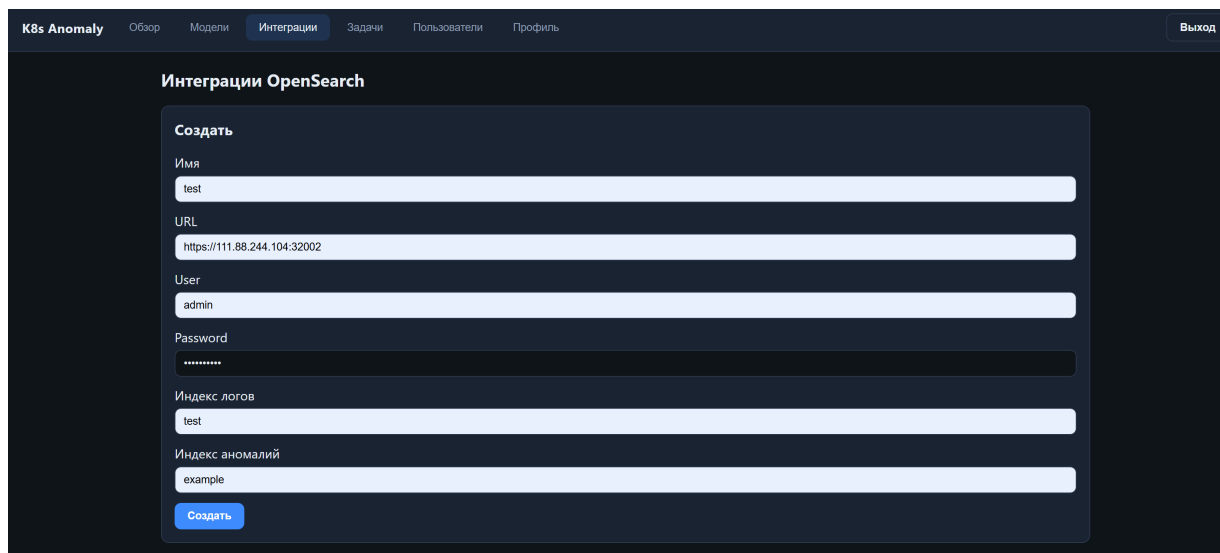


Рисунок 31 – Интерфейс взаимодействия пользователя

Данный интерфейс позволяет выполнять тестирование всех доступных методов системы, включая создание интеграций, моделей и запуск задач.

На первом этапе пользователь настраивает подключение к источнику audit-логов. Для этого через API передаются параметры подключения к OpenSearch: адрес сервера, учетные данные и имя индекса.

Пример выполнения запроса на создание интеграции представлен на рисунке 32.

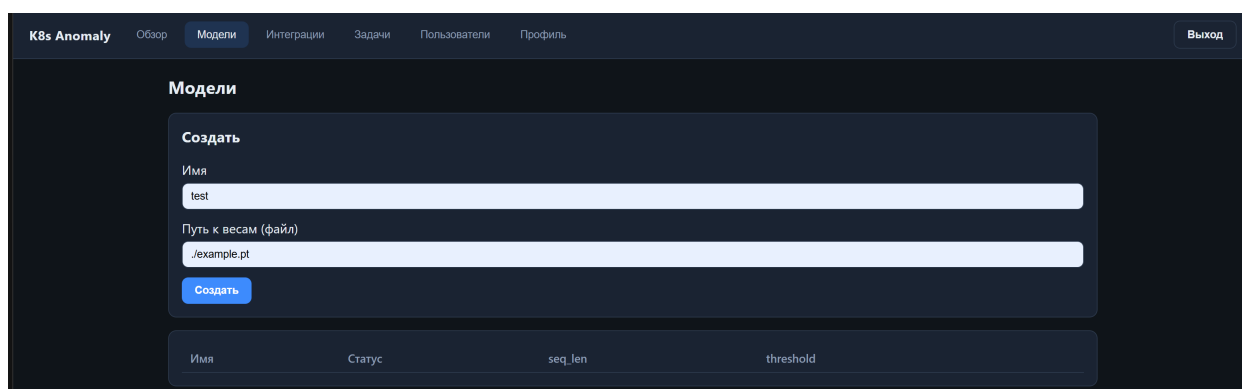


The screenshot shows the 'Интеграции OpenSearch' (OpenSearch Integrations) page in the K8s Anomaly application. The page has a dark theme and a navigation bar at the top with links: 'Обзор', 'Модели', 'Интеграции' (active), 'Задачи', 'Пользователи', 'Профиль', and a 'Выход' (Logout) button. The main content area is titled 'Интеграции OpenSearch' and contains a 'Создать' (Create) form. The form fields are: 'Имя' (Name) with value 'test', 'URL' with value 'https://111.88.244.104:32002', 'User' with value 'admin', 'Password' (masked with dots), 'Индекс логов' (Log Index) with value 'test', and 'Индекс аномалий' (Anomaly Index) with value 'example'. A blue 'Создать' button is at the bottom of the form.

Рисунок 32 – Создание интеграции с OpenSearch

После отправки запроса система выполняет проверку соединения и сохраняет параметры интеграции в базе данных.

После настройки источника данных пользователь создаёт модель обнаружения аномалий. Пример создания модели представлен на рисунке 33.



The screenshot shows the 'Модели' (Models) page in the K8s Anomaly application. The page has a dark theme and a navigation bar at the top with links: 'Обзор', 'Модели' (active), 'Интеграции', 'Задачи', 'Пользователи', 'Профиль', and a 'Выход' (Logout) button. The main content area is titled 'Модели' and contains a 'Создать' (Create) form. The form fields are: 'Имя' (Name) with value 'test', and 'Путь к весам (файл)' (Weights path (file)) with value './example.pt'. A blue 'Создать' button is at the bottom of the form. Below the form is a table with columns: 'Имя', 'Статус', 'seq. len', and 'threshold'.

Рисунок 33 – Создание модели обнаружения аномалий

После выполнения операции в базе данных создаётся запись о модели со статусом *created*. Обучение модели инициируется пользователем после создания задачи типа *train*, которая помещается в очередь задач. Пример запуска обучения представлен на рисунке 34.

K8s Anomaly
Обзор
Модели
Интеграции
Задачи
Пользователи
Профиль
Выход

Задачи

Создать

Тип

train

Модель

test

Интеграция

test

Эпохи (train)

20

Поставить в очередь

Тип	Статус	Эпохи	Создана
-----	--------	-------	---------

Рисунок 34 – Запуск задачи обучения модели

После постановки задачи в очередь она проходит несколько состояний: *starting*, *pending*, *running*, *done*. В процессе обучения вычисляются значения функции потерь, характеризующие качество модели. Пример графика изменения функции потерь приведён на рисунке 35.

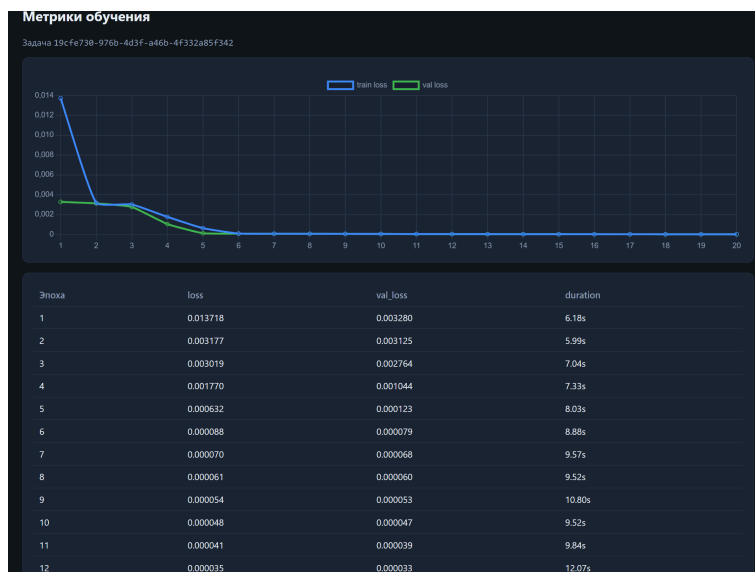


Рисунок 35 – График функции потерь в процессе обучения модели

После завершения обучения веса модели сохраняются во внешнем хранилище, а сама модель переводится в состояние *ready*.

После обучения модели пользователь может запустить задачу обнаружения аномалий. Процесс аналогичен обучению и также реализуется через очередь задач. Пример запуска задачи детектирования представлен на рисунке 36.

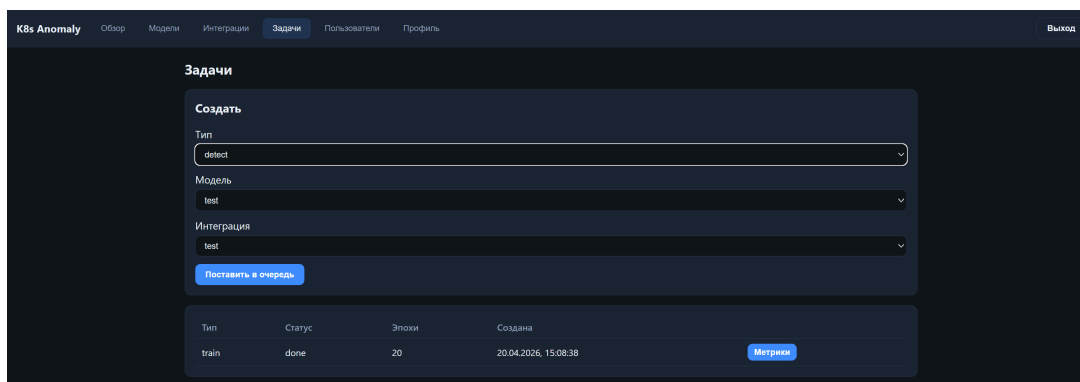


Рисунок 36 – Запуск задачи обнаружения аномалий

Worker-компонент загружает обученную модель, извлекает данные из OpenSearch и выполняет анализ последовательностей событий.

Результатом работы системы являются выявленные аномальные события, которые сохраняются в отдельном индексе OpenSearch, а также агрегированные метрики, записываемые в базу данных. Система предоставляет пользователю информацию о выполненных задачах, включая:

- количество обработанных событий;
- число обнаруженных аномалий;
- временные характеристики выполнения задачи;
- метрики качества модели.

Пример отображения результатов выполнения задачи представлен на рисунке 37.

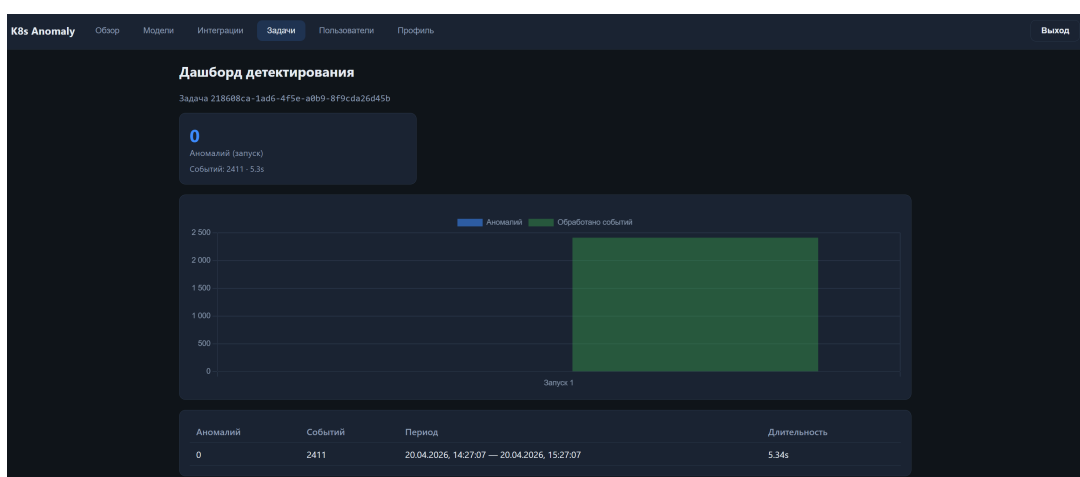


Рисунок 37 – Пример метрик выполнения задачи обнаружения аномалий

3.8 Выводы

В технологической части выпускной квалификационной работы был обоснован выбор программных средств и архитектурных решений, используемых для разработки системы обнаружения аномалий в Kubernetes-кластере. В качестве основного языка программирования выбран Python, обладающий развитой экосистемой библиотек для анализа данных и машинного обучения, а также широкими возможностями для построения распределённых систем.

Для реализации серверной части использован фреймворк FastAPI, обеспечивающий высокую производительность за счёт асинхронной модели обработки запросов и автоматической валидации данных [34]. Применение ASGI-сервера Uvicorn позволило эффективно обрабатывать параллельные запросы и обеспечить масштабируемость системы. Организация взаимодействия с базой данных выполнена с использованием SQLAlchemy и PostgreSQL, что обеспечивает надёжное хранение данных и удобную работу с ними через ORM.

Для выполнения ресурсоёмких операций, таких как обучение моделей и анализ логов, использована система распределённых задач Celery, позволяющая выносить вычисления в отдельные процессы и обеспечивать асинхронность выполнения. Это решение повышает отказоустойчивость и производительность системы при работе с большими объёмами данных.

Реализация модели обнаружения аномалий выполнена с использованием библиотеки PyTorch, которая сочетает удобство разработки и высокую вычислительную эффективность при обучении нейронных сетей [40]. Для предобработки данных и оценки качества моделей применена библиотека scikit-learn.

Разработанное программное средство реализовано в виде многомодульного приложения с разделением на слои, что соответствует принципам чистой архитектуры. Такая организация обеспечивает слабую связанность компонентов, упрощает сопровождение и позволяет масштабировать систему при увеличении нагрузки. Взаимодействие компонентов реализовано по асинхронной модели с использованием очередей задач, что особенно важно при обработке потоков событий в реальном времени.

В результате реализации была построена система, обеспечивающая полный цикл работы с данными: от получения audit-логов из OpenSearch до обучения модели и выявления аномалий. Реализованный подход позволяет эффективно обрабатывать большие объёмы логов и выявлять отклонения в поведении Kubernetes-кластера.

Исходный код разработанного приложения приведён в приложении А. Полный программный код также представлен на электронном носителе.

Таким образом, выбранный стек технологий и архитектурные решения обеспечивают необходимый уровень производительности, масштабируемости и гибкости, а также создают основу для дальнейшего развития системы, включая внедрение более сложных моделей и интеграцию с дополнительными источниками данных.

4 ОРГАНИЗАЦИОННО-ЭКОНОМИЧЕСКАЯ ЧАСТЬ

В настоящее время реализация инженерных проектов в области информационных технологий связана не только с процессами разработки и внедрения программного обеспечения, но и с организационно-экономической составляющей. Важным аспектом любого инженерного проекта является оценка экономической целесообразности разрабатываемого продукта, включающая исследование рынка, проведение финансовых расчётов и анализ полученных результатов.

В рамках организационно-экономической части выпускной квалификационной работы (ВКР) выполняется разработка бизнес-плана реализации программного комплекса обнаружения аномалий в Kubernetes-кластере на основе анализа audit-логов. Данный программный комплекс ориентирован на повышение уровня безопасности и надежности корпоративной инфраструктуры, что обуславливает его актуальность и востребованность на рынке.

Для описания бизнес-плана необходимо выполнить следующие этапы:

- исследование рынка и анализ условий внедрения программного продукта;
- определение этапов выполнения работ по реализации проекта;
- оценка трудоёмкости процессов и определение количества исполнителей;
- определение сроков выполнения работ;
- построение и оптимизация календарных и сетевых графиков проекта;
- определение структуры затрат (зарплата, оборудование, аренда и др.);
- оценка полной стоимости разработки программного средства;
- расчет срока окупаемости проекта;
- вычисление ожидаемой прибыли.

4.1 Анализ рынка и описание условий

В последние годы контейнеризация и оркестрация приложений стали одним из ключевых направлений развития корпоративной ИТ-инфраструктуры. Развитие микросервисной архитектуры и контейнерных подходов сделало Kubernetes одной из базовых платформ для эксплуатации современных приложений. Применение Kubernetes позволяет ускорять развёртывание сервисов, повышать масштабируемость систем и упрощать сопровождение распределённых приложений [19].

Вместе с тем рост числа контейнерных сред приводит к увеличению требований к их защищённости, поскольку контейнерные технологии имеют связанные с безопасностью риски, а уязвимости и ошибки конфигурации могут возникать как на этапе сборки, так и на этапе эксплуатации [44]. Ошибки конфигурации, несанкционированные действия пользователей и аномальная активность внутри кластера могут привести к нарушению доступности сервисов, утечке данных и сбоям в работе инфраструктуры [44].

Одним из наиболее информативных источников данных о действиях в Kubernetes-кластере являются audit-логи, содержащие сведения о запросах к API-серверу, выполнении административных операций и взаимодействии различных субъектов с объектами кластера [3]. Анализ таких журналов позволяет выявлять отклонения от нормального поведения, обнаруживать подозрительные действия и своевременно реагировать на потенциальные инциденты информационной безопасности. В связи с этим разработка программного средства обнаружения аномалий в Kubernetes-кластере на основе анализа audit-логов является актуальной задачей для организаций, использующих контейнерную инфраструктуру в продуктивной среде.

Спрос на подобные решения формируется, прежде всего, со стороны компаний, для которых Kubernetes уже используется или оценивается для внедрения в производственную среду. По данным CNCF Annual Survey 2024, Kubernetes в производственной среде или на стадии внедрения используют 85% респондентов, а четверть участников опроса сообщили, что почти вся их разработка и поставка выполняется с применением cloud native-подходов [45]. Это показывает, что Kubernetes остаётся массовой и развивающейся платформой, а задачи мониторинга, аудита и выявления аномалий сохраняют

практическую актуальность. Для таких организаций особенно важны инструменты, позволяющие не только контролировать состояние кластера, но и выявлять нетипичные действия администраторов, сервисных учётных записей и пользователей, имеющих доступ к API Kubernetes.

В рамках данной выпускной квалификационной работы рассматривается гипотетический заказчик, заинтересованный во внедрении программного комплекса для анализа audit-логов Kubernetes-кластера. Учитывая новизну и сложность разрабатываемого программного средства, а также необходимость интеграции с существующей инфраструктурой заказчика, можно предположить, что общая стоимость проекта составит порядка 4 500 000 рублей. Данная сумма включает затраты на разработку, тестирование, внедрение и первичную настройку системы. Предполагаемый срок реализации проекта составляет 5–6 месяцев, что соответствует типичным срокам разработки программных решений подобного класса.

4.2 Определение этапов работ

Для разработки программного средства обнаружения аномалий в Kubernetes-кластере на основе анализа audit-логов необходимо определить последовательные этапы выполнения работ, позволяющие структурировать процесс технической реализации проекта. В качестве основы для выделения стадий разработки используется ГОСТ 19.102-77 «Стадии разработки» [46], который регламентирует порядок создания программ и программной документации независимо от области их применения.

Стадия «*Техническое задание*» включает следующие этапы:

- 1) обоснование необходимости разработки ПО;
- 2) проведение научно-исследовательской работы;
- 3) разработка и утверждение технического задания.

Данная стадия необходима для постановки целей и задач проекта, анализа предметной области, изучения возможных подходов к обработке audit-логов, определения требований к функциональности и качеству программного средства, а также формирования технического задания на разработку.

Стадия «*Эскизный проект*» состоит из следующих этапов:

- 1) разработка эскизного проекта;
- 2) утверждение эскизного проекта.

На данном этапе уточняются методы решения поставленной задачи, формируется общее описание алгоритмов анализа событий в Kubernetes-кластере, определяется структура входных и выходных данных, а также подготавливается пояснительная записка с описанием проектных решений.

Стадия «*Технический проект*» состоит из следующих этапов:

- 1) разработка технического проекта;
- 2) утверждение технического проекта.

На этом этапе выполняется разработка алгоритмов обработки audit-логов, проектирование структуры программного средства, определение модулей системы, а также планирование последующей реализации и тестирования комплекса.

Стадия «*Рабочий проект*» содержит следующие этапы:

- 1) разработка программы;
- 2) разработка программной документации;
- 3) испытание программы.

На данной стадии осуществляется непосредственная реализация программного комплекса, его отладка, тестирование и корректировка. Параллельно разрабатывается комплект программной документации, включающий описание работы системы, инструкции по развертыванию и сопровождению. В соответствии с результатами прохождения испытаний и приёмо-сдаточных мероприятий производится корректировка работы программы и внесение изменений в документацию.

Стадия «*Внедрение*» заключается в подготовке программного средства к передаче заказчику, его установке в целевой инфраструктуре и организации дальнейшего сопровождения.

Таким образом, в рамках разработки программного средства обнаружения аномалий в Kubernetes-кластере на основе анализа audit-логов можно выделить следующие основные этапы:

- 1) исследовательская часть;
- 2) разработка архитектуры системы;
- 3) разработка и исследование алгоритмов НС;
- 4) анализ и выбор наилучшей модели НС;
- 5) разработка программных модулей;
- 6) интеграция выбранной модели в программный комплекс;

- 7) тестирование и отладка программного средства;
- 8) разработка технической документации.

4.3 Расчёт трудоёмкости и определение этапов работ

В данном разделе производится оценка трудоёмкости работ по разработке программного средства обнаружения аномалий в Kubernetes-кластере на основе анализа audit-логов. Общие трудозатраты по внедрению и разработке Q_p вычисляются по формуле (43).

$$Q_p = \sum_i T_i, \quad (43)$$

где T_i – трудоёмкость i -й работы.

Этапы, описанные ранее, были разбиты на отдельные работы с учётом содержания стадий разработки и особенностей создания программного средства в области информационной безопасности. Трудоёмкость работ была определена методом экспертных оценок, а также с использованием вспомогательных расчётных зависимостей. Значение T_i определяется по формуле (44).

$$T_i = \frac{3T_{i\min} + 2T_{i\max}}{5}, \quad (44)$$

где $T_{i\min}$ и $T_{i\max}$ – минимальная и максимальная экспертные оценки.

За основу оценки трудоёмкости работ этапа рабочего проекта принята трудоёмкость реализации программного средства $T^{\text{прог}}$. По экспертной оценке программное средство обнаружения аномалий в Kubernetes-кластере может быть разработано за время

$$T_{\min}^{\text{прог}} = 60 \text{ чел.-ч.}$$

Трудоёмкость тестирования и коррекции $T_{\min}^{\text{тест}}$ рассчитывается по формуле (45).

$$T_{\min}^{\text{тест}} = (k_{\text{тест}} + k_{\text{корр}}) \cdot T_{\min}^{\text{прог}}, \quad (45)$$

где $k_{\text{тест}}$ – коэффициент тестирования (отношение затрат времени тестирования к полной разработке), $k_{\text{корр}}$ – коэффициент коррекции.

Обычно принимают $k_{\text{тест}} = 0.3$ и $k_{\text{корр}} = 0.3$.

Аналогично производится вычисление трудоёмкости реализации документации $T_{\min}^{\text{док}}$ по формуле (46).

$$T_{\min}^{\text{док}} = k_{\text{док}} \cdot T_{\min}^{\text{прог}}, \quad (46)$$

где $k_{\text{док}}$ – коэффициент реализации документации. Поскольку разрабатываемое средство автоматизации предназначено для мониторинга информационной безопасности и должно быть удобно для последующего сопровождения, примем $k_{\text{док}} = 0.3$.

Перечень работ и соответствующие оценки приведены в таблице 3.

Таблица 3 – Перечень работ

Этап	№ работы	Содержание работы	$T_{i \min}$ (чел.-ч)	$T_{i \max}$ (чел.-ч)	T_i (чел.-ч)	T_i (чел.-дн.)	Исполнитель
1	1	Анализ предметной области и существующих решений	24	34	28	3.5	Аналитик
	2	Формирование требований к системе	14	24	18.0	2.3	Аналитик
2	3	Проектирование общей архитектуры системы	28	58	40	5	Инженер
	4	Проектирование структуры данных	14	28	19.6	2.5	Инженер
3	5	Подготовка и предобработка данных	40	80	56.0	7.0	Аналитик
	6	Разработка моделей НС для обнаружения аномалий	48	96	67.2	8.4	Аналитик
	7	Обучение моделей НС	36	72	50.4	6.3	Аналитик
4	8	Оценка и сравнение моделей	24	34	28	3.5	Аналитик
5	9	Реализация модуля сбора и обработки логов	80	160	112	14.0	Инженер
	10	Реализация модуля анализа и обнаружения аномалий	48	88	64.0	8.0	Инженер
6	11	Интеграция модели НС в систему	30	65	44.0	5.5	Инженер
	12	Настройка взаимодействия компонентов системы	30	65	44.0	5.5	Инженер
7	13	Функциональное тестирование	128	248	176	22	Инженер
	14	Исправление ошибок и оптимизация	80	160	112.0	14.0	Инженер
8	15	Подготовка документации	120	240	168	21	Инженер
Итого Q_p :					1027.2	128.4	

Таким образом, в соответствии с формулой (43) общая трудоёмкость работ составила 1027.2 чел.-ч или 128.4 чел.-дн.

4.4 Линейный график технологического процесса

Для планирования сроков выполнения работ и анализа их взаимосвязей применяется метод сетевого планирования. Он позволяет представить процесс разработки программного средства в виде ориентированного графа, отражающего последовательность выполнения задач и зависимости между ними.

В сетевой модели каждая работа задаётся кодом, включающим номера начального и конечного событий. События фиксируют завершение этапов, а работы описывают переходы между ними. Такой подход позволяет определить критический путь проекта, оценить его длительность и выявить временные резервы.

На основе ранее определённых работ и их трудоёмкости сформированы коды и рассчитаны параметры сетевого графа. Соответствие событий и работ приведено в таблице 5.

Таблица 5 – Соответствие событий и работ

№	Событие	Работа	T (ч)	T (дн)	Код	Тип	Исполнитель
1	Начало работ	Анализ предметной области и существующих решений	28	3.5	1-2	ОН	Аналитик
2	Завершение анализа предметной области	Формирование требований к системе	18	2.3	2-3	ОН	Аналитик
3	Утверждение технического задания	Проектирование общей архитектуры системы	40	5	3-4	ОН	Инженер
		Подготовка и предобработка данных	56.0	7.0	3-6	ОН	Аналитик
4	Проектирование общей архитектуры системы завершено	Проектирование структуры данных	19.6	2.5	4-5	ОН	Инженер
5	Проектирование структуры данных завершено	Реализация модуля сбора и обработки логов	112	14.0	5-9	ОН	Инженер
6	Подготовка и предобработка данных завершена	Разработка моделей НС для обнаружения аномалий	67.2	8.4	6-7	ОН	Аналитик
7	Разработка моделей НС завершена	Обучение моделей НС	50.4	6.3	7-8	ОН	Аналитик
8	Обучение моделей НС завершено	Оценка и сравнение моделей	28	3.5	8-10	ОН	Аналитик
9	Реализация модуля сбора и обработки логов завершена	Реализация модуля анализа и обнаружения аномалий	64.0	8.0	9-10	ОН	Инженер

Продолжение таблицы 5

№	Событие	Работа	T (ч)	T (дн)	Код	Тип	Исполнитель
10	Реализация модуля анализа и обнаружения аномалий завершена	Интеграция модели НС в систему	44.0	5.5	10-11	ОН	Инженер
11	Интеграция модели НС в систему завершена	Настройка взаимодействия компонентов системы	44.0	5.5	11-12	ОН	Инженер
12	Настройка взаимодействия компонентов системы завершена	Функциональное тестирование	176	22	12-13	ОН	Инженер
13	Функциональное тестирование завершено	Исправление ошибок и оптимизация	112.0	14.0	13-14	ОН	Инженер
14	Исправление ошибок и оптимизация завершены	Подготовка документации	168	21	14-15	ОН	Инженер
15	Завершение проекта	-	-	-	-	-	-

Построение модели выполнено в системе ProjectLibre. Исходные данные заданы в соответствии с таблицей 5. Диаграмма Ганта представлена на рисунке 38 и отражает распределение работ во времени.

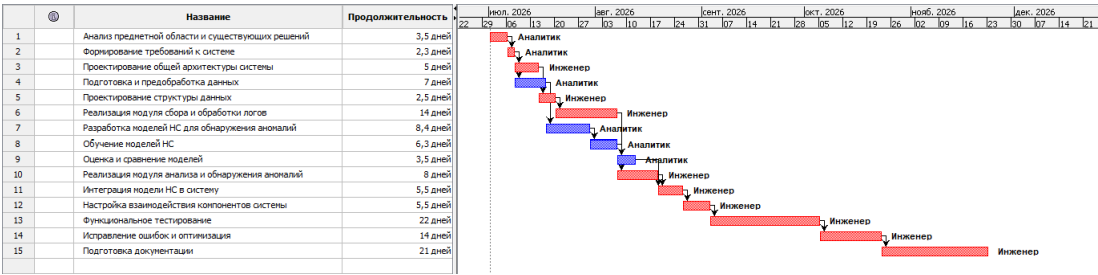


Рисунок 38 – Диаграмма Ганта.

Моделирование позволяет оценить не только календарные сроки, но и потребность в трудовых ресурсах. Сводные трудозатраты по исполнителям приведены в таблице 7.

Таблица 7 – Трудозатраты по исполнителям

Исполнитель	Трудозатраты, часы
Инженер	780
Аналитик	248

4.5 Оптимизация ресурсов

После построения диаграммы Ганта анализируется загрузка персонала для оценки возможности перераспределения ресурсов. Загрузка инженера показана на рисунке 39, аналитика — на рисунке 40.

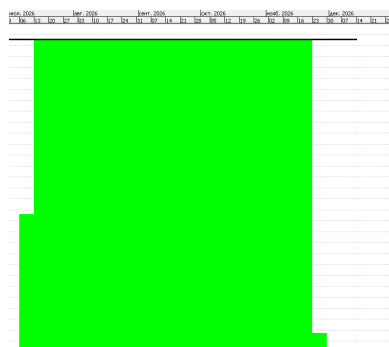


Рисунок 39 – Загрузка инженера по дням

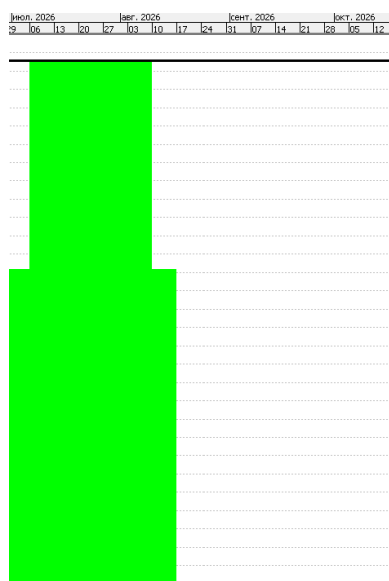


Рисунок 40 – Загрузка аналитика по дням

Анализ показывает, что после 12 августа аналитик не задействован, поскольку его задачи завершены. У инженера наблюдается простой с 1 по 6 июля, обусловленный необходимостью завершения этапа формирования требований. Эти интервалы не подлежат оптимизации в рамках текущей логики процесса. Таким образом, ресурс инженера и аналитика заполнен максимально эффективно в рамках технологического процесса.

4.6 Сетевой график работ

Для оценки предпринимаемых мер по отношению к возможным чрезвычайным ситуациям и их затратам необходимо создать сетевую диаграмму и выявить критический путь. Критический путь представляет собой самый длительный маршрут в сетевой диаграмме, включающий работы, не имеющие запаса по времени.

Задержка хотя бы одной работы на критическом пути увеличивает срок выполнения всего проекта и, как следствие, затраты на его реализацию.

Сетевой график работ проекта представлен на рисунке 41.



Рисунок 41 – Сетевой график работ

Длительность критического пути составляет 102 рабочих дня. В него входят следующие работы:

- 1) анализ предметной области и существующих решений;
- 2) формирование требований к системе;
- 3) проектирование общей архитектуры системы;
- 4) проектирование структуры данных;
- 5) реализация модуля сбора и обработки логов;
- 6) реализация модуля анализа и обнаружения аномалий;
- 7) интеграция модели НС в систему;
- 8) настройка взаимодействия компонентов системы;
- 9) функциональное тестирование;
- 10) исправление ошибок и оптимизация;
- 11) подготовка документации.

4.7 Анализ затрат на изготовление продукта

Затраты на выполнение проекта включают оплату труда исполнителей, расходы на приобретение оборудования, затраты на организацию рабочих мест, а также накладные расходы, связанные с обеспечением процесса разработки. Данный анализ позволяет оценить экономическую целесообразность создания программного продукта и определить основные статьи расходов, оказывающие наибольшее влияние на итоговую стоимость проекта.

Проектные затраты определяются по формуле (47).

$$K = C_{\text{ЗАРП}} + C_{\text{ОБ}} + C_{\text{ОРГ}} + C_{\text{НАКЛ}}, \quad (47)$$

где $C_{\text{ЗАРП}}$ – заработная плата исполнителей;

$C_{\text{ОБ}}$ – затраты на обеспечение необходимым оборудованием;

$C_{\text{ОРГ}}$ – затраты на организацию рабочих мест;

$C_{\text{НАКЛ}}$ – накладные расходы.

4.7.1 Затраты на заработную плату

Затраты на заработную плату исполнителей определяются по формуле (48).

$$C_{\text{ЗАРП}} = C_{\text{ОСН}} + C_{\text{ДОП}} + C_{\text{ОТЧ}}, \quad (48)$$

где $C_{\text{ОСН}}$ – основная заработная плата;

$C_{\text{ДОП}}$ – дополнительная заработная плата;

$C_{\text{ОТЧ}}$ – отчисления с заработной платы.

Расчет основной заработной платы при дневной оплате труда проводится на основе данных по окладам и графику занятости исполнителей. Её расчет выполняется согласно формуле (49).

$$C_{\text{ОСН}} = T_{\text{ЗАН}} \cdot O_{\text{ДН}}, \quad (49)$$

где $T_{\text{ЗАН}}$ – число дней, отработанных исполнителем проекта;

$O_{\text{ДН}}$ – дневной оклад исполнителя.

При восьмичасовом рабочем дне дневной оклад определяется по формуле (50).

$$O_{\text{дн}} = \frac{8 \cdot O_{\text{мес}}}{F_{\text{м}}}, \quad (50)$$

где $O_{\text{мес}}$ – месячный оклад;

$F_{\text{м}}$ – месячный фонд рабочего времени.

Месячный оклад вычисляется с учётом налога на доходы физических лиц по формуле (51).

$$O_{\text{мес}} = O \cdot \left(1 + \frac{N_{\text{НДФЛ}}}{100} \right), \quad (51)$$

где O – чистый оклад;

$N_{\text{НДФЛ}}$ – налог на доходы физических лиц, равный 13% для большинства налогоплательщиков.

Месячный фонд рабочего времени принят на основании производственного календаря на 2026 год. При 40-часовой рабочей неделе годовая норма рабочего времени составляет 1972 часа, следовательно,

$$F_{\text{м}} = \frac{1972}{12} = 164.3 \text{ час./мес.}$$

После составления графика работ определены уровни заработной платы исполнителей в соответствии с формулой (49). Для расчётов использованы данные HeadHunter. Средний чистый месячный оклад инженера принят равным 125000 рублей, а аналитика данных – 104400 рублей. Тогда месячный и дневной оклад для инженера составляют 141250 рублей и 6876.3 рубля, а для аналитика – 117972 рубля и 5743.1 рубля соответственно.

Подробные результаты представлены в таблице 8.

Таблица 8 – Расчёт затрат на заработную плату исполнителей

Исполнитель	$T_{\text{зан}}, \text{ дн.}$	$O, \text{ руб.}$	$O_{\text{мес}}, \text{ руб.}$	$O_{\text{дн}}, \text{ руб./дн.}$	$C_{\text{осн}}, \text{ руб.}$
Инженер	97.5	125000	141250	6876.3	670439.25
Аналитик данных	31.0	104400	117972	5743.1	178036.1

Дополнительно произведено уточнение затрат на заработную плату на основе расчётов в КСП-системе. Объём затрат на основную заработную плату для инженера составил 670439,25 рубля за 780 часов работы, а для аналитика – 178036,1 рубля за 248 часов работы. Тогда суммарная основная заработная плата определяется по формуле (52).

$$C_{\text{осн}} = 670439,25 + 178036,1 = 848475,35 \text{ руб.} \quad (52)$$

Дополнительная заработная плата учитывает выплаты за время, не связанное с производственной деятельностью, но предусмотренное законодательством, в том числе: оплата очередных отпусков, компенсация за недоиспользованный отпуск, и др.. Она определяется по формуле (53).

$$C_{\text{доп}} = 0.2 \cdot C_{\text{осн}}. \quad (53)$$

Тогда дополнительная заработная плата $C_{\text{доп}}$ равна:

$$C_{\text{доп}} = 0.2 \cdot 848475,35 = 169695,07 \text{ руб.}$$

Отчисления с заработной платы состоят в настоящее время в уплате единого социального налога. Согласно налоговому кодексу РФ применяются ставки налога для отчисления в пенсионный фонд РФ, фонд социального страхования, фонды обязательного медицинского страхования и рассчитываются по формуле (54).

$$C_{\text{отч}} = N_{\text{соц}} \cdot (C_{\text{осн}} + C_{\text{доп}}), \quad (54)$$

где $N_{\text{соц}}$ – страховые взносы, составляющие 30% от выплат и включающие:

- пенсионное страхование – 22%;
- социальное страхование – 2,9%;
- медицинское страхование – 5,1%.

Тогда

$$C_{\text{отч}} = 0.3 \cdot (848475,35 + 169695,07) = 305451,13 \text{ руб.}$$

Общие затраты на заработную плату исполнителям составляют:

$$C_{\text{ЗАРП}} = 848475,35 + 169695,07 + 305451,13 = 1323621,55 \text{ руб.}$$

4.7.2 Затраты на материалы и оборудование

Затраты на материалы и оборудование включают расходы на рабочие станции, вычислительный сервер и доступ к сети Интернет. Поскольку в ходе разработки используется преимущественно программное обеспечение с открытым исходным кодом, лицензионные отчисления в состав материальных затрат не включаются.

Общие затраты на материалы и оборудование представлены в таблице 9.

Таблица 9 – Затраты на материалы и оборудование

Наименование	Количество	Стоимость, руб.
Ноутбук HUAWEI MateBook D 16 (i5-12450H, 16GB, 512GB SSD) для инженера-программиста	1	52 824
Ноутбук ASUS VivoBook 15 X1502ZA (i5-1235U, 16GB, 512GB SSD) для аналитика	1	51 940
GPU-сервер с видеокартой NVIDIA RTX 4090 24 ГБ	1	887 217
Доступ к сети Интернет на период разработки	1	9 000
Итого	—	1 000 981

Для выполнения работ по анализу audit-логов и разработке модели обнаружения аномалий требуются два рабочих места: для инженера и для аналитика. В качестве рабочих станций выбраны ноутбуки HUAWEI MateBook D 16 и ASUS VivoBook 15, оснащённые процессорами Intel Core i5, 16 ГБ оперативной памяти и SSD-накопителями объёмом 512 ГБ. Данные характеристики достаточны для разработки, тестирования и предобработки данных.

Для обучения и проверки модели обнаружения аномалий необходим вычислительный сервер с графическим ускорителем NVIDIA RTX 4090 24 ГБ. Для проекта принят GPU-сервер стоимостью 887 217 руб., обеспечивающий достаточную производительность для ресурсоёмких операций машинного обучения.

Расходы на доступ к сети Интернет приняты укрупнённо из расчёта 1 500 руб. в месяц на период шести месяцев разработки, что составляет 9 000 руб.

Таким образом, общая величина затрат на материалы и оборудование определяется по формуле (55).

$$C_{\text{МАТ}} = 52\,824 + 51\,940 + 887\,217 + 9\,000 = 1\,000\,981 \text{ руб.} \quad (55)$$

Затраты на амортизацию оборудования определяются по формуле:

$$C_{\text{АМОРТ}} = \sum_{i=1}^n \left(C_{\text{оби}} \cdot \frac{\Delta t_i}{D_i} \right), \quad (56)$$

где $C_{\text{оби}}$ — стоимость i -го оборудования; Δt_i — время использования оборудования в днях; D_i — срок службы оборудования в днях.

Срок выполнения проекта составляет 6 месяцев (180 дней). Срок службы ноутбуков и сервера принят равным 5 годам (1825 дней).

Тогда затраты на амортизацию составляют:

$$C_{\text{АМОРТ}} = \frac{52\,824 \cdot 180}{1825} + \frac{51\,940 \cdot 180}{1825} + \frac{887\,217 \cdot 180}{1825} = 97\,834,4 \text{ руб.}$$

4.7.3 Затраты на организацию рабочих мест и накладные расходы

Общепроизводственные расходы включают затраты на обслуживание, организацию работ и управление проектом. При организации рабочих мест с ПЭВМ необходимо соблюдать санитарные нормы по площади и объёму помещения, а также по взаимному размещению рабочих мест.

На основе анализа предложений аренды коммерческой недвижимости были рассмотрены несколько вариантов помещений. Для проекта выбран вариант, обеспечивающий размещение двух рабочих мест и принтера при приемлемом уровне затрат.

Таблица 10 – Анализ вариантов аренды помещения для размещения рабочих мест

Адрес помещения	Площадь, кв. м	Аренда в месяц, руб.
Бизнес-парк «Румянцево (Корпус Е)»	25	35 417
Бизнес-парк «Румянцево (Корпус Е)»	83	117 583
Москва, Нижегородская 32А	20	14 287
Москва, Дмитровское шоссе 98с1	23	140 304

По результатам анализа выбран вариант помещения площадью 20 кв. м и стоимостью 14 287 руб. в месяц. При сроке выполнения проекта 6 месяцев затраты на организацию рабочих мест составляют 85 722 руб.

$$C_{\text{ОРГ}} = 14\,287 \cdot 6 = 85\,722 \text{ руб.} \quad (57)$$

Накладные расходы, связанные с выполнением проекта, определяются на основе затрат на основную заработную плату исполнителей. В соответствии с принятой практикой они составляют 60% от основной заработной платы.

Тогда накладные расходы определяются по формуле:

$$C_{\text{НАКЛ}} = 0.6 \cdot C_{\text{ОСН}}$$

Подставляя рассчитанное значение основной заработной платы, получаем:

$$C_{\text{НАКЛ}} = 0.6 \cdot 848475,35 = 509085,21 \text{ руб.}$$

4.7.4 Анализ расходов

По данным таблицы 12, суммарные затраты на реализацию проекта составляют 3 017 244,16 руб. Наибольшую долю в структуре расходов занимает заработная плата исполнителей — 1 323 621,55 руб., что составляет около 43,9% от общей суммы затрат. Это объясняется высокой трудоёмкостью разработки программного средства, включающей анализ предметной области, проектирование архитектуры, реализацию модулей, тестирование и подготовку документации.

Второй по величине статьёй расходов является закупка оборудования. На неё приходится 1 000 981 руб., или около 33,2% от общей стоимости проекта. Значительная доля данной статьи связана с необходимостью приобретения рабочих станций для исполнителей, а также вычислительного сервера, используемого для обучения и проверки модели обнаружения аномалий. Таким образом, материально-техническое обеспечение оказывает существенное влияние на общую стоимость проекта.

Таблица 12 – Суммарные затраты по статьям расходов

№	Статья расходов	Затраты, руб.
1	Заработная плата	1 323 621,55
2	Закупка оборудования	1 000 981
3	Амортизация оборудования	97 834,4
4	Аренда помещения	85 722
5	Накладные расходы	509 085,21
Итого		3 017 244,16

На рисунке 42 представлена диаграмма, отражающая структуру затрат на изготовление продукта.

4.8 Планирование цены и прогнозирование прибыли

На основе рассчитанных суммарных затрат определяется общая стоимость инновационного продукта $K_{по}$, которая вычисляется по формуле 58.

$$K_{по} = (\Delta K + K_{вн}) \cdot (1 + D_{приб}), \quad (58)$$

где ΔK — часть затрат на разработку, приходящаяся на один экземпляр комплекса;



Рисунок 42 – Диаграмма структуры затрат на изготовление продукта

$K_{\text{вн}}$ — затраты на внедрение;

$D_{\text{приб}}$ — процент прибыли, включённый в цену продукта.

Частичная стоимость разработки ΔK вычисляется по формуле 59.

$$\Delta K = \frac{K}{N} \cdot (1 + H_{\text{ст}}), \quad (59)$$

где K — общие затраты на проект;

N — количество внедряемых единиц комплекса за год;

$H_{\text{ст}}$ — ставка кредитования. На начало 2026 года ставка по кредиту для бизнеса в России составляет 16%. Планируется выпустить только 1 экземпляр комплекса. Тогда согласно формуле 59:

$$\Delta K = 3017244 \cdot (1 + 0.16) = 3500003 \text{ руб.}$$

Так как программное обеспечение создаётся по индивидуальному заказу, процент прибыли $D_{\text{приб}}$ принимается равным 0.2. Тогда цена продукта рассчитывается по формуле 58:

$$K_{\text{по}} = 3500003 \cdot (1 + 0.2) = 4200003 \text{ руб.}$$

Стоимость продукта полностью соответствует ожиданиям рынка, предположенных в при анализе существующих решений. Соответственно, прибыль можно определить по формуле 60 с учётом НДС, равным 22 %.

$$C_{\text{приб}} = K_{\text{по}} \cdot D_{\text{приб}} \cdot (1 - H_{\text{НДС}}) = 655200 \text{ руб.} \quad (60)$$

4.9 Выводы

В рамках организационно-экономической части выпускной квалификационной работы выполнено обоснование разработки программного комплекса обнаружения аномалий в Kubernetes-кластере на основе анализа audit-логов. По результатам проведённых расчётов можно сделать следующие выводы:

- Общая трудоёмкость выполнения работ по проекту составляет 1027,2 чел.-ч или 128,4 чел.-дн.;
- Для выполнения всех этапов разработки в установленные сроки достаточно привлечения двух исполнителей: аналитика и инженера;
- Критический путь проекта составляет 102 дня, что определяет минимально возможную длительность выполнения работ без нарушения сроков;
- Основную долю затрат при реализации проекта составляет заработная плата исполнителей, величина которой равна 1 323 621,55 руб.;
- Суммарные затраты на разработку и реализацию программного продукта составляют 3 017 244,16 руб.;
- Расчётная стоимость одного экземпляра программного продукта составляет 4 200 003 руб.;
- Ожидаемая прибыль от реализации одного экземпляра изделия составляет 655 200 руб.;
- Анализ структуры затрат показывает, что наиболее ресурсоёмкими статьями являются оплата труда, приобретение оборудования и накладные расходы;
- Результаты планирования подтверждают экономическую целесообразность разработки программного средства и его востребованность на рынке решений в области информационной безопасности.

В рамках организационно-экономической части также был построен сетевой график выполнения работ, обеспечивающий контроль за соблюдением сроков проекта. Проведённый анализ затрат и ожидаемой прибыли подтверждает экономическую целесообразность разработки: соотношение предполагаемой выручки и вложенных средств обосновывает начало реализации проекта по созданию программного комплекса обнаружения аномалий в Kubernetes-кластере.

5 ОРГАНИЗАЦИОННО-ПРАВОВОЕ ОБЕСПЕЧЕНИЕ ИБ

Выпускная квалификационная работа была выполнена с учётом требований действующего законодательства Российской Федерации, которое регулирует область применения разрабатываемого средства обнаружения аномалий в Kubernetes-кластере на основе нейронных сетей.

Разрабатываемое программное средство предназначено для анализа audit-логов Kubernetes-кластера, формируемых компонентом kube-apiserver, выявления аномального поведения пользователей и service-account, а также передачи соответствующих уведомлений в систему реагирования на инциденты.

Kubernetes-кластер обладает рядом особенностей, которые определяют специфику его правового и организационного регулирования. Audit-логи Kubernetes могут содержать сведения о действиях пользователей, идентификаторы учётных записей, IP-адреса, параметры запросов к API и другие данные, которые в ряде случаев могут быть отнесены к персональным данным или служебной информации ограниченного доступа. В связи с этим необходимо учитывать требования законодательства в области защиты информации, персональных данных, а также обеспечения безопасности критической информационной инфраструктуры.

В рамках данной работы не представляется возможным проанализировать все нормативные правовые акты, поэтому далее рассматриваются наиболее важные из них, имеющие непосредственное отношение к теме исследования:

- Конституция Российской Федерации [47];
- Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации» [48];
- Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» [49];
- Федеральный закон от 26.07.2017 № 187-ФЗ «О безопасности критической информационной инфраструктуры Российской Федерации» [50];
- Уголовный кодекс Российской Федерации [51];
- Постановление Правительства Российской Федерации от 01.11.2012 № 1119 «Об утверждении требований к защите персональных данных при их обработке в информационных системах персональных данных» [52];

– Приказ ФСТЭК России от 18.02.2013 № 21 «Об утверждении Состава и содержания организационных и технических мер по обеспечению безопасности персональных данных при их обработке в информационных системах персональных данных» [53];

– Приказ ФСТЭК России от 11.04.2025 № 117 «Об утверждении Требований о защите информации, содержащейся в государственных информационных системах, иных информационных системах государственных органов, государственных унитарных предприятий, государственных учреждений» [54];

5.1 Конституция Российской Федерации

Конституция Российской Федерации [47] является основным нормативным правовым актом, обладающим высшей юридической силой. Именно её положения формируют базовые принципы обращения с информацией, обеспечением неприкосновенности частной жизни и защитой прав граждан в информационной сфере. В частности, Конституция закрепляет право каждого на неприкосновенность частной жизни, личную и семейную тайну, защиту своей чести и доброго имени, а также устанавливает свободу поиска, получения, передачи, производства и распространения информации любым законным способом.

В контексте обработки audit-логов Kubernetes-кластера особенно актуальны следующие статьи Конституции РФ:

– **Статья 23** — закрепляет право каждого на неприкосновенность частной жизни, личную и семейную тайну, защиту своей чести и доброго имени. Это означает, что любые данные, которые позволяют идентифицировать действия конкретного пользователя или служебной учётной записи, должны обрабатываться с соблюдением принципа конфиденциальности.

– **Статья 24** — устанавливает, что сбор, хранение, использование и распространение информации о частной жизни лица без его согласия не допускаются. В контексте разработки системы обнаружения аномалий это накладывает требования к обезличиванию или псевдонимизации персональных данных, а также к ограничению доступа к audit-логам только уполномоченным сотрудникам.

Таким образом, при проектировании и внедрении системы обнаружения аномалий необходимо учитывать конституционные принципы, обеспечивающие защиту персональной и служебной информации. Любые действия по анализу и обработке логов должны быть законными, прозрачными и документированными, с соблюдением требований по минимизации сбора и обработки личных данных. Реализация программного средства обнаружения аномалий в Kubernetes-кластере должна строиться с учетом следующих конституционных принципов:

- 1) Обеспечение конфиденциальности персональных данных пользователей и служебной информации.
- 2) Законность и прозрачность методов сбора и обработки данных.
- 3) Ограничение доступа к информации только уполномоченным лицам.
- 4) Документирование всех процедур обработки логов с целью соблюдения права граждан на защиту своей частной жизни.

5.2 Федеральный закон № 149-ФЗ «Об информации, информационных технологиях и о защите информации»

Федеральный закон № 149-ФЗ «Об информации, информационных технологиях и о защите информации» [48] является базовым нормативным правовым актом, регулирующим отношения, возникающие при осуществлении права на поиск, получение, передачу, производство и распространение информации, а также при применении информационных технологий и обеспечении защиты информации. Данный закон конкретизирует конституционные положения в сфере обращения с информацией и определяет правовые основы её использования в информационных системах.

В соответствии со статьей 2 Федерального закона № 149-ФЗ под информацией понимаются сведения (сообщения, данные) независимо от формы их представления, а информационная система рассматривается как совокупность содержащейся в базах данных информации и обеспечивающих её обработку информационных технологий и технических средств. Указанные определения имеют непосредственное значение для разрабатываемого программного средства, поскольку Kubernetes-кластер и связанные с ним audit-логи могут рассматриваться как часть информационной системы, в которой осуществляется сбор, хранение и обработка данных о действиях пользователей и сервисных учётных записей.

Статья 5 указанного закона устанавливает, что информация может являться объектом публичных, гражданских и иных правовых отношений, а также может свободно использоваться и передаваться, если федеральными законами не установлены ограничения доступа к ней либо иные требования к порядку её предоставления или распространения. Для рассматриваемой работы это означает, что audit-логи могут использоваться в целях мониторинга и анализа только при соблюдении установленного режима доступа и требований к защите сведений, содержащих ограниченно доступную информацию.

Особое значение для темы исследования имеет статья 16 Федерального закона № 149-ФЗ, согласно которой защита информации представляет собой принятие правовых, организационных и технических мер, направленных на обеспечение защиты информации от неправомерного доступа, уничтожения, модифицирования, блокирования, копирования, предоставления и распространения, а также на соблюдение конфиденциальности информации ограниченного доступа. Следовательно, при разработке системы обнаружения аномалий в Kubernetes-кластере необходимо предусмотреть меры по разграничению доступа к логам, защите каналов передачи данных, контролю целостности хранимой информации и ограничению несанкционированного использования результатов анализа.

Таким образом, Федеральный закон № 149-ФЗ формирует правовую основу для обработки audit-логов Kubernetes-кластера как информации, подлежащей правовому режиму и защите. При проектировании и внедрении разрабатываемого решения должны быть обеспечены законность обработки данных, соблюдение ограничений доступа, а также применение организационных и технических мер защиты информации в соответствии с действующим законодательством Российской Федерации.

5.3 Федеральный закон № 152-ФЗ «О персональных данных»

Федеральный закон № 152-ФЗ «О персональных данных» [49] устанавливает правовые основы обработки персональных данных и направлен на обеспечение защиты прав и свобод человека и гражданина при обработке его персональных данных, включая защиту прав на неприкосновенность частной жизни, личную и семейную тайну. Данный закон развивает положения Конституции Российской Федерации и конкретизирует требования к работе с данными, позволяющими идентифицировать субъекта.

Согласно статье 3 Федерального закона № 152-ФЗ персональные данные представляют собой любую информацию, относящуюся прямо или косвенно к определённому или определяемому физическому лицу (субъекту персональных данных), а обработка персональных данных включает любые действия с ними, в том числе сбор, запись, систематизацию, накопление, хранение, уточнение, использование и передачу. Оператором персональных данных является лицо, самостоятельно или совместно с другими лицами организующее и осуществляющее обработку данных, а также определяющее цели такой обработки.

В контексте разрабатываемого программного средства анализ audit-логов Kubernetes-кластера может затрагивать персональные данные, поскольку такие логи могут содержать идентификаторы пользователей, IP-адреса, временные метки, сведения о действиях в системе и другие атрибуты, позволяющие прямо или косвенно установить связь с конкретным субъектом. В соответствии со статьёй 5 закона обработка персональных данных должна осуществляться на законной и справедливой основе, ограничиваться достижением конкретных, заранее определённых и законных целей, а также не допускать избыточности обрабатываемых данных по отношению к заявленным целям.

Статья 6 Федерального закона № 152-ФЗ определяет условия обработки персональных данных, включая необходимость получения согласия субъекта либо наличие иных законных оснований для обработки. В случае использования системы обнаружения аномалий в рамках обеспечения информационной безопасности допускается обработка персональных данных без согласия субъекта, если это необходимо для достижения целей, предусмотренных законодательством, при условии соблюдения прав и законных интересов субъектов данных.

Особое значение имеет статья 19, устанавливающая обязанность оператора принимать необходимые правовые, организационные и технические меры для защиты персональных данных от неправомерного или случайного доступа, уничтожения, изменения, блокирования, копирования и распространения. К таким мерам относятся, в частности, разграничение прав доступа, использование средств защиты информации, а также контроль и аудит действий пользователей.

Таким образом, при разработке и внедрении системы обнаружения аномалий в Kubernetes-кластере необходимо учитывать требования Федерального закона № 152-ФЗ, включая минимизацию обрабатываемых данных, определение целей обработки, обеспечение законных оснований обработки и реализацию комплекса мер по защите персональных данных. Особое внимание должно уделяться возможности ограничению доступа к audit-логам, что позволяет снизить риски нарушения прав субъектов персональных данных.

5.4 Федеральный закон № 187-ФЗ «О безопасности критической информационной инфраструктуры Российской Федерации»

Федеральный закон № 187-ФЗ «О безопасности критической информационной инфраструктуры Российской Федерации» [50] регулирует отношения, возникающие при обеспечении безопасности значимых объектов критической информационной инфраструктуры (КИИ). Закон распространяется на информационные системы, информационно-телекоммуникационные сети и автоматизированные системы управления, функционирование которых имеет ключевое значение для устойчивого функционирования государства, экономики и обеспечения безопасности граждан.

К субъектам КИИ относятся организации, осуществляющие деятельность в таких сферах, как здравоохранение, транспорт, связь, энергетика, банковская и финансовая сфера, топливно-энергетический комплекс, промышленность и иные отрасли. Для таких объектов устанавливаются специальные требования по обеспечению безопасности, включая проведение категорирования объектов КИИ, выявление актуальных угроз, реализацию мер защиты и организацию процессов реагирования на компьютерные инциденты.

В соответствии с положениями закона субъекты КИИ обязаны:

- проводить категорирование объектов критической информационной инфраструктуры;
- обеспечивать реализацию организационных и технических мер по защите информации;
- осуществлять мониторинг состояния безопасности и выявление компьютерных инцидентов;
- обеспечивать взаимодействие с государственными системами обнаружения, предупреждения и ликвидации последствий компьютерных атак;

– принимать меры по предотвращению, обнаружению и ликвидации последствий компьютерных атак.

Поскольку Kubernetes-кластеры широко используются в современной корпоративной и государственной инфраструктуре, в том числе в организациях, относящихся к субъектам КИИ, данный закон имеет прямое отношение к теме выпускной квалификационной работы. Kubernetes-кластер может являться частью значимого объекта КИИ или обеспечивать функционирование критически важных сервисов.

Разрабатываемое средство обнаружения аномалий на основе анализа audit-логов Kubernetes-кластера может рассматриваться как элемент системы мониторинга безопасности значимого объекта КИИ. Оно позволяет осуществлять сбор и анализ событий, выявлять отклонения в поведении пользователей и сервисных учётных записей, обнаруживать признаки компрометации и несанкционированного доступа.

Таким образом, применение разработанного решения способствует выполнению требований Федерального закона № 187-ФЗ в части мониторинга безопасности, выявления и реагирования на инциденты, а также повышения общего уровня защищённости критической информационной инфраструктуры.

5.5 Уголовный кодекс Российской Федерации

Уголовный кодекс Российской Федерации [51] осуществляет уголовно-правовую защиту информации, информационных систем и отношений, возникающих в сфере их использования, устанавливая ответственность за совершение преступлений в данной области. В контексте обеспечения информационной безопасности особое значение имеют положения главы 28 УК РФ, посвящённой преступлениям в сфере компьютерной информации.

Статьи 272–274, а также 272.1 УК РФ регламентируют ответственность за нарушения, связанные с несанкционированным доступом, неправомерным использованием компьютерной информации и эксплуатацией средств её обработки.

Статья 272.1 УК РФ предусматривает уголовную ответственность за незаконное использование, передачу, сбор или хранение компьютерной информации, содержащей персональные данные, полученные незаконным путём, а также за создание и функционирование ресурсов, предназначенных

для их незаконного распространения. Данная норма направлена на защиту персональных данных и пресечение их нелегального оборота, включая случаи утечек, торговли базами данных и иных форм неправомерного обращения с такой информацией.

Статья 273 УК РФ устанавливает ответственность за создание, использование и распространение вредоносных программ для ЭВМ, предназначенных для несанкционированного уничтожения, блокирования, модификации или копирования информации, а также для нейтрализации средств защиты информации.

Статья 274 УК РФ регламентирует ответственность за нарушение правил эксплуатации средств хранения, обработки или передачи компьютерной информации, а также информационно-телекоммуникационных сетей, если это повлекло уничтожение, блокирование или модификацию информации и причинило существенный вред.

5.6 Постановление Правительства Российской Федерации № 1119

Постановление Правительства Российской Федерации от 01.11.2012 № 1119 «Об утверждении требований к защите персональных данных при их обработке в информационных системах персональных данных» [52] устанавливает требования к обеспечению безопасности персональных данных при их обработке в информационных системах персональных данных (ИСПДн). Данный нормативный акт конкретизирует положения Федерального закона № 152-ФЗ и определяет подходы к выбору и реализации мер защиты информации.

В соответствии с постановлением вводится понятие уровня защищённости персональных данных, который определяется в зависимости от категории обрабатываемых персональных данных, количества субъектов персональных данных, а также актуальных угроз безопасности. Выделяются четыре уровня защищённости, для каждого из которых устанавливается необходимый состав организационных и технических мер по защите информации.

Для разрабатываемого программного средства данный нормативный акт имеет практическое значение в тех случаях, когда система анализа audit-логов Kubernetes-кластера используется в составе ИСПДн или обрабатывает данные, которые могут быть отнесены к персональным. В такой ситуации необходимо определить актуальный уровень защищённости системы и реализовать соответствующий ему набор мер защиты.

К числу таких мер относятся:

- обеспечение контроля доступа в помещения, где размещены информационные системы с ПД, чтобы исключить неконтролируемое проникновение посторонних лиц;
- утверждение руководителем оператора документа, в котором чётко прописано, кто и на каком основании имеет доступ к персональным данным;
- регистрация и учёт событий безопасности, включая действия пользователей и администраторов;
- использование сертифицированных средств защиты информации (СЗИ);

Таким образом, Постановление Правительства РФ № 1119 определяет необходимость формального подхода к оценке защищённости системы и выбору мер защиты персональных данных. При внедрении системы обнаружения аномалий необходимо учитывать уровень защищённости ИСПДн и обеспечивать соответствие реализуемых механизмов требованиям данного нормативного акта.

5.7 Приказ ФСТЭК России № 21

Приказ ФСТЭК России от 18.02.2013 № 21 «Об утверждении Составы и содержания организационных и технических мер по обеспечению безопасности персональных данных при их обработке в информационных системах персональных данных» [53] является одним из ключевых подзаконных актов, конкретизирующих требования к защите персональных данных в информационных системах персональных данных. Данный нормативный документ устанавливает состав мер, необходимых для обеспечения безопасности персональных данных, и определяет подходы к их реализации в зависимости от уровня защищённости информационной системы.

В соответствии с приказом № 21 все меры защиты подразделяются на организационные и технические и охватывают полный жизненный цикл обработки персональных данных. К основным группам мер относятся:

- **Идентификация и аутентификация субъектов доступа** — обеспечение однозначного установления личности пользователей и процессов, имеющих доступ к информационной системе, включая применение механизмов аутентификации и управление учётными записями.

– **Управление доступом** — разграничение прав доступа пользователей и процессов к информационным ресурсам в соответствии с их полномочиями, реализация принципа минимально необходимых привилегий, а также контроль доступа к данным и функциям системы.

– **Ограничение программной среды** — предотвращение запуска несанкционированного программного обеспечения, контроль состава и целостности используемых программных компонентов.

– **Защита машинных носителей информации** — обеспечение контроля доступа к носителям информации, предотвращение несанкционированного копирования и утечки данных.

– **Регистрация событий безопасности** — ведение журналов событий, связанных с доступом к системе и действиями пользователей, включая фиксацию попыток несанкционированного доступа, изменений конфигурации и иных значимых событий.

– **Антивирусная защита** — применение средств защиты от вредоносного программного обеспечения и контроль их актуальности.

– **Обнаружение вторжений** — выявление попыток несанкционированного доступа и иных инцидентов безопасности на основе анализа событий и поведения пользователей.

– **Контроль целостности** — обеспечение неизменности программного обеспечения и данных, включая контроль целостности журналов регистрации событий.

– **Обеспечение доступности** — защита системы от отказов и сбоев, а также обеспечение возможности восстановления работоспособности.

– **Защита среды виртуализации** — обеспечение безопасности виртуальных инфраструктур, включая изоляцию компонентов и контроль взаимодействия между ними.

Перечисленные меры формируют комплексную систему защиты информации и направлены на предотвращение, выявление и реагирование на инциденты информационной безопасности.

Для темы ВКР данный приказ имеет особое значение, поскольку разрабатываемое средство основано на анализе audit-логов Kubernetes-кластера, то есть на сборе, регистрации и последующей обработке событий, связанных с действиями пользователей и сервисных учётных записей. Это напрямую соответствует требованиям по регистрации событий безопасности и обнаружению вторжений, установленным приказом № 21.

В контексте Kubernetes-кластера указанные меры могут быть реализованы следующим образом:

- механизмы идентификации и аутентификации — через средства аутентификации Kubernetes;
- управление доступом — с использованием ролевой модели доступа (RBAC);
- регистрация событий — посредством audit-логирования kube-apiserver;
- обнаружение вторжений — с использованием разработанной системы анализа аномалий;
- контроль целостности — через защиту журналов и централизованные системы хранения логов.

5.8 Приказ ФСТЭК России № 117

Приказ ФСТЭК России от 11.04.2025 № 117 «Об утверждении Требований о защите информации, содержащейся в государственных информационных системах, иных информационных системах государственных органов, государственных унитарных предприятий, государственных учреждений» [54] устанавливает требования к защите информации в государственном секторе и в иных информационных системах, для которых предъявляются повышенные требования к обеспечению безопасности. Данный нормативный акт определяет современный подход к защите информации в информационных системах, где особенно важны устойчивость к инцидентам, контроль доступа и регистрация событий безопасности.

Для разрабатываемого решения приказ № 117 имеет значение в случае применения системы в государственных информационных системах либо в инфраструктуре организаций, находящихся в сфере государственного регулирования. В такой ситуации анализ audit-логов Kubernetes-кластера и автоматическое выявление аномалий должны рассматриваться как часть комплекса мер по защите информации.

Практическая ценность такого подхода заключается в том, что система позволяет повысить уровень наблюдаемости защищаемой среды, своевременно выявлять подозрительные действия, фиксировать нарушения установленного режима доступа и поддерживать процесс реагирования на инциденты. Таким образом, применение разрабатываемого средства соответствует общим требованиям к обеспечению безопасности информации, содержащейся в государственных и иных защищаемых информационных системах.

5.9 Выводы

В данной главе были рассмотрены нормативные правовые акты Российской Федерации, определяющие организационно-правовые основы обеспечения информационной безопасности при разработке и применении средств анализа audit-логов Kubernetes-кластера на основе нейронных сетей.

Установлено, что Конституция Российской Федерации закрепляет базовые права на неприкосновенность частной жизни, защиту персональной информации и законное обращение со сведениями ограниченного доступа. Эти положения формируют общую правовую основу для последующей регламентации обработки информации в информационных системах.

Федеральный закон № 149-ФЗ «Об информации, информационных технологиях и о защите информации» определяет информацию как объект правовых отношений, устанавливает классификацию информации по доступу и закрепляет необходимость её защиты от неправомерного доступа, уничтожения, модифицирования и распространения. Для разрабатываемого программного средства данный закон имеет базовое значение, поскольку audit-логи Kubernetes-кластера представляют собой информацию, требующую соблюдения установленного режима доступа и мер защиты.

Федеральный закон № 152-ФЗ «О персональных данных», а также Постановление Правительства Российской Федерации № 1119 определяют правовые основы обработки персональных данных и требования к обеспечению их безопасности в информационных системах персональных данных. В рамках рассматриваемой темы это особенно важно, поскольку audit-логи могут содержать сведения, позволяющие прямо или косвенно идентифицировать пользователя или сервисную учётную запись. Следовательно, при проектировании системы необходимо учитывать принципы минимизации, законности обработки, разграничения доступа и защиты данных.

Приказ ФСТЭК России № 21 конкретизирует состав организационных и технических мер по защите персональных данных, включая идентификацию и аутентификацию, управление доступом, регистрацию событий безопасности, контроль целостности и обнаружение инцидентов. Указанные требования непосредственно соотносятся с функциональностью разрабатываемого средства, поскольку анализ audit-логов и выявление аномалий являются частью механизма контроля безопасности информационной системы.

Федеральный закон № 187-ФЗ «О безопасности критической информационной инфраструктуры Российской Федерации» показывает значимость рассматриваемого решения для организаций, чьи информационные системы относятся к объектам критической информационной инфраструктуры. В таких условиях система обнаружения аномалий может использоваться как элемент мониторинга и реагирования на компьютерные инциденты, повышающий устойчивость защищаемой инфраструктуры.

Также было установлено, что нормы Уголовного кодекса Российской Федерации, в частности статьи 272–274, устанавливают ответственность за неправомерный доступ к компьютерной информации, создание и использование вредоносных программ, а также нарушение правил эксплуатации средств хранения, обработки и передачи информации. Это подтверждает необходимость применения средств своевременного выявления подозрительной активности и предотвращения инцидентов информационной безопасности.

Таким образом, проведённый анализ нормативных правовых актов показывает, что разработка системы обнаружения аномалий в Kubernetes-кластере на основе анализа audit-логов соответствует действующим требованиям законодательства Российской Федерации в области информации, персональных данных и защиты информации. Рассмотренные правовые нормы определяют необходимость применения организационных и технических мер защиты, обеспечивающих законность обработки данных, ограничение доступа к ним и своевременное выявление инцидентов безопасности.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы была разработана система обнаружения аномалий в Kubernetes-кластере на основе методов нейронных сетей, предназначенная для анализа audit-логов и выявления отклонений от нормального поведения в контейнерной среде.

В исследовательской части были рассмотрены основные подходы к обнаружению аномалий в информационных системах, а также проанализированы особенности обеспечения безопасности в Kubernetes-кластерах. Был выполнен обзор статических методов, алгоритмов машинного обучения и нейросетевых подходов к анализу последовательностей событий. Установлено, что наибольший потенциал для решения поставленной задачи имеют методы глубокого обучения, способные учитывать временную структуру данных и выявлять сложные зависимости в журналируемых событиях. Отдельное внимание было уделено механизму audit-логирования Kubernetes, поскольку audit-логи API-сервера являются наиболее информативным источником данных для обнаружения аномалий в действиях пользователей, сервисных аккаунтов и компонентов кластера.

В конструкторской части была обоснована постановка задачи обнаружения аномалий на основе анализа последовательностей audit-событий. В условиях отсутствия достаточного объёма размеченных данных для всех возможных аномальных сценариев наиболее целесообразным был признан подход на основе автоэнкодеров, позволяющий обучать модель на данных, описывающих нормальное поведение системы. В рамках работы были рассмотрены и сравнены архитектуры AE, RNN-AE и LSTM-AE, а также определены входные данные, критерии обучения и метрики качества. Проведённый анализ показал, что использование рекуррентных нейросетевых моделей, в частности LSTM-автоэнкодера, является наиболее эффективным для выявления аномалий в последовательностях audit-логов. По результатам экспериментов именно модель LSTM-AE продемонстрировала наилучшие значения качества, обеспечив $AUC_{PR} = 0,9996$, $\text{precision} = 0,931$, $\text{recall} = 1,00$ и F1-мера = 0,96.

В технологической части было разработано программное средство, реализующее полный цикл обработки audit-логов: от получения и предобработки данных до обучения модели и применения её для выявления аномалий. При разработке использовались современные программные инструменты и

библиотечные средства, обеспечивающие надёжность, производительность и удобство сопровождения системы. Применение Python в качестве основного языка программирования, фреймворка FastAPI для серверной части, Uvicorn в качестве ASGI-сервера, PostgreSQL и SQLAlchemy для работы с данными, Celery для организации распределённых вычислений, а также PyTorch и scikit-learn для реализации и оценки нейросетевой модели позволило создать масштабируемое и технологически целостное решение. Разработанное приложение обеспечивает автоматизированный анализ журналов аудита и своевременное обнаружение аномальных событий в Kubernetes-кластере.

В организационно-экономической части была выполнена оценка трудоёмкости, затрат на разработку и реализацию программного продукта, а также определена его экономическая целесообразность. Результаты расчётов показали, что разработка проекта требует значительных, но обоснованных ресурсов, а построенный календарный план и оценка затрат подтверждают реалистичность выполнения работ в установленные сроки. Проведённый анализ свидетельствует о том, что разработка данного решения является экономически оправданной и востребованной на рынке средств обеспечения информационной безопасности.

В правовой части был проведён анализ нормативных правовых актов Российской Федерации, регулирующих вопросы информации, персональных данных и защиты информации. Установлено, что разрабатываемое программное средство соответствует требованиям действующего законодательства и может применяться в информационных системах при соблюдении необходимых организационных и технических мер защиты. Это подтверждает правомерность и практическую применимость предложенного решения.

Таким образом, в ходе выполнения выпускной квалификационной работы была достигнута поставленная цель, а именно разработано средство обнаружения аномалий в Kubernetes-кластере на основе методов нейронных сетей, обеспечивающее высокую точность выявления аномальных событий. Полученные результаты подтверждают эффективность выбранного подхода и создают основу для дальнейшего развития системы, расширения функциональности и её интеграции в практические сценарии мониторинга и обеспечения безопасности контейнерных инфраструктур.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microservices: Yesterday, Today, and Tomorrow / N. Dragoni [и др.]. — 2017. — DOI: 10.1007/978-3-319-67425-4_12.
2. Subramanian S. Honnavalli P.B. S. S. Container Security: An Extensive Roadmap. — 2021.
3. Auditing. — URL: <https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/> (дата обращения 02.03.2026).
4. Kubernetes security report. — URL: <https://core247.kz/blog/kubernetes-security-report> (дата обращения 24.09.2025).
5. Varun Chandola Arindam Banerjee V. K. Anomaly Detection: A Survey. — 2009.
6. Roesch M. Snort: Lightweight Intrusion Detection for Networks. — 1999.
7. Kubernetes Security Documentation. — URL: <https://kubernetes.io/docs/concepts/security/> (дата обращения 24.09.2025).
8. Axelsson S. Intrusion Detection Systems: A Survey and Taxonomy. — 2000.
9. Falco documentation. — URL: <https://falco.org/> (дата обращения 28.09.2025).
10. Open Policy Agent documentation. — URL: <https://www.openpolicyagent.org/> (дата обращения 28.09.2025).
11. What is the k-nearest neighbors (KNN) algorithm? — URL: <https://www.geeksforgeeks.org/machine-learning/what-is-isolation-forest/> (дата обращения 30.09.2025).
12. Liu F. T. Ting K. M. Z. Z.-H. Isolation Forest. — 2008.
13. Robust Random Cut Forest Based Anomaly Detection on Streams / S. Guha [и др.]. — 2016.
14. Point-Denoise: Unsupervised outlier detection for 3D point clouds enhancement. — URL: [https://www.researchgate.net/publication/352017898_Point - Denoise _ Unsupervised _ outlier _ detection _ for _ 3D _ point _ clouds _ enhancement](https://www.researchgate.net/publication/352017898_Point_-_Denoise_-_Unsupervised_-_outlier_-_detection_-_for_-_3D_-_point_-_clouds_-_enhancement) (дата обращения 30.09.2025).

15. Elman J. L. Distributed Representations, Simple Recurrent Networks, and Grammatical Structure // Machine Learning. — 1991. — Т. 7, № 2. — С. 195—225. — DOI: 10.1007/BF00114844.
16. RNNs and LSTMs. — URL: <https://web.stanford.edu/~jurafsky/slp3/13.pdf> (дата обращения 04.11.2025).
17. Hochreiter S., Schmidhuber J. Long Short-Term Memory // Neural Computation. — 1997. — Т. 9, № 8. — С. 1735—1780. — DOI: 10.1162/neco.1997.9.8.1735.
18. Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion / P. Vincent [и др.] // Journal of Machine Learning Research. — 2010. — Т. 11, № 110. — С. 3371—3408.
19. Kubernetes Documentation. — URL: <https://kubernetes.io/docs/home/> (дата обращения 02.03.2026).
20. Kubernetes cluster architecture. — URL: <https://kubernetes.io/images/docs/kubernetes-cluster-architecture.svg> (дата обращения 02.03.2026).
21. kube-apiserver | Kubernetes Documentation. — URL: <https://kubernetes.io/docs/reference/generated/kube-apiserver/> (дата обращения 02.03.2026).
22. Sharpening Kubernetes Audit Logs with Context Awareness (K8NTEXT) / M. Franzil [и др.] // arXiv preprint arXiv:2506.16328. — 2025. — URL: <https://arxiv.org/abs/2506.16328>.
23. Anomaly Detection in Microservices Architecture Using Graph Neural Networks / M. Osswald [и др.]. — 2025. — DOI: 10.1109/PDP66500.2025.00085.
24. High Performance Cloud Native Networking (CNI). — URL: <https://cilium.io/use-cases/cni/> (дата обращения 10.10.2025).
25. The Cloud Native Application Proxy. — URL: <https://traefik.io/traefik> (дата обращения 10.10.2025).
26. KubAnomaly: Anomalydetection for the Docker orchestration platform with neural network approaches / C.-W. Tien [и др.]. — 2019.
27. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning / M. Du [и др.]. — 2017. — DOI: 10.1145/3133956.3134015. — URL: <https://doi.org/10.1145/3133956.3134015>.

28. HDFS Log Dataset for Anomaly Detection. — URL: <https://github.com/logpai/loghub/tree/master/HDFS> (дата обращения 18.04.2026).
29. Goodfellow I., Bengio Y., Courville A. Deep Learning. — MIT Press, 2016. — URL: <https://www.deeplearningbook.org/>.
30. Davis J., Goadrich M. The Relationship Between Precision-Recall and ROC Curves. — 2006. — DOI: 10.1145/1143844.1143874. — URL: <https://doi.org/10.1145/1143844.1143874>.
31. Feature Hashing for Large Scale Multitask Learning / K. Weinberger [и др.] // arXiv preprint arXiv:0902.2206. — 2009. — URL: <https://arxiv.org/abs/0902.2206>.
32. Cramér H. Mathematical Methods of Statistics. — Princeton University Press, 1946.
33. Python Documentation. — URL: <https://docs.python.org/> (дата обращения 19.04.2026).
34. FastAPI. — URL: <https://fastapi.tiangolo.com/> (дата обращения 19.04.2026).
35. Uvicorn. — URL: <https://uvicorn.dev/> (дата обращения 19.04.2026).
36. SQLAlchemy Documentation. — URL: <https://docs.sqlalchemy.org/> (дата обращения 19.04.2026).
37. PostgreSQL. — URL: <https://www.postgresql.org/> (дата обращения 19.04.2026).
38. asyncpg Documentation. — URL: <https://magicstack.github.io/asyncpg/> (дата обращения 19.04.2026).
39. Celery - Distributed Task Queue. — URL: <https://docs.celeryq.dev/> (дата обращения 19.04.2026).
40. PyTorch Documentation. — URL: <https://docs.pytorch.org/> (дата обращения 19.04.2026).
41. scikit-learn. — URL: <https://scikit-learn.org/> (дата обращения 19.04.2026).
42. OpenSearch Documentation. — URL: <https://docs.opensearch.org/latest/> (дата обращения 19.04.2026).
43. OpenSearch Python Client Documentation. — URL: <https://opensearch-project.github.io/opensearch-py/> (дата обращения 19.04.2026).

44. Application Container Security Guide. — URL: <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-190.pdf> (дата обращения 28.03.2026).
45. Cloud Native 2024: Approaching a Decade of Code, Cloud, and Change. — 2025. — URL: <https://www.cncf.io/reports/cncf-annual-survey-2024/> (дата обращения 28.03.2026).
46. ГОСТ 19.102-77. ЕСПД. Стадии разработки. — 1977. — URL: <https://www.swrit.ru/doc/espd/19.102-77.pdf> (дата обращения 28.03.2026) ; Государственный стандарт.
47. Конституция Российской Федерации. — URL: <http://kremlin.ru/acts/constitution/item> (дата обращения 08.04.2026).
48. Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации». — URL: <https://base.garant.ru/12148555/> (дата обращения 08.04.2026).
49. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных». — URL: <https://base.garant.ru/12148567/> (дата обращения 08.04.2026).
50. Федеральный закон от 26.07.2017 № 187-ФЗ «О безопасности критической информационной инфраструктуры Российской Федерации». — URL: <https://fstec.ru/dokumenty/vse-dokumenty/zakony/federalnyj-zakon-ot-26-iyulya-2017-g-n-187-fz> (дата обращения 08.04.2026).
51. Уголовный кодекс Российской Федерации. — URL: <https://base.garant.ru/10108000/> (дата обращения 08.04.2026).
52. Постановление Правительства РФ от 01.11.2012 № 1119 «Об утверждении требований к защите персональных данных при их обработке в информационных системах персональных данных». — URL: <https://base.garant.ru/70252506/> (дата обращения 08.04.2026).
53. Приказ ФСТЭК России от 18.02.2013 № 21 «Об утверждении состава и содержания организационных и технических мер по обеспечению безопасности персональных данных при их обработке в информационных системах персональных данных». — URL: <https://fstec.ru/dokumenty/vse-dokumenty/prikazy/prikaz-fstek-rossii-ot-18-fevralya-2013-g-n-21> (дата обращения 08.04.2026).

54. Приказ ФСТЭК России от 11.04.2025 № 117 «Об утверждении требований о защите информации, содержащейся в государственных информационных системах, иных информационных системах государственных органов, государственных унитарных предприятий, государственных учреждений». — URL: <https://fstec.ru/dokumenty/vse-dokumenty/spetsialnye-normativnye-dokumenty/trebovaniya-utverzhdeny-prikazom-fstek-rossii-ot-11-aprelya-2025-g-n-117> (дата обращения 08.04.2026).

ПРИЛОЖЕНИЕ А

Код программы

Код представлен на электронном носителе